



Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ciencia de Datos e Inteligencia Artificial

Datuen Zientzia eta Adimen Artifizialeko gradua

Proyecto fin de grado

Gradu amaierako proiektua

Enhancing SLM reasoning capacities via a custom
GRPO algorithm and semantic-entropy strategies

Endika Blanco Cabana

Bilbao, junio de 2026



Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ciencia de Datos e Inteligencia Artificial

Datuen Zientzia eta Adimen Artifizialeko gradua

Proyecto fin de grado

Gradu amaierako proiektua

Enhancing SLM reasoning capacities via a custom
GRPO algorithm and semantic-entropy strategies

Endika Blanco Cabana

Bilbao, junio de 2026

Abstract

Building upon a custom memory-optimized computing platform, this project adapts advanced reinforcement learning algorithms for mathematical and logical reasoning in SLMs under strict resource constraints. Its main algorithmic focus is GRPO. By using a group-based statistical baseline to estimate advantages, GRPO removes the need for an auxiliary Critic/Value network, keeping policy updates stable while reducing the theoretical memory footprint needed for convergence.

One practical problem in reasoning tasks is length bias: models can improve apparent reward by producing unnecessarily long answers. In 8 GB VRAM environments, long generations increase key-value (KV) cache use and can trigger out-of-memory (OOM) failures. To control this, the project implements mechanisms inspired by Group Relative Policy Optimization Done Right (Dr. GRPO). The objective uses centered group-relative advantages without group standard-deviation scaling, while response length is handled through response caps, truncation handling, diagnostic metrics and an optional reward-side length penalty. These controls push the model toward shorter reasoning paths that fit the hardware constraints instead of rewarding verbosity.

The training pipeline also uses two entropy-driven data strategies to improve sample efficiency. The first is an entropy-based selective backpropagation mask, which blocks gradients for trivial tokens. The second is Semantic-Entropy Curriculum Learning (SENT), which orders training samples from low to high semantic uncertainty. The implementation is mainly in Python with PyTorch, supported by LoRA, GSM8K utilities and selective Triton/CUDA acceleration. The expected result is a validated reinforcement-learning framework for testing how GRPO, length-bias control and uncertainty-aware scheduling affect reasoning in compact models under hardware limits.

Descriptors

GRPO, Small Language Models, Reinforcement Learning, Semantic Entropy, Reasoning Optimization

Index

1. Introduction.....	1
1.1 Context and motivation.....	1
1.1.1 Evolution of LLMs and the need for efficient fine-tuning.....	1
1.1.2 RL as model alignment technique: from PPO to GRPO.....	1
1.1.3 Hardware limitations of consumer GPUs.....	1
1.1.4 Gap between theoretical research and practical implementation.....	2
1.1.5 Personal motivation: bridging the gap between research and real constraints.....	3
1.2 Background and previous situation.....	4
1.3 Technical and economic justification.....	4
1.4 Project objectives.....	6
1.5 Scope and limitations.....	6
1.6 Structure of the memory.....	8
2. Methodology and planning.....	10
2.1 Integrated development methodology.....	10
2.2 Comprehensive validation methodology.....	13
2.3 Global planning.....	15
2.4 Task distribution by work area.....	19
2.5 Budget overview and cost distribution.....	22
3. Theoretical foundations.....	24
3.1 Post-training mechanics in language models.....	24
3.2 Reasoning as an optimization target.....	26
3.3 RLHF, PPO and their limits for reasoning.....	27
3.4 Verifiable rewards and RLVR.....	28
3.5 GRPO foundations.....	30
4. State of the art.....	32
4.1 Reasoning-oriented Reinforcement Learning.....	32
4.2 GRPO variants and objective refinements.....	33
4.3 Reward design and credit assignment.....	36
4.4 Exploration, entropy and curriculum.....	37
4.5 Research gap and project positioning.....	42
5. Data, task and learning setting.....	44
5.1 Base model and learning task.....	44
5.2 Dataset scope and selection criteria.....	45
5.3 Data preparation and prompting.....	46
5.4 Measurement definitions.....	48
6. GRPO implementation and reward design.....	50
6.1 Implemented GRPO objective.....	50
6.2 Reward components.....	52
6.2.1 Numerical correctness and answer extraction.....	52

6.2.2 Length penalty and truncation handling.....	53
6.2.3 Reward composition and sparse-signal risk.....	54
6.3 Selective optimization mechanisms.....	55
6.3.1 Token-level entropy mask.....	55
6.3.2 SENT as example-level control.....	57
6.4 Implementation validation.....	58
7. Training campaign and experimental iterations.....	60
7.1 Baseline and quick-screen stage.....	60
7.2 Finalist selection and repeated-seed runs.....	66
7.3 SENT and stage-based training attempts.....	70
7.4 Dataset pivot and later diagnostic runs.....	75
7.5 Experiment traceability.....	79
8. Results and model evaluation.....	80
8.1 Evaluation package and checkpoint set.....	80
8.2 Training-run evidence behind the evaluated adapters.....	83
8.3 Broad deterministic reasoning evaluation.....	84
8.4 Core sampled reasoning evaluation.....	87
8.5 Prompt-matched 768-token final stress test.....	91
8.6 Qualitative sampled examples.....	94
8.7 Result summary.....	96
9. Statistical interpretation and discussion.....	97
9.1 Evidence hierarchy.....	97
9.2 Why the metrics disagree.....	98
9.3 SENT, entropy and distribution shaping.....	99
9.4 Reward, length, truncation and experimental breadth.....	101
9.5 Defensible claim boundaries.....	103
10. Ethics.....	105
10.1 Bias and fairness.....	105
10.2 Security and misuse.....	106
10.3 Sustainability and accessibility.....	107
10.4 Use of AI assistance and academic integrity.....	107
10.5 Alternatives, decisions and ethical assessment.....	108
11. Conclusions and future work.....	110
11.1 Objective fulfillment review.....	110
11.2 Final conclusions.....	111
11.3 Limitations.....	113
11.4 Future work.....	113
12. Bibliography.....	115

Index of figures

Figure 1.1: Motivation chain of the project.....	3
Figure 2.1: Integrated development methodology.....	11
Figure 2.2: Integrated 600-hour FDP planning baseline.....	18
Figure 2.3: Integrated FDP task network.....	22
Figure 4.1: SENT framework with semantic and token entropy.....	38
Figure 4.2: High-entropy token ranges.....	40
Figure 5.1: Data preparation and evaluation flow.....	47
Figure 6.1: Group-relative advantage flow.....	50
Figure 7.1: Quick-screen terminal reward.....	65
Figure 7.2: Finalist follow-up terminal reward.....	67
Figure 7.3: Endurance held-out accuracy.....	69
Figure 7.4: Collapse response-length trajectories.....	72
Figure 7.5: Collapse selected-token trajectories.....	73
Figure 7.6: Collapse accuracy, reward, and loss inactivity.....	74
Figure 7.7: Open-RS signal failure under the token cap.....	76
Figure 8.1: Final evaluation lenses.....	85
Figure 8.2: Prompt-matched accuracy and length.....	92

Index of tables

Table 1.1: Hardware regime, learning stance, and main exclusions.....	6
Table 1.2: Project boundary.....	8
Table 2.1: Methodological evidence chain.....	13
Table 2.2: Formal 600-hour planning baseline.....	16
Table 2.3: Planned work allocation by activity block.....	18
Table 2.4: Task distribution by work area.....	20
Table 2.5: Human-resource cost distribution.....	23
Table 2.6: Itemized project budget.....	23
Table 3.1: Reward and optimization fit for mathematical reasoning.....	28
Table 4.1: GRPO-family refinement trade-offs.....	35
Table 5.1: Measurement vocabulary for training and evaluation.....	49
Table 6.1: Local GRPO objective design choices.....	52
Table 6.2: Token-level and example-level selectivity.....	55
Table 6.3: Implementation validation coverage.....	59
Table 7.0: Baseline hyperparameter rationale.....	63
Table 7.1: Quick-screen configuration outcomes.....	64
Table 7.2: Finalist follow-up outcomes.....	66
Table 7.3: Endurance selector held-out snapshots.....	69
Table 7.4: Collapse and flat-run diagnostics.....	72
Table 7.5: Corrected dataset admission probe.....	78
Table 8.1: Final evaluation checkpoint set.....	81
Table 8.2: Final benchmark inventory.....	82
Table 8.3: Training and evaluation cost evidence.....	83
Table 8.4: Broad deterministic benchmark by model.....	86
Table 8.5: Deterministic dataset-level review.....	87
Table 8.6: Core sampled benchmark by model.....	88
Table 8.7: Core sampled benchmark by dataset.....	90
Table 8.8: Core sampled bootstrap comparison.....	90
Table 8.9: Practical prompt-regime comparison.....	92
Table 8.10: Prompt-matched 768-token sampled benchmark.....	93
Table 8.11: Prompt-matched dataset-level result.....	94
Table 8.12: Qualitative sampled behavior patterns.....	95
Table 9.1: Evidence hierarchy for final interpretation.....	98
Table 9.2: Metric meanings behind the mixed results.....	99
Table 9.3: Mechanism evidence and boundaries.....	101
Table 9.4: Statistical variables behind reward interpretation.....	102
Table 9.5: Claim boundaries for final interpretation.....	104
Table 10.1: Ethical alternatives, decisions and justification.....	109
Table 11.1: Objective closure summary.....	111

1. INTRODUCTION

1.1 CONTEXT AND MOTIVATION

1.1.1 Evolution of LLMs and the need for efficient fine-tuning

Large language models have moved from general-purpose text generation toward tasks that require more structured reasoning, such as mathematics, programming, and step-by-step problem solving. This shift made post-training more important, because the usefulness of a model increasingly depends on how it is adapted after pretraining and how reliably it can follow a reasoning process.

The project started from that technical shift, but also from a practical constraint. Reasoning-oriented work such as DeepSeek-R1 made reinforcement learning for reasoning concrete enough to study seriously [1], but reproducing that kind of training directly is not realistic in a small local environment. The relevant question is narrower and more useful: how much of a verifier-based reasoning post-training pipeline can be implemented, measured, and interpreted when the available hardware, response budget, and rerun budget are all limited.

1.1.2 RL as model alignment technique: from PPO to GRPO

Reinforcement learning (RL) is relevant because it optimizes model behavior from feedback about generated outputs, not only from examples of desired text. In mathematical reasoning, that feedback can often be tied to verifiable outcomes: the final answer can be checked, the response format can be inspected, and the reward signal can be computed without training a separate human-preference model.

The traditional Reinforcement Learning from Human Feedback (RLHF) path usually uses Proximal Policy Optimization (PPO) with a learned value model to estimate advantages [2] [3]. That design is powerful, but it adds an extra model component and a difficult estimation problem for long reasoning traces. Group Relative Policy Optimization (GRPO) is a better fit for this project because it replaces the learned critic with comparisons inside groups of sampled responses, a group-relative formulation introduced in DeepSeekMath [4]. The method keeps the central idea of policy optimization while reducing the machinery needed to turn verifiable rewards into an update. For a compact local study, that matters: the policy objective must remain analyzable, implementable, and compatible with the available compute.

1.1.3 Hardware limitations of consumer GPUs

The project is defined by a hard hardware constraint: a single NVIDIA RTX 3060 Ti graphics processing unit (GPU) with 8 GB of video random-access memory (VRAM). That limit shapes the experimental

design, because model size, rollout group size, response length, checkpoint frequency, and rerun count all affect which conclusions can be defended.

Consumer GPUs are attractive because they are accessible, but their memory ceiling is unforgiving. Parameter-efficient fine-tuning reduces the number of trainable parameters, yet it remains sensitive to sequence length, batch size, optimizer state, and precision choices on this class of hardware [5] [6]. Reinforcement learning adds further pressure because each training step may involve generation, reward computation, stored log-probabilities, activations, and repeated policy updates. As a result, the learning problem cannot be separated from the resource regime in which it is studied.

1.1.4 Gap between theoretical research and practical implementation

There is a clear distance between published reasoning-RL recipes and what can be studied on a single local GPU. Large-scale systems can use longer responses, larger rollout groups, more seeds, and broader evaluation suites. A local project cannot simply reproduce those settings at smaller scale without changing the meaning of the experiment. The smaller setting changes the signal-to-noise ratio of rewards, the chance of zero-advantage groups, the effect of truncation, and the amount of statistical evidence available after training.

This gap is methodological as much as technical. Running a training loop and reporting a final score would leave the central questions unanswered. The experiment must explain whether the reward signal is informative, whether the grouped objective produces usable contrast, whether response length or truncation is distorting the result, and whether auxiliary controls such as entropy masking or curriculum ordering improve training behavior. The project therefore studies a compact verifier-based GRPO pipeline as an experimental object, not as a claim to reproduce frontier-scale reasoning training.

Figure 1.1 summarizes the conceptual constraint chain behind that question. The arrows do not describe the runtime order of the system; they show why the project moves from reasoning-model post-training to memory pressure, then to a local hardware constraint, and finally to an inspectable implementation strategy.

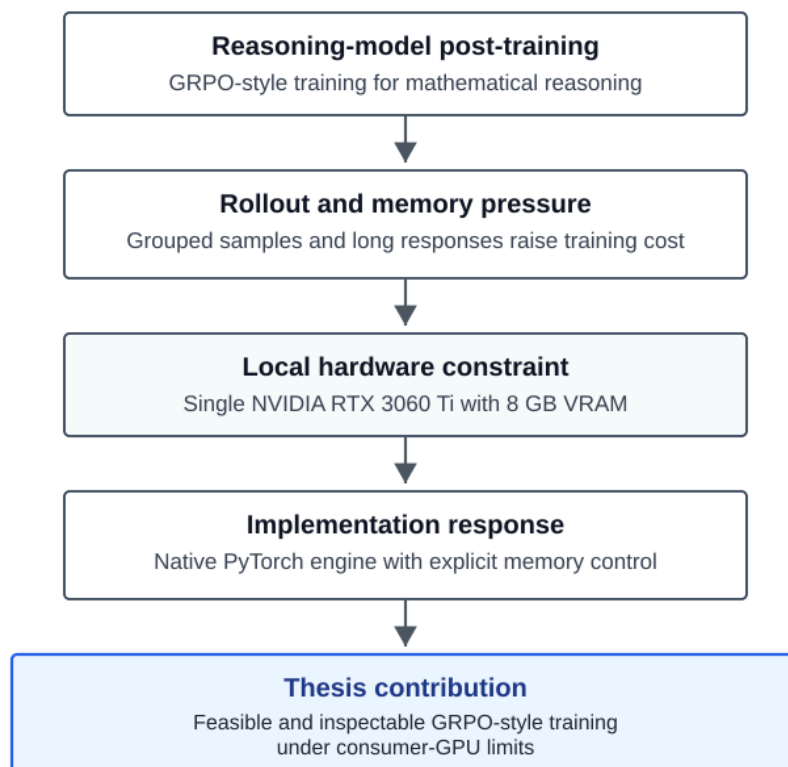


Figure 1.1: Motivation chain of the project.

1.1.5 Personal motivation: bridging the gap between research and real constraints

The personal motivation was to move beyond using reasoning models as black boxes. I wanted to understand how reinforcement-learning post-training behaves when the policy, reward, data path, and evaluation loop are built directly and then forced to run inside real limits. That meant accepting a smaller scale, but it also made the work easier to inspect: reward spikes, flat runs, unstable responses, truncation, or failed curriculum stages had to be explained through code, metrics, and preserved artifacts.

The value of the project lies in that transparency. A constrained experiment cannot prove universal reasoning improvement, but it can show how a compact reasoning model reacts to a concrete GRPO-family objective, a deterministic mathematical verifier, entropy-based selectivity, and curriculum ordering. The goal is a controlled local study whose limitations are visible rather than hidden behind a larger training stack.

1.2 BACKGROUND AND PREVIOUS SITUATION

Before this project began, the tooling available for compact post-training already covered much of the basic workflow. Hugging Face Trainer and Transformer Reinforcement Learning (TRL) provided high-level abstractions for supervised and reinforcement-learning style training. Parameter-Efficient Fine-Tuning (PEFT) simplified Low-Rank Adaptation (LoRA) integration. bitsandbytes made low-precision loading practical. Unsloth also provided practical reference material on GRPO, LoRA, Quantized Low-Rank Adaptation (QLoRA), and low-VRAM reasoning-model workflows [7] [8]. Those references showed that compact reasoning post-training was feasible, but still tied to framework-specific abstractions.

The unresolved question was how those tools and design choices behave together under a small local training budget. A verifier-based GRPO run depends on generation, reward computation, old-policy or frozen-policy scoring, grouped advantage calculation, adapter updates, checkpointing, and evaluation. Each step can be described cleanly in isolation, while the full training process remains sensitive to response length, truncation, reward sparsity, rollout-group composition, and GPU memory pressure.

Three limitations defined the previous situation:

- **Abstraction and control overhead.** High-level training stacks can make a run easy to launch while hiding the exact scoring passes, tensor lifetime, adapter state, and retry behavior that determine whether the learning process remains interpretable.
- **Sparse and fragile rewards.** Final-answer verification is transparent, but it gives little information about intermediate reasoning steps. Group-relative methods can also produce weak learning signals when all sampled responses receive the same reward.
- **Constrained experimental evidence.** Response length, rollout size, truncation, rerun scarcity, and GPU memory pressure all affect the interpretation of results. A configuration that runs successfully is not necessarily a configuration that improves reasoning in a defensible way.

These limitations point to two connected gaps. The first is an observability gap: the training loop must remain visible enough to run reliably near the hardware boundary. The second is an algorithmic and evaluative gap: the project must determine whether GRPO-style training, reward shaping, entropy masking, and curriculum ordering produce measurable and interpretable changes in reasoning behavior under that constraint.

1.3 TECHNICAL AND ECONOMIC JUSTIFICATION

Given those conditions, a native PyTorch implementation is justified by both control and interpretability. The training loop must expose the stages that determine the learning signal:

response generation, reward computation, old-policy or frozen-policy scoring, grouped advantage calculation, adapter update, checkpointing, and evaluation. Keeping those stages visible makes it possible to connect an observed training behavior to the mechanism that produced it. This is especially important for a project whose main claims depend on reward dynamics, response length, truncation, format compliance, and held-out accuracy rather than on final reward alone.

There is also an economic argument behind the same decision. Frontier reasoning-model training is normally associated with long runs, cluster-scale infrastructure, and costs that place systematic experimentation outside the reach of most individual students or small teams. Compact-model work such as Tina shows that meaningful reasoning-oriented post-training can still be studied at the 1.5B-parameter scale when the model, adapter strategy, reward path, and evaluation protocol are controlled [9]. Optimizing for a local RTX 3060 Ti lowers direct financial cost and dependence on rented infrastructure, which matters in a project built around repeated modification, measurement, and interpretation.

The accessibility argument follows from the same constraint. If reasoning-model post-training can only be explored by teams with large compute budgets, then the range of testable ideas is narrowed in advance. A local implementation cannot match datacenter-scale training, but it can make the mechanics of verifier-based post-training visible, reproducible, and easier to inspect. That is the contribution pursued here: a constrained but transparent experimental setting for studying how a compact reasoning model behaves under a GRPO-family objective.

Table 1.1 condenses the hardware regime, learning stance, and main exclusions that shape the rest of the document.

Aspect	Project decision
Target hardware	Single NVIDIA RTX 3060 Ti with 8 GB VRAM
Base model family	DeepSeek-R1-Distill-Qwen-1.5B
Learning objective	GRPO-family post-training for verifier-compatible mathematical reasoning
Feasibility strategy	4-bit NormalFloat 4 (NF4) backbone, LoRA adaptation, bounded rollout groups, explicit response budget
Main learning priorities	reward design, grouped advantages, response length, entropy masking, curriculum behavior, held-out evaluation

Core operating assumption	Single-GPU local execution with limited rerun budget
Out of scope	frontier-scale training, broad multi-domain claims, learned reward models, process-level supervision, distributed execution

Table 1.1: Hardware regime, learning stance, and main exclusions.

1.4 PROJECT OBJECTIVES

The work is organized around five objectives:

- **O1. Implement a native GRPO-family post-training pipeline for compact reasoning models.** The main objective is to build a working pipeline that can sample groups of mathematical responses, evaluate them with deterministic rewards, compute group-relative advantages, update LoRA parameters, and preserve enough traceability to analyze the resulting behavior.
- **O2. Analyze the selected policy-optimization formulation.** The project must explain the transition from PPO-style actor-critic training to GRPO-style grouped comparison, justify the centered group-relative objective used locally, and distinguish the implemented formulation from broader variants proposed in the literature.
- **O3. Improve mathematical reasoning under consumer-hardware constraints.** The project must seek measurable gains in verifier-compatible reasoning while keeping the training regime compatible with a single 8 GB GPU. This objective includes controlling response length, reducing harmful truncation, avoiding unbounded verbosity, and checking whether shorter or better-structured responses can preserve or improve final-answer correctness.
- **O4. Validate reasoning behavior with task-aligned metrics.** Execution alone is not enough. The training process must be evaluated through final-answer correctness, format compliance, reward evolution, response length, truncation behavior, held-out accuracy, and stability across comparable runs.
- **O5. Study selective controls under constrained evidence.** The project must evaluate whether reward shaping, entropy masking, and curriculum ordering provide useful control over the learning process, while keeping conclusions bounded by the available datasets, response cap, hardware budget, and rerun count.

1.5 SCOPE AND LIMITATIONS

The scope includes the algorithmic and evaluative components that define the learning pipeline: the GRPO-family objective, centered group-relative advantage computation, the rule-based reward and

verifier path for mathematical final-answer tasks, response-format handling, entropy masking during optimization, optional Semantic-Entropy Curriculum Learning (SENT) support, and the evaluation and interpretation of the resulting metrics.

Several lines of work remain outside the current scope. The project does not attempt open-ended multi-domain training, process-reward supervision, a separate learned reward model, or a broad benchmark claim across all reasoning domains. It also excludes a persistent reference-model-heavy regularization setup in the main training preset because that would increase memory pressure beyond what the target environment can comfortably support.

The analysis is inseparable from the hardware setting in which the training runs occur. Group size, response length, the presence or absence of reference-model terms, and the overall ambition of the experimental plan are all bounded by the memory and wall-clock cost of a single consumer GPU. The hardware constraint is therefore part of the experimental condition under which the learning results must be interpreted.

The final limitation concerns experimental breadth. Reinforcement-learning training is slow, sensitive to configuration, and expensive in wall-clock time even for compact models. Because the project prioritizes a functioning local pipeline and defensible benchmark-backed conclusions, the number of long runs, seed repetitions, and broader transfer evaluations remains limited. Stronger multi-seed significance and broader cross-dataset validation during training remain outside the current scope.

Table 1.2 summarizes the project boundary.

Axis	This project studies	Explicit boundary
Base model	DeepSeek-R1-Distill-Qwen-1.5B with adapter-based reinforcement-learning post-training	No claim that this is the globally best reasoning model
Main task	Verifier-compatible mathematical reasoning with deterministic answer verification	No broad multi-domain or transfer claim
RL formulation	GRPO-family training with centered group-relative advantages	No strict reproduction of every DeepSeek-R1 or Dr. GRPO component

Reward path	Rule-based numerical correctness plus answer-format structure	No learned reward model and no process-level supervision
Auxiliary controls	Entropy masking and optional SENT curriculum support	Bounded controls, not proven universal improvements
Hardware condition	Single RTX 3060 Ti with 8 GB VRAM	Results must be interpreted under this local constraint, not as datacenter-scale evidence

Table 1.2: Project boundary.

1.6 STRUCTURE OF THE MEMORY

The document is organized as follows:

- Chapter 1 introduces the project context, motivation, technical justification, objectives, scope, limitations, and document structure.
- Chapter 2 presents the methodology, planning, validation approach, workload distribution, and budget baseline.
- Chapter 3 develops the theoretical foundations needed to understand the learning problem, including post-training mechanics, reasoning as an optimization target, RLHF, PPO, verifiable rewards, and GRPO.
- Chapter 4 reviews the state of the art in reasoning-oriented reinforcement learning, GRPO variants, reward design, credit assignment, entropy, curriculum learning, and the research gap addressed by the project.
- Chapter 5 defines the data, task, base model, dataset criteria, preprocessing path, prompting strategy, and measurement definitions.
- Chapter 6 explains the implemented GRPO objective, reward components, selective optimization mechanisms, and implementation validation.
- Chapter 7 describes the training campaign, including baseline screening, finalist selection, repeated-seed runs, SENT attempts, dataset pivots, and experiment traceability.
- Chapter 8 presents the model-evaluation results, quantitative comparisons, training dynamics, qualitative errors, and result summary.
- Chapter 9 interprets the evidence statistically and methodologically, with attention to reliability, sparse rewards, length, truncation, cross-metric reading, and claim boundaries.
- Chapter 10 discusses ethics, sustainability, responsible use, academic integrity, and the social implications of local reasoning-model training.

- Chapter 11 summarizes objective fulfillment, final conclusions, limitations, and future work.
- Chapter 12 lists the bibliography used throughout the memory.

2. METHODOLOGY AND PLANNING

2.1 INTEGRATED DEVELOPMENT METHODOLOGY

The project was developed through an integrated methodology that combined software-engineering iteration with a data-science workflow oriented to reinforcement learning experimentation. This structure followed directly from the nature of the work. The system had to be implemented as a native PyTorch training engine with explicit control over quantization, LoRA injection, memory cleanup, checkpointing, and OOM recovery, but the same codebase also had to support an experimental process centered on verifiable mathematical reasoning tasks, GRPO-style optimization, deterministic reward verification, curriculum learning, and benchmark-backed validation. Grade School Math 8K (GSM8K) served as the initial reference task, while later work generalized the data path to broader math datasets.

For that reason, the project did not follow a linear waterfall sequence. It advanced through short iterative cycles in which a hypothesis or requirement was implemented, tested under the target hardware constraint, measured, and then refined. In practice, many tasks returned repeatedly to earlier stages. A change in the training loop could require new checks in the GRPO implementation, new profiler runs to inspect VRAM behavior, and new benchmark comparisons to determine whether the modification represented a real improvement or only a local optimization with hidden regressions. The practical reference on the engineering side was an Agile style of work based on incremental implementation, fast feedback, and continuous correction. On the experimental side, the workflow followed the logic of the Cross-Industry Standard Process for Data Mining (CRISP-DM): problem framing, task and data definition, iterative modeling, bounded evaluation, and interpretation of results. These stages were not isolated. They overlapped throughout development.

Figure 2.1 summarizes this integrated process. The diagram shows how the Agile engineering cycle and the CRISP-DM data-science cycle were joined in practice: shared framing led to an engineering increment, a data-science modeling step, integrated validation, joint interpretation, and a decision point that either triggered a new iteration or consolidated the next methodological baseline.

Integrated Development Methodology

How Agile engineering iteration and CRISP-DM data-science workflow were combined in one development cycle

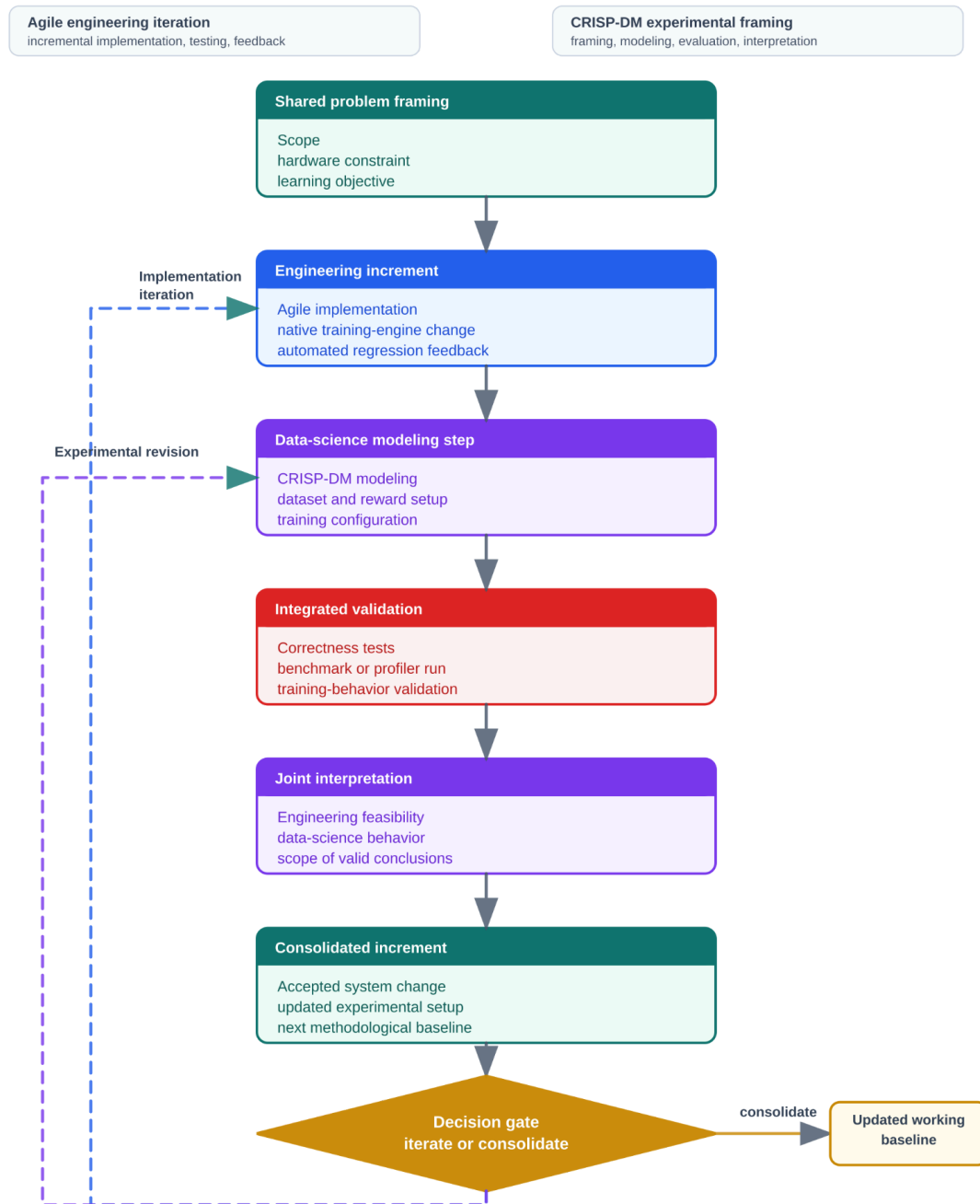


Figure 2.1: Integrated development methodology.

Version control was treated as a methodological tool rather than a simple backup mechanism. Development was organized around Git and GitHub with a branch-based workflow centered on main, develop, and feature branches. This structure mattered because the training engine contained

low-level changes that could introduce subtle regressions in memory behavior, generation logic, or optimizer-state handling, and because experimental work often required temporary branches for trying kernels, runtime policies, or benchmarking workflows before promoting them to the main development line. The project log history preserved a clear sequence of implementation phases, from the initial native GRPO engine through Semantic-Entropy Curriculum Learning (SENT) integration, memory-safety improvements, profiler development, Triton experimentation, packing and checkpointing changes, optimizer automation, benchmark-suite consolidation, and final preset selection. That sequence supported both software maintenance and academic traceability, because each implementation phase could be tied back to a concrete state of the codebase.

Experimental tracking also relied on several complementary mechanisms. During training, the system generated structured logs for runtime metrics and debugging. When enabled, Weights & Biases preserved convergence-related signals and run metadata. In parallel, the benchmark workflow produced repeatable command-line suites with machine-readable outputs such as JSON, CSV, and JSONL records. Configuration choices, benchmark outputs, intermediate reports, and final LoRA weights were all treated as reproducible artifacts tied to a concrete code state. That discipline reduced the risk of relying on isolated console outputs or handwritten observations when comparing runs.

The same methodological rigor was used in testing and validation. The project adopted a correctness-first philosophy because failures in a low-level reinforcement-learning training engine are expensive to diagnose and easy to misinterpret. An apparently minor change in generation, entropy masking, gradient accumulation, or checkpointing can alter the whole learning process or make a benchmark incomparable with earlier runs. For that reason, the automated test suite was organized around several layers:

- Unit tests for mathematically sensitive or structurally delicate components, including the GRPO loss, clipping behavior, reward handling, memory management, and data preprocessing.
- Integration tests for the trainer loop, checkpoint-resume behavior, Kullback-Leibler (KL) and reference passes when applicable, SENT stage transitions, and OOM retry handling.
- Specialized tests for Triton correctness, profiler behavior, benchmark logic, optimizer promotion gates, and hardware-sensitive fallbacks.
- Direct numerical checks for the mathematical behavior of the GRPO objective, so that the loss was verified through reproducible calculations rather than informal inspection alone.

Within that framework, the broader trainer behavior was validated through reproducible integration scenarios before the corresponding runs were interpreted as evidence.

Table 2.1 makes the methodological evidence chain explicit.

Method layer	Concrete project mechanism	What it validates
Iterative implementation	branch-based Git workflow on main, develop, and feature branches	that algorithm and systems changes remained attributable and reversible
Correctness-first testing	unit, integration, verifier, benchmark, profiler, and SENT tests	that sensitive mathematical and pipeline contracts were checked before interpreting runs
Experimental tracking	structured runtime logs plus metrics.jsonl, config.json, and benchmark reports	that comparisons relied on preserved machine-readable artifacts rather than console impressions
Benchmark authority	bounded suites with fixed trainer-step budgets and saved outputs	that shortlist and finalist decisions came from comparable runs
Statistical caution	repeated-seed finalist validation and explicit claim boundaries	that methodological rigor remains separate from overclaiming

Table 2.1: Methodological evidence chain.

2.2 COMPREHENSIVE VALIDATION METHODOLOGY

Because the project was constrained to a single consumer GPU, validation began with feasibility rather than with final accuracy alone. The first question was whether the complete reinforcement-learning pipeline could run, remain stable, and preserve enough headroom to avoid frequent catastrophic failures. The methodology therefore tracked several validation dimensions:

- Runtime performance, especially throughput and step time.
- Response behavior, including response length and truncation rates.

- Memory behavior, including peak VRAM, average VRAM, allocator pressure, and phase-level hotspots.
- Runtime stability, including OOM-recovery events and behavior under longer diagnostic runs.
- Benchmark comparability, using standardized outputs across configurations.

Runtime logging recorded the operational metrics of the trainer, benchmark suites produced standardized outputs, and a dedicated profiling stack inspected memory hotspots and phase-level behavior. Under an 8 GB budget, performance and stability could not be separated. A configuration that looked faster in a short window but produced frequent truncation or OOM backoff in a longer diagnostic run could not be accepted as a reliable operating point.

System validation followed an evidence hierarchy:

1. The relevant regression tests had to pass.
2. The modified system had to survive the benchmark workflow without breaking comparability conditions.
3. The resulting metrics had to show a measurable advantage in throughput, step time, or stability.

This prevented the project from treating isolated profiler gains as sufficient proof of improvement. The benchmark methodology served as the bridge between implementation work and experimental conclusions. Rather than comparing configurations informally, the project used repeatable suites and control scripts to standardize inputs, consolidate metrics, and determine whether a candidate configuration could replace the current baseline.

Model evaluation initially used GSM8K because it offered a compact, verifiable reasoning task aligned with the scope of the project. The same evaluation contract was then kept for the broader normalized math-dataset path through three checks:

- Final-answer correctness, verified through deterministic comparison with the target answer.
- Compliance with the expected response format, so that evaluation checked more than whether the right number appeared somewhere in the response.
- Reward and training dynamics across runs, including variance, outliers, instability, and temporal evolution.

The rule-based verifier fit the hardware constraint because it avoided the need for a separate learned reward model while preserving clear reward semantics. Format compliance still mattered because the generated answer had to respect the structure required by the task and the verifier. Reward behavior also required separate analysis, since sparse binary rewards and group-relative comparisons can generate unstable dynamics even when the implementation itself is technically correct.

2.3 GLOBAL PLANNING

The project was planned around a global workload of 600 hours, divided into 300 hours for Computer Engineering work and 300 hours for Data Science work. This baseline served as the reference framework for the schedule, the task distribution, and the budget model. It does not imply that the work happened in a perfectly sequential order. In practice, implementation, debugging, benchmarking, and writing overlapped repeatedly.

For planning purposes, the work was organized into four internal phases that describe the project clearly from a technical point of view: foundation, core implementation, optimization and validation, and documentation and review. The foundation phase covered problem definition, scope delimitation, theoretical orientation, feasibility analysis, and setup of the development environment. The core implementation phase concentrated most of the engineering effort: native training-loop construction, GRPO and reward logic, memory-management mechanisms, benchmarking infrastructure, and the main feature additions. The optimization and validation phase focused on profiler analysis, Triton integration, benchmark comparison, and methodological checks. Finally, the documentation and review phase addressed the preparation of the two memory documents, final consolidation of results, and defense preparation.

The formal baseline is summarized in Table 2.2.

Internal phase	Planning block	Weeks	Computer Engineering hours	Data Science hours	Total hours
Foundation	Definition and initial planning	1-4	70	70	140
Core implementation	Development	5-10	170	110	280
Optimization and validation	Evaluation and consolidation	11-14	40	80	120
Documentation and review	Final review and writing	15-16	20	40	60

Total	Full project plan	1-16	300	300	600
--------------	--------------------------	-------------	------------	------------	------------

Table 2.2: Formal 600-hour planning baseline.

The project schedule is summarized in Figure 2.2. The Gantt chart expands the formal 600-hour planning baseline into the concrete work packages used by both FDPs, while preserving the five-stage structure expected for the project plan.

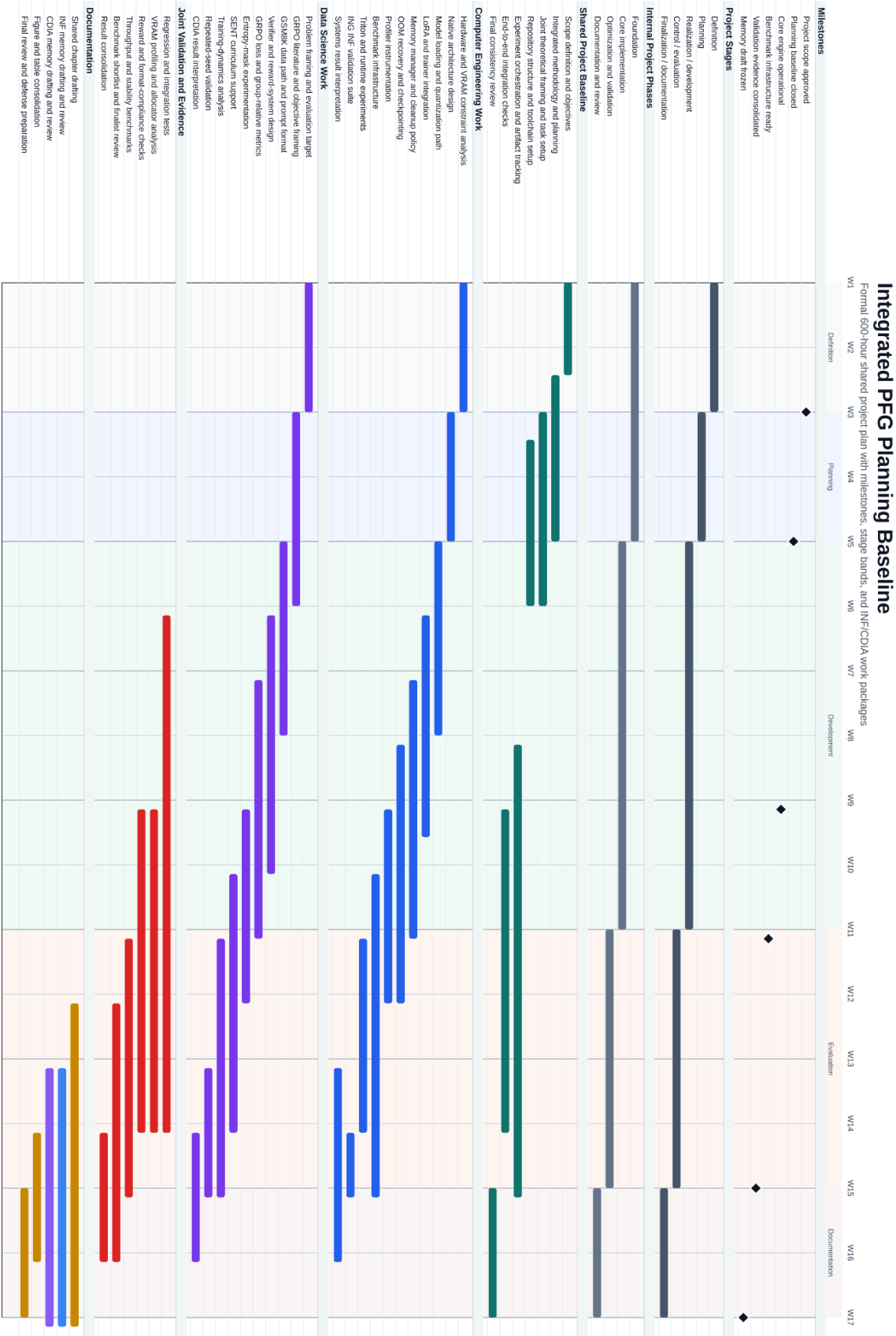


Figure 2.2: Integrated 600-hour FDP planning baseline.

In addition to the phase-based planning, the project was distributed by activity type in order to reflect the real balance between implementation and experimental work. The resulting allocation is shown in Table 2.3.

Activity block	Hours	Share
Implementation	210	35%
Profiling and optimization	120	20%
Experimental runs and validation	150	25%
Analysis and documentation	120	20%
Total	600	100%

Table 2.3: Planned work allocation by activity block.

This distribution reflects the character of the project. The dominant effort was technical construction and controlled validation. The implementation block covered the native training engine, memory manager, reward pipeline, benchmark tooling, and associated infrastructure. Profiling and optimization appeared as a distinct block because, under an 8 GB VRAM constraint, performance analysis and memory-behavior inspection were not optional refinements. Experimental runs and validation occupied a quarter of the total plan because each major claim required bounded benchmark execution, artifact review, and regression checking. The remaining time covered interpretation, chapter writing, synthesis of results, and final review.

The project also required an explicit critical-path interpretation:

1. The theoretical and methodological dependency came first: the GRPO formulation, the target task, the reward structure, and the system requirements had to be clear before implementation could converge.

2. The technical dependency followed: core engine components such as model loading, LoRA injection, memory management, and the trainer loop had to exist before useful benchmarks or quality diagnostics could be run.
3. The evaluative dependency came next: benchmark suites, structured logging, and validation tests had to mature before configuration decisions could be treated as final.
4. Only after those stages could the documentation be written with enough confidence to distinguish between safe claims and still pending ones.

The main bottleneck of the critical path was experimental time. Even when the implementation was correct, long-run reinforcement-learning experiments and controlled reruns on a single consumer GPU imposed a hard wall-clock limit. For this reason, the plan included implicit buffers for reruns, debugging after OOM incidents, and re-benchmarking after sensitive changes. In practice, that meant the project could not assume that every planned experiment would succeed on the first attempt or that every apparent optimization would survive the full validation workflow.

2.4 TASK DISTRIBUTION BY WORK AREA

Computer Engineering work concentrated on the system-level responsibilities of the project. These tasks included the design of the native architecture, management of the constrained VRAM budget, implementation of explicit memory-cleanup and checkpointing strategies, construction of OOM-recovery behavior, and development of profiling and benchmarking infrastructure. This area also covered the systems interpretation of Triton experimentation, low-level performance bottleneck analysis, and the engineering justification for avoiding higher-level abstractions that would have reduced runtime visibility.

Data Science work concentrated on the algorithmic and evaluative components. Its responsibilities included the formulation and verification of the GRPO-style objective, design of the rule-based reward and verifier path, preparation of the verifiable math-dataset workflow, integration of entropy masking and SENT curriculum support, and interpretation of reward behavior and model evaluation results. It also carried the methodological burden of defining how benchmark outputs, training dynamics, and reasoning-quality signals should be interpreted without overclaiming beyond the available experimental budget.

Not all tasks can be assigned cleanly to one area. The project also included shared work that had to be developed collaboratively and then distributed between both tracks in the planning model. These shared tasks included overall project definition, feasibility analysis, integrated planning, end-to-end integration testing, experimental orchestration, benchmark review, interpretation of cross-cutting results, and the writing of the common introductory and methodological sections. The fact that the two memories are distinct does not imply that the implementation process was divided into isolated

tracks. On the contrary, the project depended on continuous exchange between the system view and the algorithmic view.

Table 2.4 summarizes the formal distribution.

Phase	Computer Engineering tasks	Data Science tasks	Shared tasks
Foundation	hardware constraints analysis, native architecture criteria, memory-risk identification	problem framing, GRPO literature review, verifiable math-task evaluation framing, metric definition	scope definition, feasibility study, integrated planning, toolchain setup
Core implementation	model loading path, LoRA integration, memory manager, OOM recovery, profiler instrumentation, benchmarking infrastructure	GRPO loss implementation, verifier and reward logic, normalized dataset pipeline, entropy-mask logic, SENT support, evaluation scripts	end-to-end integration, experiment orchestration, debugging loops
Optimization and validation	throughput analysis, VRAM profiling, Triton experiments, benchmark comparison, runtime stability validation	training-dynamics analysis, benchmark interpretation, statistical checks, reward-behavior inspection, error analysis	acceptance criteria review, rerun prioritization, result consolidation
Documentation and review	Computer Engineering memory writing, systems diagrams, engineering justification, defense preparation	Data Science memory writing, methodology/results discussion, limitation analysis, defense preparation	final consistency review, shared introduction and methodology revision

Table 2.4: Task distribution by work area.

Figure 2.3 summarizes the task network behind that distribution. Unlike the Gantt chart, it does not represent calendar duration. It shows the dependency logic that connected planning, the two technical work areas, integrated validation, result consolidation, and final memory preparation.

Integrated PFG Task Network

Dependency map for the shared methodology chapter: planning, dual-track implementation, validation, and final consolidation

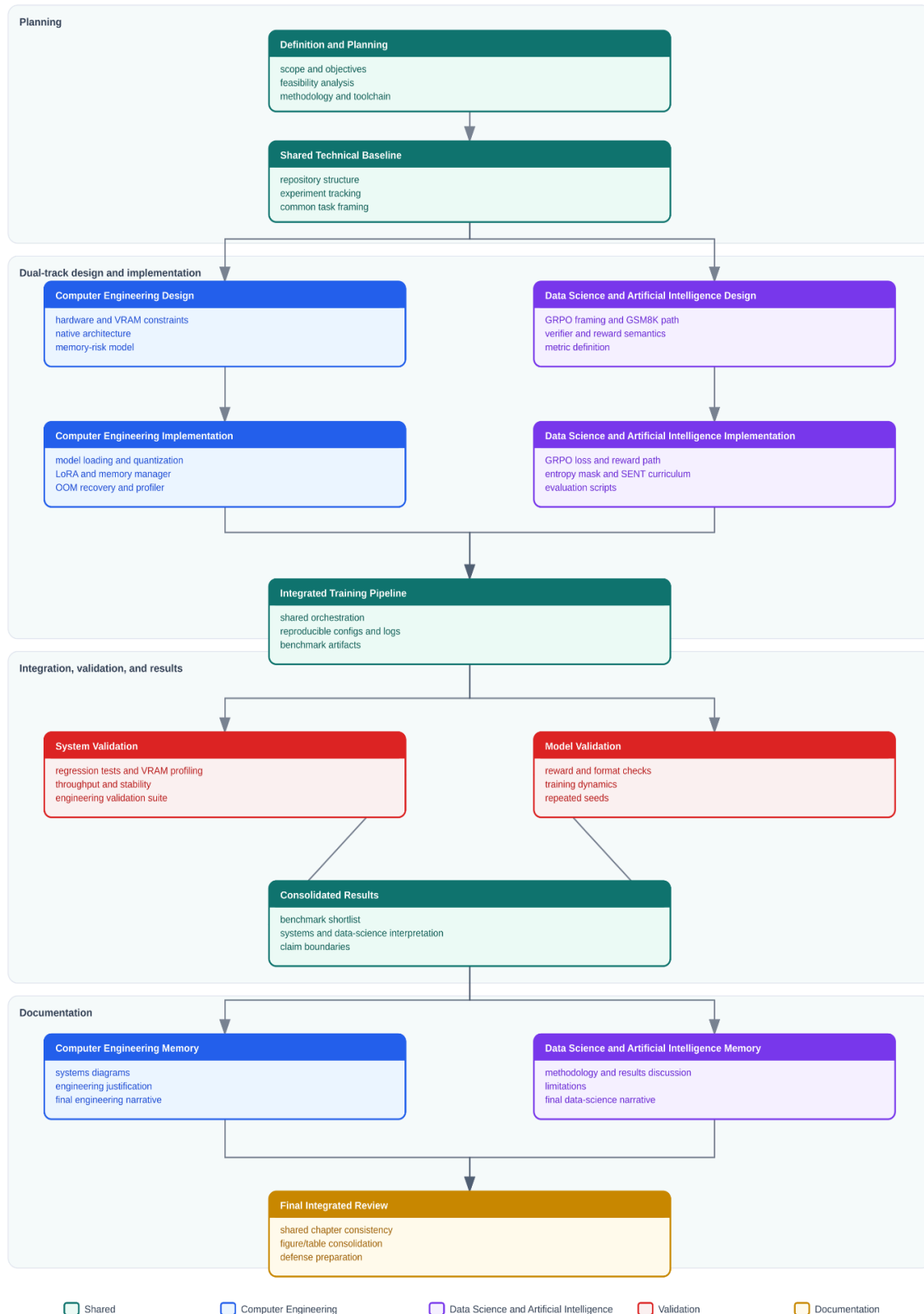


Figure 2.3: Integrated FDP task network.

2.5 BUDGET OVERVIEW AND COST DISTRIBUTION

The budget was prepared as a project resource model. The central hardware asset was the development workstation used throughout the project, with the NVIDIA RTX 3060 Ti as the main computational constraint. For budget presentation, the workstation is treated through the amortized project-use fraction of its approximate 1100 EUR reference acquisition cost, including the GPU as part of the same working machine. With a four-year useful life and a six-month project period, the attributable hardware cost is 137.50 EUR.

Direct software expenditure was effectively zero. The project relied on open-source or no-cost tools such as PyTorch, Transformers, Triton, bitsandbytes, pytest, Git, and GitHub. Weights & Biases was used when needed for experiment tracking, and the free plan covered that use. Electricity consumption is acknowledged as an operational factor, but it was not isolated from the household environment and therefore is not itemized as a separate amount.

The largest cost component is the planned dedication of work time. The human-resource estimate follows the roles performed during the project and keeps the 600-hour planning baseline established for the integrated work. Table 2.5 distributes those hours across the main roles involved in planning, systems engineering, model experimentation, evaluation, and documentation.

Role	Main responsibility	Computer Engineering hours	Data Science hours	Total hours	Rate	Cost
Project manager	Planning, coordination, scope control, and documentation schedule	35	35	70	30 EUR/h	2100 EUR
Systems engineer	Architecture, memory management, profiling, optimization, and testing infrastructure	170	20	190	35 EUR/h	6650 EUR
ML engineer and data scientist	Reward design, dataset preparation, training	20	170	190	35 EUR/h	6650 EUR

	configuration, and evaluation analysis					
Evaluation engineer	Benchmark execution, result validation, and experiment comparison	45	45	90	30 EUR/h	2700 EUR
Technical writer	Final documentation, revision, and defense preparation	30	30	60	25 EUR/h	1500 EUR
Total student labor	Integrated project workload	300	300	600	-	19600 EUR

Table 2.5: Human-resource cost distribution.

The remaining cost components correspond to the technical resources used during development. The workstation provides the material basis for the implementation and experiments, while the software stack adds no direct licensing cost. Table 2.6 consolidates the itemized budget.

Cost block	Amount
Human-resource cost	19600 EUR
Amortized development workstation including RTX 3060 Ti	137.50 EUR
Software and libraries	0 EUR
Weights & Biases	0 EUR
Electricity	not itemized
Total itemized budget	19737.50 EUR

Table 2.6: Itemized project budget.

3. THEORETICAL FOUNDATIONS

The theoretical basis of the project starts from the parts of language-model training that directly affect the experiments. A generated answer can be treated as the output of a policy, mathematical reasoning can be evaluated through verifiable final answers, and GRPO turns groups of sampled responses into the comparison signal used for learning. These concepts provide the vocabulary needed to describe the design choices, the training campaign, and the later interpretation of results.

3.1 POST-TRAINING MECHANICS IN LANGUAGE MODELS

A decoder-only language model defines a probability distribution over the next token conditioned on the prompt and on the tokens already generated. If the prompt and the generated response are represented symbolically, the response probability factorizes autoregressively:

$$\pi_{\theta}(o \mid q) = \prod_{t=1}^T \pi_{\theta}(o_t \mid q, o_{<t})$$

This factorization is enough to connect language modeling with reinforcement learning. The model can be treated as a policy: at each decoding step it chooses the next token, and the complete response receives some score after generation. In standard pretraining, the objective is imitation of the text distribution through next-token prediction. In instruction tuning, the same next-token objective is applied to curated prompt-response examples. Reinforcement-learning post-training changes the supervision signal: the generated response is sampled from the current or recent policy, evaluated by a reward function, and then used to update the policy toward responses with higher reward.

Reasoning-oriented behavior cannot be reduced to stored knowledge. A model may contain enough information to solve a problem and still fail to generate a coherent solution path, stop at the wrong intermediate value, or present the final answer in a form that cannot be checked reliably. Post-training shapes this generated behavior after broad linguistic and factual competence has already been learned. The training target becomes the quality of sampled responses rather than only the likelihood of stored demonstrations.

For a response-level reward function, the policy objective can be written as:

$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, o \sim \pi_{\theta}(\cdot \mid q)} [R(q, o)]$$

The policy-gradient view then increases the probability of generated tokens that contributed to a positive advantage and decreases the probability of tokens attached to negative advantage:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(o_t \mid q, o_{<t}) A_t \right]$$

The advantage term is the comparison signal. It expresses whether the sampled behavior was better or worse than a baseline. Different post-training algorithms mainly differ in how they define the reward, how they estimate the baseline, how they keep policy updates stable, and how much auxiliary machinery they require.

Full-parameter updates are not the only way to modify a language model. Parameter-efficient adaptation modifies only a small set of trainable parameters while keeping most pretrained weights fixed. Low-Rank Adaptation applies this idea by freezing a pretrained weight matrix and learning its update through two smaller trainable matrices [10]:

$$W = W_0 + \Delta W, \Delta W = BA,$$

The pretrained matrix remains frozen, while the update is learned through two much smaller trainable matrices. The LoRA paper motivates this design from the hypothesis that adaptation updates have low intrinsic rank: a task may require a meaningful change in behavior without requiring an independent update direction for every parameter in the original matrix [10]. The rank therefore controls the capacity of the adapter. A small rank gives a small trainable subspace and a lower-cost optimization problem; a larger rank gives the adapter more freedom and moves the method closer to ordinary fine-tuning.

For an input activation, the original projection and the LoRA update are applied in parallel and their outputs are added:

$$h = W_0x + \frac{\alpha}{r}BAx.$$

The scaling factor keeps the size of the adapter contribution comparable when the rank changes. The original paper initializes one of the two LoRA matrices to zero, so the added update is initially zero and the model starts from the pretrained behavior rather than from a perturbed policy [10]. During training, gradients from the downstream objective update only the adapter matrices. LoRA is therefore a controlled adaptation model: the observed reward or loss signal can change the policy, but only through a restricted low-rank correction to selected projections.

This matters for reasoning post-training because the data signal is sparse and expensive. Each generated group provides limited information, and full-parameter reinforcement-learning updates would greatly increase memory use, optimizer state, and the risk of unstable drift. LoRA keeps the pretrained model as the fixed base distribution while learning a compact behavioral shift from verifier rewards. LoRA defines where the policy can change; the reward and policy objective define why it changes.

3.2 REASONING AS AN OPTIMIZATION TARGET

Reasoning in a language model is not directly observed as an internal cognitive state. What can be observed is the generated trace: the sequence of tokens in which the model decomposes a problem, performs intermediate operations, revises or checks a candidate solution, and eventually emits an answer. For mathematical tasks, this trace often contains intermediate computations followed by a final numerical or symbolic result. The response can be divided conceptually into two parts: the path that leads toward the answer, and the answer-bearing segment that can be compared with a target.

Chain-of-Thought prompting made this distinction visible by encouraging models to articulate intermediate steps before giving the answer. The relevant point here is not that human-written chains of thought are the training target. Intermediate generated text can expose part of the model's problem-solving process and can change task performance. DeepSeek-R1 places this history in the background of reasoning-model training: chain-of-thought prompting and supervised multi-step trajectories improved complex-task performance, but dependence on human-annotated reasoning traces limits scalability and can constrain exploration [1]. The project setting follows the later branch of that argument: generated reasoning traces are produced by the model, while the main supervision signal comes from verifiable outcomes.

Mathematical reasoning is suitable for this kind of study because the final answer can often be checked without asking another model or a human judge to decide whether a response is good. If a problem has a known numerical answer, the evaluator can extract the model's final answer, normalize it, and compare it against the ground truth. This does not solve the whole reasoning problem. It only makes one part of the problem measurable. A response may contain a convincing explanation and still end with a wrong answer; another response may reach the correct answer through a terse or hard-to-interpret path. Final-answer correctness is a clean signal, but it is also incomplete.

This creates the main tension behind reinforcement learning for reasoning. The training loop can reward a correct result, but the reward may arrive only after hundreds or thousands of generated tokens. Many intermediate tokens might be irrelevant, some might contain useful exploration, and one late arithmetic error can make the whole response incorrect. Outcome-based training optimizes a compressed view of reasoning. It encourages responses that lead to verifiable success, but it does not directly label each step as valid or invalid. This is the credit-assignment problem in long generated solutions [11].

Fluency is not a sufficient substitute for verification. A model can produce a polished explanation with an incorrect computation, or it can bury the correct number among several intermediate values. In a verifiable mathematical setting, the evaluator must distinguish the final answer from the rest of the trace. Reasoning quality is tied to both correctness and answer structure. The text must solve the problem, and the answer must be recoverable in a stable way.

3.3 RLHF, PPO AND THEIR LIMITS FOR REASONING

Reinforcement Learning from Human Feedback became a standard post-training pattern for aligning language models with human preferences. In a typical RLHF pipeline, humans compare candidate responses, a reward model is trained from those preferences, and a policy is optimized against that learned reward model. The reward model acts as a proxy for an underlying preference signal that cannot be measured directly [3] [12].

This design is useful when the target behavior is subjective or hard to specify with rules. Helpfulness, harmlessness, tone, and preference quality are not usually reducible to a single exact answer. A learned reward model can absorb many such judgments into a scalar signal. The same property becomes less attractive for mathematical reasoning. When a correct final answer is available, using a learned preference model introduces an extra approximation layer between the generated solution and the objective. The policy may then learn to exploit the reward model's weaknesses instead of improving the target behavior. Reward hacking is a known failure mode of proxy reward optimization: proxy reward can rise while true quality stagnates or degrades [12].

PPO is the usual reference point for online RLHF [2] [3]. Its role is to update the policy without allowing it to drift too far from the policy that generated the training samples. For a generated token, the probability ratio compares the new policy with the old policy:

$$r_t(\theta) = \frac{\pi_{\theta}(o_t \mid q, o < t)}{\pi_{\theta_{\text{old}}}(o_t \mid q, o < t)}$$

The clipped PPO surrogate bounds the effect of this ratio:

$$L_{\text{PPO}}(\theta) = \mathbb{E}_t [\min (r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

The clipping mechanism matters because language-model updates can otherwise push the policy too far toward a small set of high-reward samples. PPO also commonly relies on a value function to estimate the baseline used in the advantage:

$$A_t = R_t - V\phi(s_t)$$

The value function reduces variance, but it adds another learned component. In actor-critic terminology, this component is the critic: it estimates how good the current partial generation state is before the final answer is known. In long-form reasoning, this component has a difficult job: it must estimate the value of partial text states even though the main reward may only be known after the final answer is extracted and verified. A separate critic also increases implementation complexity and memory use. Advantage and value-function estimation are therefore part of the practical cost of PPO-style online reinforcement learning [1] [2].

The limitation is not that PPO is conceptually wrong. PPO supplies the conservative update machinery that later methods keep. The mismatch is practical. For verifier-compatible reasoning, the training loop does not need a learned model to decide whether a final numerical answer is correct. It needs a policy objective that can use sparse rule-based rewards without requiring a separate value network or a heavier actor-critic system. Table 3.1 summarizes that distinction.

Approach	Reward signal	Extra learned component	Fit for mathematical verification
RLHF with a reward model	Learned approximation of human preference	Reward model trained from comparisons	Useful for subjective quality, but indirect when an exact answer is available
PPO-style online optimization	Scalar reward combined with a clipped policy update	Often a value function or critic for advantage estimation	Provides useful update control, but adds cost and complexity for sparse final-answer rewards
GRPO with verifiable rewards	Rule-based reward from extracted answer correctness and format checks	No separate learned critic in the basic group-relative formulation	Matches tasks where the final answer can be checked automatically, but does not label intermediate reasoning steps

Table 3.1: Reward and optimization fit for mathematical reasoning.

3.4 VERIFIABLE REWARDS AND RLVR

Reinforcement Learning from Verifiable Rewards (RLVR) replaces subjective or learned preference feedback with a reward that can be computed from an external rule. In mathematical reasoning, the most direct rule is final-answer correctness. A generated answer is extracted, normalized, and compared against the known solution. In this setting, the verifier is that extraction and comparison procedure, not a separate learned judge. In code tasks, a related verifier may execute tests. The shared idea is that the reward is grounded in an automatic check rather than in a trained preference model.

The important theoretical distinction is the source of authority for the reward. RLHF relies on a learned approximation of human preference, whereas RLVR relies on an external rule that can be inspected independently of the model being trained. This does not make the reward complete, but it changes the kind of uncertainty involved. The experiment no longer depends on whether a reward

model has learned human judgment correctly. It depends on whether the rule measures the part of the task being optimized. For mathematical final-answer tasks, that rule is limited, but defensible.

DeepSeek-R1 uses this principle in its reasoning-oriented reinforcement-learning stage. Its rule-based reward system includes accuracy rewards and format rewards for mathematical, coding, and logical reasoning domains [1]. Accuracy reward checks whether the response is correct. Format reward encourages the response to place the reasoning and answer in a structure that the verifier can inspect. In mathematical final-answer tasks, this creates a sparse signal: the generated answer either matches the expected result after extraction and normalization, or it does not.

The basic correctness reward for a mathematical task can be written as:

$$R_{\text{acc}}(q, o) = \mathbb{1}[\text{extract}(o) = y(q)]$$

where the extracted answer is compared with the ground truth after normalization. A format component can be added when the response structure itself affects whether the answer can be checked:

$$R(q, o) = R_{\text{acc}}(q, o) + \lambda R_{\text{fmt}}(q, o)$$

The format term should not be confused with reasoning quality. It is a support mechanism for verification. It reduces ambiguity by encouraging a predictable answer location or tag structure. A well-structured wrong answer remains wrong, and an unstructured response may still contain useful reasoning. The reason to include format reward is practical: without stable extraction, correctness reward becomes noisy because the verifier may select an intermediate number or fail to locate the final answer.

Verifiable rewards also have a clear weakness: they evaluate only what can be checked. The process-level blind spot has several forms:

- **No direct step supervision.** In an outcome-reward setting, the final answer can be verified, but the intermediate reasoning process is not directly labelled.
- **Late-error equivalence.** A partially correct solution with one late error receives the same correctness reward as a response that fails immediately.
- **Trace-quality ambiguity.** A correct final answer can receive reward even if the trace is terse or not fully faithful to the actual computation.

The resulting limitation is a credit-assignment problem. The training loop observes the final outcome, but it does not directly know which intermediate step made the response correct or incorrect.

3.5 GRPO FOUNDATIONS

Group Relative Policy Optimization keeps the conservative update idea associated with PPO but removes the learned critic. Instead of estimating a value baseline with a separate model, GRPO samples several responses for the same prompt and compares their rewards within the group. DeepSeekMath introduced this group-relative alternative to PPO, and DeepSeek-R1 later used the same idea to simplify reasoning-oriented training and reduce PPO resource consumption [4] [1].

The foundation is the mechanism, not the full local variant. GRPO defines a critic-free way to turn response-level rewards into a policy update by comparing several sampled completions for the same prompt. Later implementation choices can change normalization, regularization, or token-selection details, but they adapt this base mechanism rather than replacing it. The underlying concept is still prompt-local comparison under a clipped policy update.

For a prompt q , the old policy generates a group of G responses. This is the rollout group: several alternative completions for the same question, sampled before the update so their rewards can be compared against one another.

The old policy is the policy snapshot that produced the sampled responses before the current update. Keeping this snapshot fixed during the update makes the probability ratio meaningful: it compares the updated model against the model that actually generated the data.

$$o_1, o_2, \dots, o_G \sim \pi_{\text{old}}(\cdot \mid q)$$

Each response receives a scalar reward:

$$R_i = R(q, o_i)$$

The group mean gives a prompt-local baseline:

$$\mu_R = \frac{1}{G} \sum_{j=1}^G R_j$$

The standard GRPO advantage normalizes the reward inside the group:

$$\hat{A}_i = \frac{R_i - \mu_R}{\sigma_R}$$

with:

$$\sigma_R = \sqrt{\frac{1}{G} \sum_{j=1}^G (R_j - \mu_R)^2}$$

This advantage says whether response i was better or worse than the other responses generated for the same prompt. The comparison is local to the question. In mathematical reasoning, that locality is useful because different prompts have different difficulty levels: a raw reward of zero may mean something different for an easy question and for a hard one, but the group comparison asks whether one sampled response was better than its peers for the same input.

The token-level policy ratio is the same kind of object used in PPO:

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_i, t \mid q, o_i, < t)}{\pi_{\theta_{\text{old}}}(o_i, t \mid q, o_i, < t)}$$

A compact GRPO surrogate can be written as:

$$L_{\text{GRPO}}(\theta) = \mathbb{E}_{q, o_i} \left[\frac{1}{G} \sum_i \left(\frac{1}{|o_i|} \sum_t \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right) \right]$$

DeepSeek-R1 also includes a KL term against a reference policy in its reported formulation [1]. Abstractly, this can be represented as:

$$L_{\text{GRPO+KL}}(\theta) = L_{\text{GRPO}}(\theta) - \beta \text{KL}(\pi_{\theta} \mid \pi_{\text{ref}})$$

The KL term discourages the updated policy from moving too far from a reference model. The group-relative part supplies the critic-free advantage estimate; clipping bounds policy change; the optional KL term adds a second stability pressure.

GRPO fits verifier-based reasoning because it matches the structure of the available signal:

- **Verifier signal.** The verifier assigns scalar rewards to complete responses.
- **Grouped baseline.** The grouped rollout creates a prompt-local baseline without training a critic.
- **Conservative update.** The clipped policy ratio limits how far the updated policy can move from the policy that produced the samples.
- **Remaining limits.** The method does not solve sparse rewards, process-level credit assignment, or verbosity by itself.

The result is a direct route from verifiable final-answer rewards to policy optimization. The implemented variant is an adaptation of this foundation rather than a separate algorithmic family.

4. STATE OF THE ART

The theoretical foundations already establish the policy-optimization mechanics used here: verifiable rewards, sparse final-answer signals, PPO-style clipping, and grouped GRPO advantages. The literature review treats those mechanisms as a baseline. The focus now is the external research record: which systems made this training pattern visible, which variants changed the objective, which reward-design limits remain unresolved, and where a compact local study can still make a bounded contribution.

4.1 REASONING-ORIENTED REINFORCEMENT LEARNING

Recent work on reasoning-oriented language models has moved reinforcement learning from a secondary alignment technique to one of the main ways of improving mathematical and procedural reasoning. The change is most visible in domains where the output can be checked automatically. Mathematics, coding, and structured science, technology, engineering and mathematics (STEM) tasks give the training loop a sharper signal than open-ended preference judgment: a final answer may be wrong or correct, a program may pass or fail tests, and a required answer format may be inspected by a parser.

DeepSeek-R1 is the main reference point for this line of work because it made rule-based reinforcement learning central to the reasoning-model literature. Its important shift was to treat self-generated trajectories as trainable evidence for reasoning behavior, rather than treating supervised or distilled Chain-of-Thought traces as the only plausible route for improving reasoning models [1]. Later work is measured against that change: it either tries to stabilize the same reinforcement-learning pattern, reduce its cost, or replace parts of its reward and optimization design.

The same report also shows why the result is difficult to reproduce directly in a small local setting. DeepSeek-R1-Zero samples 16 outputs per question and uses response budgets of 32768 tokens before later increasing the maximum length [1]. Those values are part of the method's empirical context. Long responses and large rollout groups give the optimizer more search space and richer within-prompt comparison, but they also multiply memory use, generation time, and the cost of each training step. DeepSeek-R1 is a methodological template rather than a recipe that can be copied unchanged on consumer hardware.

The literature after DeepSeek-R1 has split into two related directions. One direction studies how to improve or stabilize GRPO-family training at larger scale. Another asks how much of the reasoning-RL pattern can be retained in smaller models and cheaper training setups. Tina is important in the second direction because it trains DeepSeek-R1-Distill-Qwen-1.5B with LoRA-based reinforcement learning and reports meaningful reasoning gains at very low post-training cost [9]. Its main

contribution is not a new optimization objective. It shows that compact reasoning-model post-training can be academically relevant when the base model, adapter strategy, reward path, and evaluation protocol are controlled.

Reasoning benchmarks in this literature are chosen for verifiability. DeepSeek-R1 reports results across American Invitational Mathematics Examination (AIME), MATH500, coding, and STEM-style tasks [1]. Tina evaluates a 1.5B setting across AIME24, AIME25, American Mathematics Competitions 2023 (AMC23), MATH500, Graduate-Level Google-Proof Question Answering (GPQA), and Minerva, with a consistent inference setup used for comparison [9]. The survey literature describes the broader field in the same terms: reinforcement learning for large reasoning models is now concentrated around mathematical and coding tasks because they expose long-horizon reasoning while still allowing clearer reward definitions than open-domain generation [11].

This benchmark framing narrows the claim to what the evidence can support. Mathematical reasoning is not a substitute for general intelligence; instead, it is a practical way to test how models handle reasoning during the final stages of training. A local study remains useful when it explains how a Group Relative Policy Optimization pipeline behaves under a verifiable mathematical reward, even without frontier-laboratory scale or infrastructure.

4.2 GRPO VARIANTS AND OBJECTIVE REFINEMENTS

After the first R1-style results, GRPO stopped being treated as one fixed recipe and became a family of objective designs. The central debate shifted toward the behavior of grouped objectives under sparse mathematical rewards, long generated traces, limited rollout groups, and bounded response budgets. Small changes in normalization, reward scaling, response-length treatment, or regularization can shift the effective learning signal even when the method still belongs to the same GRPO family.

Here, normalization refers to how the objective rescales rewards or token contributions before the policy update is applied. That detail matters because the rescaling can change which completion receives more influence, even when the final reward value is unchanged.

Dr. GRPO is one of the clearest examples of this shift from algorithm selection to objective auditing. The paper writes the commonly used GRPO surrogate with two normalization terms that later become the object of critique [13]:

$$JGRPO(\pi\theta) = \mathbb{E}_{q \sim p_Q, o_i \sim 1^G \sim \pi\theta_{old}(\cdot | q)} \left[\frac{1}{G} \sum_i i = 1^G \frac{1}{|o_i|} \sum_t t = 1^{|o_i|} \min \left(\frac{\pi\theta(o_i, t | q, o_i, < t)}{\pi\theta_{old}(o_i, t | q, o_i, < t)} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi\theta(o_i, t | q, o_i, < t)}{\pi\theta_{old}(o_i, t | q, o_i, < t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right) \right]$$

with the group-normalized advantage:

$$\hat{A}_{i,t} = \frac{R(q, o_i) - \text{mean}(R(q, o_1), \dots, R(q, o_G))}{\text{std}(R(q, o_1), \dots, R(q, o_G))}.$$

The critique is that response-length averaging and group-standard-deviation scaling are not harmless implementation details. Averaging the token loss by generated response length changes the effective weight of a completion according to how many tokens it produced. Dividing the centered reward by the group standard deviation changes update scale according to reward dispersion inside the sampled group. These choices are especially delicate under binary or near-binary rewards, where a group may contain little variation or may switch abruptly between no contrast and strong contrast [13].

The proposed Dr. GRPO correction removes those two terms and keeps the centered group comparison:

$$\tilde{A}_{i,t} = R(q, o_i) - \text{mean}(R(q, o_1), \dots, R(q, o_G)).$$

The resulting surrogate can be read as the same clipped policy-ratio update, but without response-length normalization or group standard-deviation normalization:

$$J_{\text{Dr.GRPO}}(\pi\theta) = \mathbb{E}_{q \sim p_Q, o_i = 1^G \sim \pi_{\text{old}}(\cdot | q)} \left[\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(\frac{\pi\theta(o_i, t | q, o_i, < t)}{\pi_{\text{old}}(o_i, t | q, o_i, < t)} \tilde{A}_{i,t}, \text{clip} \left(\frac{\pi\theta(o_i, t | q, o_i, < t)}{\pi_{\text{old}}(o_i, t | q, o_i, < t)}, 1 - \epsilon, 1 + \epsilon \right) \tilde{A}_{i,t} \right) \right].$$

The Dr. GRPO ablations make the point empirically rather than only mathematically. The paper compares variants that remove length normalization, standard-deviation normalization, or both, and reports that the debiased variants outperform vanilla GRPO in training reward and evaluation accuracy. It also observes that removing length normalization produces shorter responses than the original formulation [13]. The important lesson is not simply that shorter answers are better. The stronger claim is that objective normalization can create incentives that are not visible if one only inspects the final reward curve. Response length becomes part of the optimization problem.

This matters for reasoning tasks because verbosity can look useful while being harmful. A long solution may include more checking, more self-correction, or more intermediate exploration, but it also consumes generation budget, raises truncation risk, and gives the verifier more candidate numbers to disambiguate. A method that increases reward by encouraging long answer tails may be less robust than its reward suggests. Dr. GRPO shifts the evaluation from raw reward gain to the source of that gain: the objective should reward the behavior the experiment is trying to train.

GRPO-LEAD extends the same audit from objective normalization to reward and example weighting. It makes conciseness, wrong-answer treatment, and prompt difficulty explicit parts of the training signal [14]. KL control means penalizing movement away from a reference policy, while reward shaping means adding extra signals beyond final correctness. Both add further control points, but also add reference-policy dependence, possible exploration limits, or uncertainty from the shaping signal [15] [16]. Table 4.1 separates these refinements by the weakness they isolate and the extra assumption they introduce.

Refinement	Problem targeted	Mechanism	Benefit	Caveat for compact studies
Dr. GRPO	Length and variance-normalization bias	Removes response-length normalization and group standard-deviation normalization while keeping centered group comparison [13].	Makes the objective easier to audit when response length becomes part of the learning signal.	Does not by itself solve sparse rewards, reward hacking, or process-level credit assignment.
GRPO-LEAD	Sparse binary reward, verbosity, weak treatment of wrong answers, and uneven example difficulty	Combines length-regularized rewards, explicit penalties for wrong solutions, and difficulty-aware advantage reweighting [14].	Gives the reward path more control over conciseness, incorrectness, and informative prompts.	Adds assumptions about penalty strength, dataset difficulty, and whether the same scales transfer across models.
Clipping, KL and reference-policy controls	Policy drift during reinforcement learning	Uses conservative update bounds, constrains the updated policy against a reference policy, or adds an explicit KL term [1] [15].	Can limit destructive movement away from the starting model.	Can reduce exploration and introduces dependence on a reference-policy choice.
Reward shaping	Coarse credit assignment under sparse final-answer rewards	Adds denser or implicit signals derived from heuristics, another policy, or a learned approximation [16].	May provide a more informative gradient than final correctness alone.	Moves uncertainty into the shaping signal and makes attribution harder.

Table 4.1: GRPO-family refinement trade-offs.

The practical consequence is that a compact study gains little from importing every recent refinement at once. Combining all of them would make the resulting behavior difficult to attribute. A narrower formulation is more defensible when it selects the few biases most relevant to the local evidence,

states which refinements are omitted, and evaluates the resulting training dynamics under controlled conditions.

4.3 REWARD DESIGN AND CREDIT ASSIGNMENT

The literature treats reward design as a choice about which part of a reasoning trace is allowed to influence the policy update. The basic verifier is only the starting point. Recent work asks whether a final-answer signal is enough, when it becomes too coarse, and what extra machinery can recover information about intermediate reasoning. DeepSeek-R1 is the reference point for the lightweight branch because it uses rule-based accuracy and format rewards for reasoning domains, while later surveys classify such rewards as sparse but reliable when the task has an objective checker [1] [11].

The central distinction is between outcome-level and process-level supervision. Outcome-level supervision gives one score to the completed response. Process-level supervision tries to score intermediate steps, which can give more information but requires a trustworthy way to judge those steps.

The trade-off can be expressed by comparing an outcome reward with a process-aware reward over a generated trace made of several intermediate steps:

$$R_{\text{out}}(q, o) = v(q, o), R_{\text{proc}}(q, o) = \sum_j 1^m r_j(q, s \leq j).$$

Here the first term is a verifier score for the completed response, while the second assigns local feedback to each step. The first form is easy to audit because it depends on a final answer or another external check. The second form can shorten the credit-assignment path, but only if the step-level signal is trustworthy. Recent reward-modeling surveys frame dense or shaped rewards as useful but risky: they may provide more informative gradients, yet they also add assumptions through reward models, structural heuristics, or extra verification procedures [16] [11].

This distinction matters because reasoning traces are not uniformly informative. Some tokens or steps express ordinary language continuation, while others commit to an arithmetic operation, choose a branch, or state the final answer. Outcome reward treats the whole sampled response as one unit. If the final answer is wrong, locally correct work before the first mistake receives no separate protection. If the final answer is right, irrelevant or accidental steps may still be reinforced because they belong to a successful completion. The problem is that outcome reward does not say where the decisive reasoning event occurred.

InT makes that missing location information explicit. The paper argues that standard outcome-reward RL assigns credit at the final-answer level and can suppress correct intermediate steps inside failed traces [17]. Its alternative is not to add a conventional critic to every step. Instead, the model proposes a targeted intervention for the first erroneous step of an incorrect rollout, producing a

patched prefix that can be used for supervised training before another RL stage. This positions InT as a credit-assignment method between full process supervision and plain outcome reward: it uses local corrections, but it avoids training a separate process reward model as the main mechanism.

Process reward models and verifiable process reward models occupy a heavier part of the same design space. They try to score intermediate steps directly, which can make learning more sample efficient and more interpretable when the step labels are accurate. The cost is substantial. A process signal may require annotated reasoning steps, symbolic checkers, reference solutions, repeated rollouts from partial states, or a learned judge whose own errors become part of the training objective. For compact local experiments, these requirements are not a small extension of final-answer verification. They change the data requirement, the validation burden, and the failure modes being studied.

This comparison sets a boundary rather than rejecting process supervision. Final-answer verification is sparse, but its semantics are transparent: the reward means that the extracted answer matched the target under a deterministic verifier. Process-level methods promise better credit assignment, but they would make the experiment about the reliability of intermediate supervision as much as about GRPO behavior. The selected branch stays with outcome-level verification and studies the weaknesses that remain visible under that choice: zero-advantage groups, response length pressure, truncation, format dependence, and the need for selective controls such as entropy masking and curriculum ordering.

4.4 EXPLORATION, ENTROPY AND CURRICULUM

Exploration is a persistent difficulty in reasoning-RL training. If the policy becomes too narrow, it may stop sampling useful alternative solution paths. If it remains too diffuse, the reward signal becomes noisy and learning slows. Entropy is one way to observe this tension because it measures uncertainty in the token distribution. The survey literature identifies entropy collapse, KL control, length regularization, and sampling policy as recurring issues in reinforcement learning for reasoning models [11].

Here entropy is used in two related but different ways. Token entropy measures uncertainty about the next token during generation. Semantic entropy measures whether several sampled responses lead to different answer meanings. Curriculum learning then uses this kind of difficulty or uncertainty estimate to decide which examples should appear earlier or later during training.

SENT formalizes this uncertainty at token level with the policy entropy over the vocabulary [18]:

$$H_t = - \sum_{v \in V} \pi_{\theta}(v \mid q, o_{<t}) \log \pi_{\theta}(v \mid q, o_{<t}).$$

This formula is useful for reading the entropy literature because it separates two different claims. High token entropy means that the next-token distribution is uncertain; it does not by itself mean that the token is semantically important or mathematically correct. Low entropy means that the policy is confident, but that confidence may reflect a useful stable step or an early collapse into a narrow pattern.

The SENT framework extends this token-level view into a two-level control scheme. Figure 4.1 shows how the paper separates semantic-entropy ordering of the training data from token-level entropy and covariance controls during optimization [18].

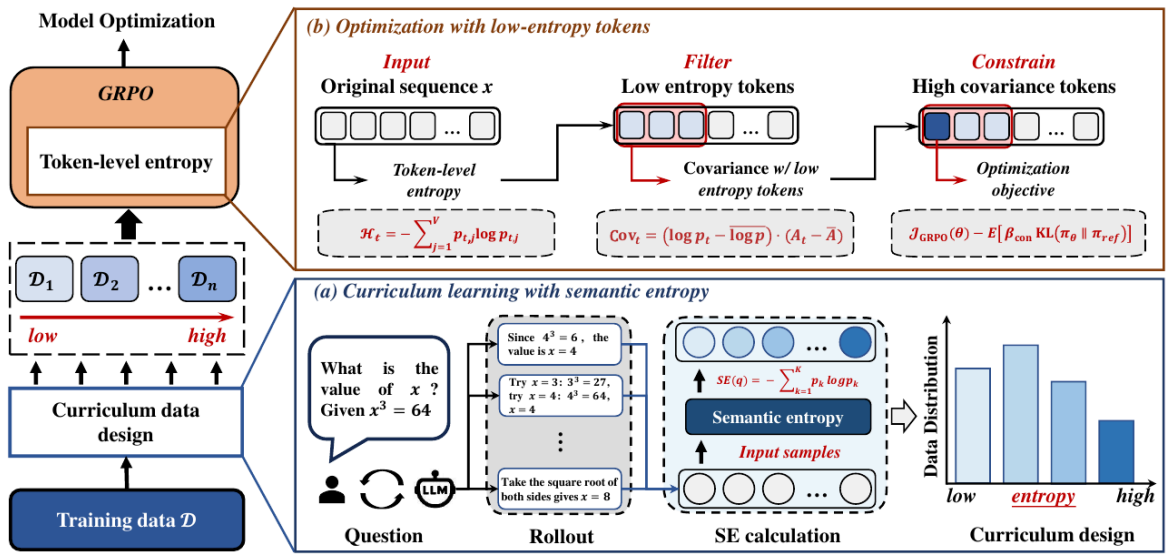


Figure 4.1: SENT framework with semantic and token entropy, extracted from [18], Fig. 2.

The 80/20 paper sharpens this token-level view. It argues that reasoning traces contain a low-entropy majority of fluency-preserving tokens and a high-entropy minority of "forking" tokens, where the model chooses between alternative reasoning directions [19]. Its central claim is not that entropy should be maximized everywhere, but that the useful exploration signal in RLVR is concentrated at a small subset of uncertain decision points. This matters for reading entropy-based methods: entropy is not only a global symptom of exploration or collapse, but also a possible selector for where policy-gradient pressure should be applied.

The paper formalizes this selective update by retaining the policy-gradient loss only for tokens whose entropy exceeds a batch threshold. The retained fraction determines the threshold, and the high-entropy objective multiplies the clipped policy-gradient term by a selection indicator before normalizing by the number of selected tokens [19]:

$$J_{\text{HighEnt}}^B(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G \sum_{t=1}^{|o_i|} \mathbb{I}[H_t^i \geq \tau_\rho^B]} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \mathbb{I}[H_t^i \geq \tau_\rho^B] \min \left(r_t^i(\theta) \hat{A}_{i,t}, \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t} \right) \right].$$

This formulation gives the 80/20 result a concrete algorithmic reading. The reported experiments compare full-token updates with updates restricted to the top high-entropy fraction, and the strongest setting often uses roughly the top 20% of tokens rather than the whole trace [19]. The same discussion also warns against a uniform entropy bonus for chain-of-thought reasoning: increasing entropy across all tokens can disturb the low-entropy language structure that keeps reasoning readable, whereas selective pressure on high-entropy positions can preserve ordinary fluency while leaving room for exploration.

Figure 4.2 compares high-entropy token-range updates with the all-token baseline and includes a bottom-80% low-entropy-only condition for Qwen3-8B. The comparison supports the paper's claim that the high-entropy minority carries much of the useful update signal, while low-entropy-only training can weaken the training dynamics [19].

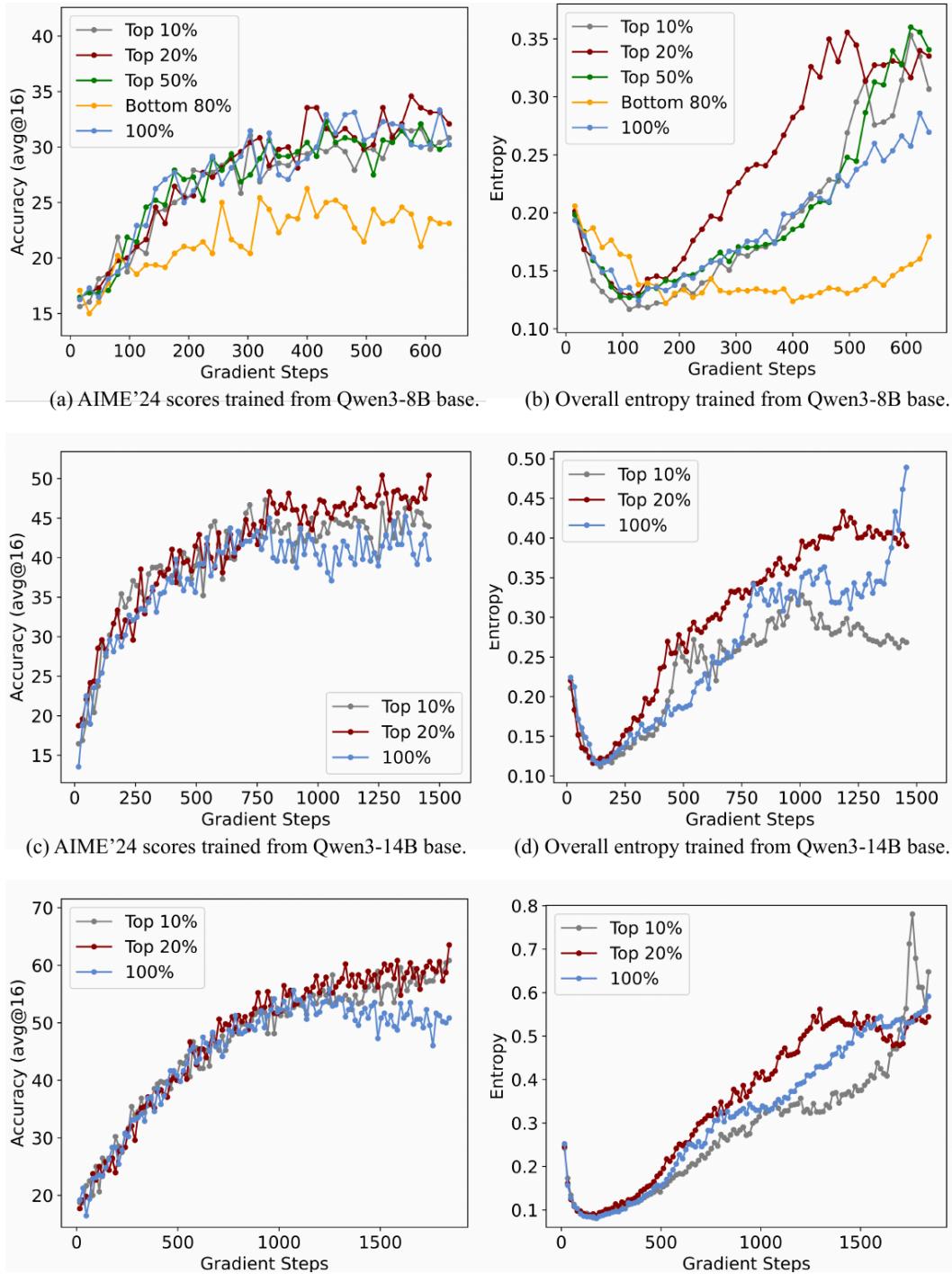


Figure 4.2: High-entropy token ranges, extracted from [19], Figure 6.

Later directional analyses make the interpretation more cautious. Work on RLVR update direction argues that entropy and KL are magnitude-style diagnostics; they can identify sparse regions of change, but they do not fully indicate whether RL has made a token more or less likely. Signed

log-probability shifts, written as $\Delta \log p$ between the base and RLVR policies, are introduced to capture that direction [20]. For this reason, high entropy is best treated as a useful selection signal, not as a complete explanation of credit assignment.

Entropy and curriculum appear in recent work as responses to sparse, uneven learning signal. The point is comparative: several papers argue that not all prompts, tokens, or completions are equally useful for learning at a given time. The local design uses that idea in a specific way, but the literature question is broader. It asks whether uncertainty, difficulty, or semantic dispersion can guide training more efficiently than uniform treatment of all generated text and all examples.

Recent entropy-aware work treats this problem at both the token level and the data level. The SENT paper argues that accuracy-oriented RLVR can reduce policy exploration and lead to entropy collapse. Its proposed method uses semantic entropy to organize data from lower to higher uncertainty and applies token-level constraints to preserve exploration at selected positions [18]. The premise is that examples and tokens do not all have the same learning value. Some prompts may be too easy to generate useful gradients, while others may be too difficult for the current policy. Some tokens may be nearly deterministic, while others mark real decision points in the reasoning trace.

The same paper defines semantic entropy over clusters of sampled responses. For a query q , responses are sampled from the current policy,

$$\{o_1, o_2, \dots, o_M\} \sim \pi_\theta(\cdot \mid q),$$

then grouped into semantic clusters. The probability of a cluster is obtained by summing the probabilities of responses in that cluster,

$$P(C_i \mid q) = \sum_{o \in C_i} P(o \mid q),$$

with normalized cluster probabilities:

$$\hat{P}(C_i \mid q) = \frac{P(C_i \mid q)}{\sum_{j=1}^K P(C_j \mid q)}.$$

Semantic entropy is then computed over the cluster distribution:

$$H_{SE}(q) = - \sum_{i=1}^K \hat{P}(C_i \mid q) \log \hat{P}(C_i \mid q).$$

This gives the curriculum idea a more precise meaning. A low semantic-entropy value means that sampled responses tend to collapse into the same semantic answer cluster. A high value means that

the model spreads probability across different answer clusters, so the prompt exposes more uncertainty for the current policy [18].

Curriculum learning follows the same logic at the prompt level. Prompt Curriculum Learning studies how prompt selection and batch configuration affect convergence in RL training. It reports a recurring insight from recent work: prompts of intermediate difficulty can improve data efficiency because they are neither already solved nor hopeless for the current policy [21]. The paper also emphasizes that identifying such prompts can be expensive if it relies on repeated online rollouts, so curriculum design has to balance selection quality against generation overhead.

These ideas are closely related to GRPO-LEAD's difficulty-aware advantage reweighting. GRPO-LEAD penalizes verbosity and changes how examples contribute to the update based on difficulty [14]. Entropy-aware methods, curriculum methods, and difficulty-aware weighting all point to the same research direction: reasoning-RL training should not treat every token, completion, or prompt as equally informative.

Entropy masking and semantic-entropy ordering are not presented as complete solutions to credit assignment or sparse rewards. They are bounded selective controls: one acts inside the response by changing which tokens receive stronger optimization pressure, and the other acts at the example level by changing training order. The literature justifies studying those controls, but it does not guarantee that they will help under a small group size, short response cap, and limited rerun budget.

4.5 RESEARCH GAP AND PROJECT POSITIONING

The current literature leaves a gap between large-scale reasoning-RL results and small, inspectable training studies. DeepSeek-R1 demonstrates the power of RLVR and GRPO-style training at a scale that uses large rollout groups, very long responses, and specialized infrastructure [1]. Tina shows that LoRA-based RL on DeepSeek-R1-Distill-Qwen-1.5B can produce meaningful reasoning gains at much lower cost [9]. Dr. GRPO, GRPO-LEAD, InT, entropy-aware methods, and curriculum work each isolate a different weakness: length bias, sparse rewards, weak credit assignment, entropy collapse, and inefficient prompt selection [13] [14] [17] [18] [21].

Less settled is how those concerns interact inside one compact experimental setting. Large training pipelines can report strong final performance without making it easy to separate which mechanism caused the improvement. Small studies can expose mechanisms more clearly, but they must accept narrower claims. A local GRPO-family experiment cannot settle whether RLVR scales to all reasoning domains. It can study whether a verifiable reward, a centered group-relative objective, response-length controls, entropy masking, and curriculum ordering produce coherent training dynamics under constrained conditions.

This creates the gap addressed by the project. It is not a new frontier-scale reasoning model and does not claim to reproduce DeepSeek-R1. It studies a compact distilled reasoning model, a verifier-compatible mathematical task family, a Dr. GRPO-inspired objective, and selective controls that are realistic within a local training budget. Its contribution is analytical: it connects reward design, objective choice, response length, entropy-based token selection, and curriculum ordering to observable training behavior.

Several boundaries remain open:

- **Process quality.** Outcome reward cannot fully explain whether the generated reasoning path is locally valid at each step.
- **Generalization.** A small held-out benchmark slice cannot prove broad transfer across reasoning domains.
- **Statistical strength.** Limited reruns cannot support the same statistical claims as a large multi-seed training campaign.

5. DATA, TASK AND LEARNING SETTING

5.1 BASE MODEL AND LEARNING TASK

The policy is initialized from DeepSeek-R1-Distill-Qwen-1.5B, a compact model from the DeepSeek-R1 distillation family. It is not a blank language model: it already contains general linguistic ability and distilled reasoning behavior before the reinforcement-learning stage begins. The learning question is whether a bounded GRPO-style post-training loop can adapt a small reasoning policy toward more reliable verifier-compatible mathematical behavior under a strict local hardware budget.

The model size is part of the design. A larger model would be closer to frontier reasoning systems, but it would also make the study incompatible with the single consumer GPU constraint. The 1.5B model is small enough to leave memory for LoRA adapters, grouped sampling, benchmark evaluation, checkpointing, and recovery logic, while still producing multi-step mathematical solutions. TINA provides a useful comparison point at this scale because it also post-trains the DeepSeek-R1-Distill-Qwen-1.5B family with LoRA-based reinforcement learning and treats small-model reasoning post-training as a serious research target [9].

The target behavior is mathematical reasoning with checkable final answers. The optimization problem is not open-ended dialogue quality, general helpfulness, or broad preference alignment. Each prompt asks the model to solve an arithmetic or GSM-style word problem, generate a reasoning trace, and end with a final answer that can be extracted by a deterministic verifier. The reward path stays auditable because the final answer can be checked without loading a separate reward model or asking a human judge to score every completion.

The task setting is chosen for its operational properties rather than only for its presence in the literature. The training loop needs prompts whose answers can be normalized, completions whose final answer can be extracted, and groups of sampled responses that produce reward contrast. Mathematical word problems satisfy those conditions while still forcing the model to generate multi-step traces. The local version is much smaller than the DeepSeek-R1 setting: it uses a 1.5B distilled policy, LoRA adaptation, a feasible rollout group, and a response cap measured in hundreds rather than tens of thousands of tokens [1].

The learning objective is deliberately narrow. Correctness is the main target, but the generated response must also remain extractable, bounded in length, and compatible with the verifier. A response that contains the correct number in an unstable position can make the reward signal ambiguous. A response that reaches the generation cap without a usable final answer consumes rollout budget while providing little or no contrast for the policy update. For that reason, the task is defined jointly by the dataset, prompt format, verifier, response cap, and measurement protocol.

The fixed learning constraints are the available hardware, rollout grouping, LoRA-only adaptation, and bounded generation length. Training is designed for one 8 GB consumer GPU. The accepted local operating profile uses grouped rollouts with a feasible group size and a maximum response length of 768 tokens, which was selected as the current compromise between premature truncation and the cost of longer generations. The cap is a runtime limit and a data-selection rule: it determines how much reasoning space the model has, which examples are suitable for training, and when a completion should be treated as unusable for the verifier.

5.2 DATASET SCOPE AND SELECTION CRITERIA

The primary data scope is GSM-style mathematical reasoning. GSM8K is the initial controlled benchmark because its examples match the reward assumptions cleanly: each item contains a short English arithmetic word problem and a final numeric answer that can be normalized for deterministic checking [22]. The project also evaluates GSM-style variants as later candidates when plain GSM8K becomes a weak main training target, especially after long runs show little stable movement.

This scope is intentionally limited. GSM-style data mainly represents grade-school arithmetic expressed as natural-language problems, with solutions that reduce to a numeric answer. That makes it suitable for exact verification, but it also limits the claims that can be made from it. The task is a controlled arithmetic-reasoning setting, not a test of broad dialogue ability, multilingual robustness, symbolic theorem proving, or long open-ended mathematical derivation.

GSM8K gives the first clean data contract: a prompt, a generated solution attempt, an extracted final answer, and a target value. The standard split sizes also make it practical as both a training source and a held-out benchmark, with 7,473 training examples and 1,319 test examples in the local loading path [22]. That balance justifies starting with GSM8K before admitting broader mixed benchmarks, whose labels, lengths, or solution formats can make the reward path harder to validate.

The controlled path separates optimization data from evaluation data. Training prompts supply the rollout groups used for GRPO updates. Held-out examples are used for benchmark and validation checks. This separation is modest compared with large multi-benchmark evaluation suites, but it is necessary: a post-training loop should not be evaluated only on the prompts that directly produced its policy gradients.

A dataset is usable only if it satisfies five conditions derived from the learning objective.

- **Verifier target.** The final answer must be normalizable into a target value that the deterministic verifier can compare with the generated answer.

- **Prompt budget.** The formatted prompt must fit the configured input budget after the tokenizer and chat template have been applied.
- **Response budget.** Most generated responses must have enough room to reach an extractable conclusion under the 768-token response cap.
- **Reward variation.** Rollout groups must contain enough variation for GRPO to compute a useful advantage signal; a group where every completion receives the same reward gives no useful comparison, even if the run is technically executing.
- **Split separation.** The dataset must provide, or allow construction of, a clean separation between optimization prompts and held-out evaluation examples. That requires enough examples, traceable preprocessing, and no obvious train/evaluation leakage through duplicated, near-duplicated, or reused prompts.

This keeps dataset selection tied to signal quality rather than to dataset prestige. Harder or more recent data is useful only if it remains compatible with deterministic verification, within-group reward contrast, the generation budget, and split separation. A simpler dataset can still be valuable when it gives stable, interpretable measurements of arithmetic reasoning under controlled conditions.

GSM8K therefore defines the initial task, verifier target format, held-out benchmark path, and comparison surface. Later dataset decisions follow the same compatibility lens. DAPO/Open-RS-style data was rejected as a main path after length filtering left only very small usable subsets under the 768 cap [23]. GSM-Plus then became a stronger candidate because it extends GSM8K with perturbed grade-school math problems while preserving the same broad arithmetic setting [24]. Locally, it also produced lower initial accuracy, acceptable extraction, and useful reward variation under the same response budget.

5.3 DATA PREPARATION AND PROMPTING

Training and evaluation use the same preparation path for each raw example.

1. **Question and answer extraction.** In the GSM8K path, the loader reads the natural-language question and the final numeric answer from the dataset record.
2. **Target normalization.** The answer text is reduced to the target value used by the deterministic verifier.
3. **Prompt rendering.** The question is passed through the same prompt-formatting helper used by training, preprocessing, and evaluation.
4. **Tokenization and length estimation.** The model receives the tokenizer-rendered prompt, and prompt length is estimated after formatting.

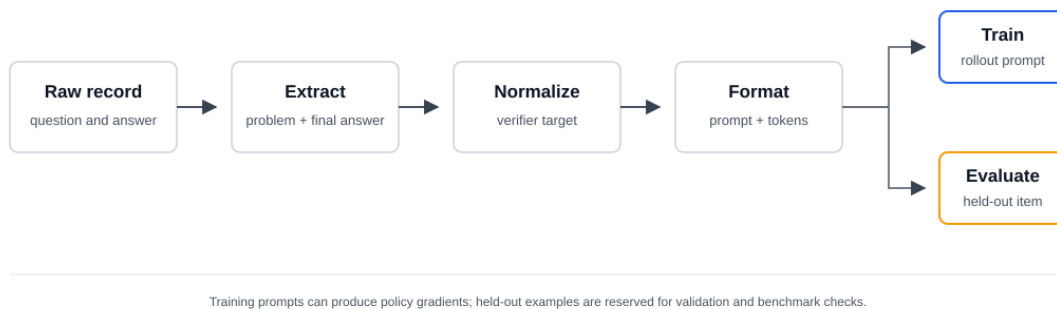


Figure 5.1: Data preparation and evaluation flow.

The prompt path is kept stable across training and preprocessing. The formatting helper applies the tokenizer chat template when one is available and falls back to a safe plain-text form when template rendering fails. A chat template can add tokens that are invisible in the raw question, so estimating length from the unformatted string would understate the true input budget and could lead to a misleading interpretation of truncation.

The data split policy follows the same principle of controlled measurement. Training examples provide rollout prompts for optimization. Held-out examples provide periodic benchmark checks and validation measurements. In the initial GSM8K path, training uses the train split and benchmark checks draw from held-out examples. The later normalized dataset path generalizes this into explicit train, validation, and test handling when a dataset provides those splits. The split names may vary by source, but the rule is fixed: reported evaluation quality must come from examples outside the rollout prompts used to update the policy.

Tokenization and response budgeting define the usable part of the task. The formatted prompt must fit the prompt budget, and the generated completion must fit the response budget. The accepted response cap is 768 tokens. Earlier response-length checks made 768 the working compromise: it leaves room for multi-step arithmetic without returning to the fragility observed at longer caps such as 1024. A dataset whose examples routinely require longer completions to reach an extractable answer is a poor fit for this configuration, even if the underlying mathematical content is relevant.

Preprocessing is conservative. Examples are normalized only as far as needed to produce a prompt, a tokenized input, and a verifier target. Hard filtering is avoided by default because it can change the task distribution. Filtering becomes justified only when a dataset or subset cannot provide usable reward signal under the configured response cap, for example because too many completions hit the cap before producing an answer or because the group rewards collapse to the same value too often.

Curriculum support is kept separate from dataset definition. SENT can reorder or stage access to examples according to semantic-entropy preprocessing, but that does not make an incompatible dataset compatible by itself. A curriculum can change the order in which examples are seen; it cannot replace the need for extractable answers, reward variation, split separation, and completion lengths that fit the local budget.

5.4 MEASUREMENT DEFINITIONS

Before later results are interpreted, the measurements separate task quality, verifier compatibility, training signal, and evidential strength.

Measurement family	What it measures	Why it matters	Caveat
Numerical correctness	The verifier extracts a final answer and compares it with the ground truth after numeric normalization.	Exact numeric matching is reproducible and auditable, so it is the primary task metric.	It is not a complete description of reasoning quality: intermediate steps may be wrong, and a plausible derivation can still fail if the final answer is not extractable.
Format compliance	Whether the model places its answer where the verifier can read it.	It separates mathematical failure from communication or extraction failure.	Better format compliance can improve measured performance without proving deeper mathematical improvement, while worse compliance can hide partial reasoning progress. Correctness and format therefore have to be read together.
Reward dynamics	Average reward describes how favorable completions are under the local reward definition; reward dispersion shows whether sampled completions differ enough for group-relative comparison.	These measurements show whether the training signal is informative or saturated.	Reward is the scalar signal used by GRPO, but it is not identical to final model quality. The exact reward composition and loss behavior belong to the optimization design.

Response length and truncation	Average response length tracks shorter or longer solution drift; truncation ratio records how often completions hit the response cap.	They show whether the dataset and response budget give the model enough room to produce evaluable answers.	They are training-health measurements, not direct quality measures.
Optimization-signal health	Zero-loss rows, zero reward variance, zero advantage variance, and completed optimizer updates.	They define whether two training artifacts can be compared before any quality claim is made.	A run can execute many rollout rows while learning little from them, and a run with fewer completed optimizer updates because of accumulation changes or repeated failures cannot be ranked directly against one with a different effective update budget.
Evidence category	Whether an artifact is an exploratory screen, repeated-seed check, final evaluation, or diagnostic failure.	Exploratory screens remove weak directions, repeated-seed checks test stochastic robustness, and final evaluations support stronger held-out quality claims.	Diagnostic failures are still evidence, but they explain why a dataset, cap, or configuration does not provide a usable learning signal under the local constraints rather than proving final model quality.

Table 5.1: Measurement vocabulary for training and evaluation.

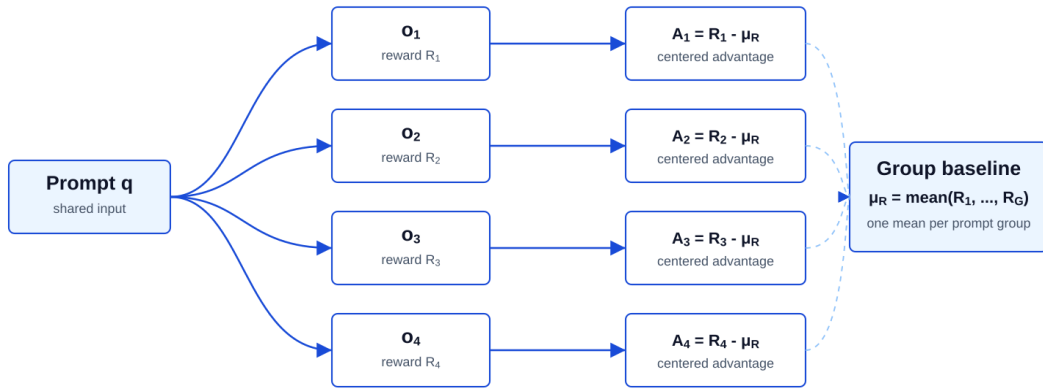
This vocabulary keeps the later analysis bounded. Correctness asks whether the model solves held-out problems. Format and extraction ask whether the verifier can read the output. Reward dynamics ask whether the training loop has usable signal. Length and truncation ask whether the response budget is compatible with the dataset. Evidence categories determine how much confidence can be assigned to each run. This keeps the evidence useful without overstating what a compact, constrained reinforcement-learning setting can prove.

6. GRPO IMPLEMENTATION AND REWARD DESIGN

6.1 IMPLEMENTED GRPO OBJECTIVE

The implemented optimizer starts from the previously defined GRPO mechanism: several completions are sampled for the same mathematical prompt, each completion receives a deterministic verifier reward, and the update is computed from the reward contrast inside that prompt group [1]. The local variant is narrower than a new reinforcement-learning family. It fixes the exact advantage, clipping and normalization choices used under sparse verifier rewards, bounded response length and a single-GPU execution path.

Figure 6.1 shows the group-relative signal path used by the objective. The verifier assigns one reward to each sampled completion, the prompt-local reward mean becomes the baseline, and each completion receives a centered advantage before the token-level policy update is computed.



The learning signal is relative inside each prompt group: identical rewards produce zero advantages.

Figure 6.1: Group-relative advantage flow.

For a prompt group of size G , the implementation centers each completion reward against the mean reward of its own group:

$$A_i = R_i - \frac{1}{G} \sum_{j=1}^G R_j.$$

The advantage is not divided by the group standard deviation. This follows the Dr. GRPO critique introduced earlier: under sparse or nearly binary rewards, standard-deviation scaling can turn small changes in group dispersion into large changes in update scale [13]. Uniformly correct or uniformly failed groups therefore produce zero advantages, because they contain no within-prompt preference signal.

The token update then uses the policy ratio inherited from PPO-style objectives, but applies it to the centered group advantage. For each selected response token, the loss compares the unclipped term with a clipped term:

$$L_i(\theta) = - \sum_t \min(r_i, t(\theta) A_i, \text{clip}(r_i, t(\theta), 1 - \epsilon, 1 + \epsilon_{\text{high}}) A_i) .$$

The asymmetric upper clipping bound and the ratio safety cap are update controls around the same surrogate. They limit unusually large token-ratio changes without changing the source of the learning signal: reward contrast inside a prompt group. Table 6.1 summarizes the local objective choices and the boundary attached to each one.

The other Dr. GRPO finding used here concerns response length. As explained earlier, per-response length normalization can change the effective incentive on generated length [13]. That matters in practical terms, not only as a loss-design detail. Long completions increase generation cost, raise truncation risk, and enlarge the sequence tensors handled during scoring and backpropagation on an 8 GB GPU. The implementation therefore avoids per-response loss averaging and controls length through the response cap, length penalty, truncation handling and diagnostics instead of hiding the length effect inside the loss denominator.

Implementation choice	Purpose in the local objective	Risk controlled	Boundary kept explicit
Centered group advantage without standard-deviation scaling	Keeps the within-prompt reward baseline while using the raw contrast between correct and incorrect completions.	Avoids an undefined or artificial scale factor when rewards are identical, sparse or nearly binary.	Uniform groups produce zero advantages rather than manufactured signal.
Asymmetric two-sided clipping and ratio safety cap	Limits excessive probability increases for positive-advantage completions and excessive reductions for	Prevents a compact run from being dominated by a small number of unstable token-ratio changes.	The cap is a safety control around the surrogate loss, not a new objective.

	negative-advantage completions.		
Global loss normalization	Keeps per-completion response length out of the policy-gradient denominator.	Avoids an implicit length incentive while keeping long responses visible as a memory, truncation and runtime risk.	Length remains controlled through response cap, optional length penalty, truncation mask and logged response-length metrics.
Optional reference and KL hooks outside the main memory-heavy preset	Keeps compatibility with reference-policy regularization without requiring full reference scoring during every update.	Avoids duplicating the policy model in the main single-consumer-GPU training path.	The implementation is Dr. GRPO-inspired, not a complete reproduction of the paper system.

Table 6.1: Local GRPO objective design choices.

The resulting objective is narrower than the full paper setting. It adopts the two bias-control choices that fit the local verifier setting, centered group-relative advantages without reward-standard-deviation scaling and global loss normalization, while leaving reference-heavy regularization outside the main training path [13]. The signal path stays inspectable: reward design determines the scalar signal, grouped centering determines the advantage, and clipping controls policy movement.

6.2 REWARD COMPONENTS

The implemented reward system turns the verifier principles introduced earlier into the scalar signal consumed by the GRPO loss. Its main signal is numerical correctness, supported by answer-format handling, response-length control and truncation policy. The design stays intentionally simple: it is cheap, reproducible and compatible with grouped rollouts, but it remains an outcome-level reward and therefore inherits the sparse credit-assignment limits discussed in the theoretical and literature chapters [1] [17].

6.2.1 Numerical correctness and answer extraction

The primary reward is binary numerical correctness. A generated completion receives a positive reward when the extracted answer matches the ground truth after normalization; otherwise it receives zero before any explicit length penalty is applied. Batch verification applies the same logic to

every completion in a group, so the reward tensor entering the GRPO objective has one scalar per sampled response.

The rule-based verifier follows a conservative extraction hierarchy:

1. prioritize structured final-answer regions when they are present;
2. accept boxed expressions as high-confidence answer markers;
3. fall back to equation-like tails and final numeric text only when safer markers are unavailable;
4. normalize extracted numbers before comparison with the ground truth;
5. apply incomplete-response and truncation guards before treating a reasoning tail as a final answer.

The order matters because the reward is used for training as well as reporting. A model may produce several intermediate numbers before the final answer. Rewarding the last arbitrary number in a partially generated trace would corrupt the signal, especially when the completion is truncated or syntactically incomplete. The conservative policy reduces false positives at the cost of occasionally missing a correct but poorly formatted answer. For training, that trade-off is preferable: a missed reward weakens a step, whereas a false positive teaches the policy that an invalid completion was acceptable.

Format handling supports this correctness reward rather than replacing it. Structured answer markers and boxed answers make verification less ambiguous, and they reduce the chance that an intermediate calculation is mistaken for the final answer. The reward path encourages a stable answer interface: reasoning may be long or short, but the final result must be recoverable by deterministic rules. In grouped RL, a small extraction inconsistency can change which completion receives the positive advantage inside a group.

The local reward design therefore treats format reward as verifier support, not as an independent style objective. A response is not rewarded for sounding polished, verbose or pedagogically complete. It is rewarded for making the final answer recoverable in the expected structure. This matters for interpretation: an improvement in format compliance can make correctness measurement more reliable, but it is not itself proof that the model has learned better mathematical reasoning.

6.2.2 Length penalty and truncation handling

Length control is implemented explicitly in the reward path. After the verifier assigns the correctness reward, the trainer can subtract a length penalty proportional to the number of generated response tokens:

$$R'_i = R_i - \lambda_{\text{len}} |o_i|.$$

This term does not claim that shorter responses are inherently better. Reasoning models often improve partly by producing longer deliberations, and DeepSeek-R1-style training reports increased response length alongside reasoning gains [1]. The implemented objective already avoids the per-response length normalization criticized by Dr. GRPO, so the length penalty is not correcting that loss-denominator bias [13]. It adds an explicit reward-side cost for verbosity under the local response cap and memory budget. The model may use enough tokens to solve the problem, but it should not receive an unbounded advantage merely for occupying the response budget.

Truncation is handled as a distinct training hazard. A completion that reaches the response cap may contain a correct intermediate number, an unfinished derivation or a final answer cut off by the token limit. The implementation computes a truncation mask for generated responses and can exclude truncated completions from the advantage calculation and loss contribution. When truncation masking is active, a capped sample does not distort the group mean or receive an update as if it were a complete answer.

Truncation also changes the statistical meaning of the reward. If capped completions are rewarded or penalized as ordinary complete samples, the optimizer may learn from malformed evidence. If all capped completions are ignored, the effective group size can shrink and the reward signal can become sparser. The implementation exposes that trade-off through configuration and metrics instead of hiding it inside the verifier.

6.2.3 Reward composition and sparse-signal risk

The final reward used by the advantage calculation is the verifier reward after explicit shaping terms such as the length penalty. Format compliance affects the reward through answer extractability and high-confidence final-answer detection rather than through a large independent style score. This keeps the reward aligned with the task: the generated solution must end in a correct, recoverable answer.

The main weakness is sparsity. Binary correctness produces many identical rewards when the group is too easy, too hard or too homogeneous. In a centered group-relative objective, identical rewards imply zero advantages. That is mathematically coherent, but it can slow learning because the policy receives no directional update from that prompt. The implementation logs reward mean, reward dispersion, positive-advantage ratio, response length and truncation statistics. These diagnostics make subsequent evaluation interpretable. A reward trace is meaningful only when the reward signal still has variance and the generated responses remain complete enough to verify.

Difficulty-aware weighting existed as an experimental setting. It could multiply the centered advantage by a factor derived from a difficulty proxy, following the GRPO-LEAD idea that prompts do not contribute equally to learning [14]. It was not included in the main reward definition or in the

accepted reward path. The training-campaign chapter explains why the exploratory run was kept outside the strict finalist comparison.

6.3 SELECTIVE OPTIMIZATION MECHANISMS

The reward is assigned to a whole completion, but the update is computed over a sequence of generated tokens. That mismatch creates a credit-allocation problem inside the local GRPO loop. A final-answer reward can say that one sampled response was better than another response in the same group, but it does not say which tokens inside the response carried the useful decision. Some positions are nearly deterministic continuations, while others correspond to branching points, arithmetic choices or answer commitments. Treating every response token as equally informative is a simple default, not a neutral one.

The implementation uses two selective mechanisms to make this allocation more explicit. Both respond to the same limitation of sparse outcome rewards: useful learning signal is unevenly distributed. Their effects must remain separated because they intervene at different points in the pipeline.

Mechanism	Unit selected	When it acts	What it changes	What it does not prove
Token-level entropy mask	Response token positions	After grouped completions have been sampled and scored.	Routes the GRPO loss through selected response tokens.	It is not a process reward and does not explain the semantic role of a token.
SENT curriculum control	Prompt examples	Before future grouped completions are generated.	Orders or stages examples according to semantic-entropy estimates.	It is not token-level credit assignment and does not measure absolute mathematical difficulty.

Table 6.2: Token-level and example-level selectivity.

6.3.1 Token-level entropy mask

The token-level entropy mask is the implementation counterpart of entropy-selection work that studies high-uncertainty minority tokens in reasoning traces [19]. It computes the entropy of the policy distribution at each shifted token position and builds a binary mask over the response portion of the sequence. Prompt positions are excluded because the objective updates the generated answer,

not the fixed conditioning text. The selected response positions are then passed into the GRPO loss, where they define the effective token set that contributes to the policy-gradient term.

This use of entropy is narrower than a global entropy bonus. The project does not try to make the whole policy more diffuse. It uses entropy as a local signal for positions where the current policy is less settled. Low-entropy positions are often formulaic or already decided by the model. High-entropy positions can contain real decision points, but entropy alone does not identify mathematical reasoning. It can also mark unstable formatting, repeated tail behavior or local uncertainty that has little value for the final answer. The mask is therefore a loss-routing mechanism, not a process reward and not a semantic explanation of the trace.

The mechanism also fits the training budget. Token entropy is derived from logits already produced by the model during the update. It does not require a learned process-reward model, a reference model kept active during every step, or additional human annotations. In a training loop already constrained by generation cost, old-policy scoring, optimizer state and response-length pressure, that difference is material. The intervention remains small enough to inspect after training: the reward stays completion-level, the grouped advantage is unchanged, and only the response-side loss geometry is altered.

The implementation supports both fixed and percentile variants because they encode different assumptions. A fixed threshold keeps the same absolute entropy cutoff across runs, which makes it easy to inspect. Its weakness is calibration drift: the same numeric threshold may select different kinds of positions as the model distribution changes. A percentile rule instead keeps a relative fraction of the highest-entropy response positions. That adapts to the batch, but it can select tokens even when the whole response is low-information. The code also enforces a minimum-token safeguard, so a mask does not accidentally remove every possible contribution in ordinary cases.

The threshold choice is not a separate reward formulation. It controls how much token-level uncertainty is allowed to shape the optimization path. A permissive mask approaches ordinary full-response training and weakens the point of selectivity. A strict mask concentrates the update, but can remove the surrounding trajectory that gives a token its role in the response. The adopted position is cautious: entropy masking is part of the accepted optimization path, but it is not justified by large speed-up claims. Its role is selective optimization and response-regime control under the bounded comparisons available locally.

The validation suite treats mask geometry as a correctness condition. The entropy mask must have the same response-aligned shape as the tensors consumed by the loss, must be zero over prompt positions, and must be accepted by the GRPO loss without shape or broadcasting errors. This is more than a unit-test detail. The GRPO loss is computed after shifting logits and targets, so an off-by-one mask can silently move loss from response tokens onto prompt tokens or select the wrong response positions. The tests pin the mask-to-loss contract before any training behavior is interpreted.

6.3.2 SENT as example-level control

SENT introduces selectivity at the dataset level. The preprocessing path is ordered:

1. sample several responses for each question under the chosen generation setup;
2. extract final answers from the sampled completions;
3. cluster responses by answer equivalence;
4. estimate semantic entropy from the answer-cluster frequencies;
5. write a cache that can order or stage examples from lower to higher semantic entropy.

A question whose sampled completions converge to the same answer has low semantic entropy. A question whose completions split across several answer clusters has higher semantic entropy.

This mechanism changes the training distribution rather than the loss formula. A prompt-level curriculum does not decide which response tokens deserve stronger pressure. It decides which questions are exposed to the grouped rollout process at a given stage. The grouped GRPO objective then operates normally on the selected slice. This distinction matters because curriculum effects can be confused with objective effects if both are described only as optimization improvements.

The local implementation follows the data-organization idea of SENT without claiming to reproduce every token-level regularization term from the paper. The paper combines semantic-entropy curriculum learning with token-level entropy-aware optimization [18]. The implementation uses the curriculum side as an example-selection mechanism and keeps the separate entropy mask as the token-selection mechanism.

Semantic entropy is a behavioral proxy for difficulty. It measures how dispersed the model's sampled final answers are, not how hard the mathematical problem is in an absolute sense. A prompt can be low-entropy because the model confidently repeats the same wrong answer. A prompt can be high-entropy because sampling produces several unstable answer forms without producing useful reasoning. The measure is still useful, but its scope is narrower than ground-truth difficulty estimation. It orders examples according to observed uncertainty under the current policy and sampling setup.

SENT preprocessing is cache-based because semantic entropy is expensive to compute online. The cache records the configuration, tokenizer-related metadata and completion status so that training can reject incompatible or incomplete caches. The training loop can also fall back to the standard dataset path if SENT is unavailable, depending on configuration. This prevents curriculum support from becoming a hidden dependency that invalidates the base GRPO path.

The interaction between SENT and the entropy mask is complementary but not interchangeable. A failure or success in one mechanism should not be attributed automatically to the other. An experiment may change prompt difficulty, token selectivity, response length and reward dispersion at

the same time, so the intervention boundary has to remain explicit. The reward path remains rule-based and completion-level. The GRPO path remains grouped and critic-free.

6.4 IMPLEMENTATION VALIDATION

The implementation is low-level enough that validation is part of the method. The training loop does not delegate GRPO, reward verification, curriculum preprocessing or entropy masking to a single high-level trainer. It needs targeted checks for the mathematical objective, reward extraction, data handling, benchmark behavior and reproducibility controls before empirical runs can be read as evidence about model behavior.

Table 6.3 groups the validation coverage by the kind of failure it controls. Every reported training trace must be interpretable as behavior of the implemented objective, verifier, curriculum path and benchmark machinery rather than as a shape bug, extraction bug or evaluation side effect.

Validation area	Checks covered	Risk controlled
GRPO algorithm	Reward grouping, exact centered advantages, zero advantages for uniform rewards, log-probability extraction, importance ratios, lower and upper clipping, ratio safety capping, unclipped and clipped surrogate terms, global normalization and full loss computation.	Protects the Dr. GRPO-inspired choices from accidental regression, especially centered rewards without standard-deviation scaling and global loss normalization.
Reward and verifier path	Boxed answers, numeric extraction, final-answer priority after reasoning blocks, exact matches, wrong answers, empty responses, batch verification, equation-style tails, decimal and currency-like values, lightweight LaTeX formatting, incomplete responses and truncated reasoning tails.	Keeps conservative training extraction intact so an intermediate number is not rewarded when the final answer is missing or the response ended abruptly.
Data loading and SENT curriculum	GSM8K prompt-length percentiles, tokenizer chat-template prompt cap, answer clustering, semantic-entropy computation for single and multiple clusters, cache saving and loading, metadata creation, formatted prompts, cache validation, stage slicing, checkpoint resume behavior and ascending entropy ordering.	Ensures curriculum stages follow the intended semantic-entropy order and cached preprocessing cannot silently drift away from the training configuration.

Selective optimization and edge cases	Mask shape against shifted logits, attention mask and target ids; prompt positions masked out; response positions selectable; GRPO loss compatibility; zero-length responses, very long responses, extreme advantages, negative rewards and single-token sequences.	Reduces the risk that training behavior is an artifact of tensor-shape mismatch, prompt-token updates or unhandled boundary conditions.
Benchmark path	Safe import without optional tracking dependencies, expected single-response generation contract, scoring of a generated response, and preservation of Python and PyTorch random-number generator states across initialization and execution.	Prevents evaluation from perturbing training state or changing generation semantics while results are being measured.
Run reproducibility controls	Explicit configuration, seed, output directory, checkpoint sequence, structured reward, length, truncation, loss and benchmark logs, checkpoint metadata, latest-step records, resume state and deterministic naming of related artifacts.	Makes comparisons auditable by tying reported metrics back to the configuration, checkpoint state and run context, even though stochastic training is not deterministic in a strict mathematical sense.

Table 6.3: Implementation validation coverage.

This validation layer does not prove final model quality. It proves the optimization machinery is internally coherent enough that later training runs can be interpreted as behavior of the implemented GRPO and reward system, rather than as obvious failures of loss math, reward extraction, curriculum preprocessing, mask alignment or benchmark side effects.

7. TRAINING CAMPAIGN AND EXPERIMENTAL ITERATIONS

The training process followed a staged experimental design, with each phase refining the next configuration. Each stage had a practical function: reduce the configuration space, check whether a short-run signal survived stronger validation, identify failure modes, and decide whether the next compute budget should go into longer training, curriculum, or dataset changes. The values reported here are campaign evidence. They explain promotion, demotion, stopping, and redesign decisions. Final model-quality claims are reserved for the frozen evaluation results reported later.

The campaign used the same measurement vocabulary throughout:

- **Training-reward behavior:** terminal training reward, reward trajectory, reward peak, and peak-to-final drop.
- **Response-control behavior:** response length, truncation, and drift toward capped completions.
- **Optimization-signal health:** selected-token ratio, zero-loss rows, reward variation, and completed optimizer steps.
- **External checks:** held-out benchmark snapshots where the artifact stream included them.

Configurations were compared through their training trajectories rather than a single final value. A useful run needed reward variation, sufficiently stable progress, and no drift into capped completions that made the GRPO update uninformative.

7.1 BASELINE AND QUICK-SCREEN STAGE

The baseline configuration was a starting point, not an attempt to copy one paper recipe onto an 8 GB consumer GPU. The closest small-model reference was Tina: it trains the same DeepSeek-R1-Distill-Qwen-1.5B family with LoRA-based reinforcement learning, AdamW, a cosine learning-rate schedule, gradient accumulation, and four generations per prompt [9]. Tina still runs in a larger hardware setting, with a lower learning rate, a longer completion budget, and a larger total batch than this project. DeepSeek-R1 gives the large-scale GRPO/RLVR reference, but its rollout group, batch size, and token budget are far beyond the local regime [1]. The defaults below therefore separate paper-backed choices from values reduced for memory and values introduced as local safety controls.

Default group	Initial value	Starting rationale

Base policy	DeepSeek-R1-Distill-Qwen-1.5B	Selected because Tina shows that the same compact reasoning-model family can be post-trained with LoRA-based RL, while the 1.5B size leaves room for grouped sampling and checkpoints on local hardware [9].
Quantization and memory mode	4-bit NF4, bf16 compute, double quantization, gradient checkpointing	Driven by the 8 GB GPU limit. Consumer-GPU LoRA/QLoRA profiling supports quantized adapter training as a practical route for 1.5B models under tight VRAM budgets [5].
LoRA adapter	rank 16, alpha 32, dropout 0.0, q/k/v/o projections	LoRA was the parameter-efficient update path [10]. Tina uses a larger default rank, but its ablation keeps rank 16 competitive, so the local run used the cheaper adapter setting [9].
Rollout group	group size 4	Kept as the smallest group that still gives within-prompt reward contrast. It matches Tina's generation multiplier, while DeepSeek-R1-style groups of 16 were outside the local memory and time budget [9] [1].
Batching	batch size 1, gradient accumulation 4	Used accumulation instead of a larger prompt batch to stay below the VRAM ceiling. Published reasoning-RL recipes use larger total batches, so the first screen included an effective-batch variant instead of treating this value as settled [9] [1].
Learning rate and optimizer	AdamW, learning rate 1e-4, weight decay 0.01	AdamW matches Tina, but the learning rate does not: Tina and DeepSeek-style runs report values around 1e-6 to 3e-6. The higher local value was an aggressive short-screen hypothesis, with a lower-rate sanity branch included because the literature suggested caution [9] [1].
Schedule and clipping	cosine schedule, 100 warmup steps, max gradient norm 1.0	The cosine schedule follows nearby LoRA-RL practice. Warmup and gradient clipping were conservative stability defaults for a noisy short run, not published optima [9].

GRPO clipping	epsilon 0.2, epsilon_high 0.3, delta 1.5	Clipped policy ratios come from the GRPO/PPO-family objective. The asymmetric upper clip follows the clip-higher idea used in recent RLVR work, while the local screen also tested a tighter upper bound to check stability [1] [19] [23].
KL path	configured coefficient 0.1, reference KL disabled	The coefficient remained configurable, but the main campaign disabled reference-model KL because keeping a separate reference path increased memory pressure. This is a local constraint divergence from larger GRPO recipes that include KL control [1].
Response budget	engine prompt preset 4096; response 1024 in the first screen	The prompt budget was deliberately large enough to avoid losing question text in GSM8K-style tasks. The first response cap was intentionally much shorter than large-scale reasoning budgets, but still long enough to expose truncation and verbosity failures before the later accepted 768-token operating profile [1].
Sampling	temperature 0.7, top-p 0.9, sampling enabled	Grouped GRPO requires diverse completions for the same prompt. The values are close to common reasoning-evaluation settings such as temperature 0.6 to 1.0 with nucleus sampling, but slightly restrained to reduce runaway long completions [1] [9].
Entropy mask	enabled, percentile 0.5, minimum 10 tokens	Motivated by entropy-aware RLVR work showing that high-entropy reasoning tokens carry disproportionate update value. The 0.5 threshold was a cautious midpoint, so the quick screen also tested lower, higher, and disabled variants [19] [18].
Length and truncation controls	length penalty 0.001, truncated completions masked	As explained earlier, Dr. GRPO avoids per-response length normalization in the objective, but local runs still needed reward-side verbosity control because capped completions can exhaust the 8 GB budget and remove useful verifier signal [13].

Curriculum branch	SENT disabled in the first screen; four semantic-entropy samples at temperature 1.0 reserved for later runs	The first screen isolated reward, entropy-mask, clipping, batch, and truncation controls before changing data order. The later SENT branch kept the paper's idea of sampling multiple responses at temperature 1.0 to estimate semantic uncertainty [18].
Screen length	120 trainer steps under a nominal 3-epoch cap	A local budget choice, not a published default. The step cap was long enough to reveal reward variation, truncation behavior, and inactive-loss cases before spending compute on repeated-seed or longer runs.

Table 7.0: Baseline hyperparameter rationale.

The first screen used 120 trainer steps and 10 configuration families. Its purpose was not to select a final model. It was a low-cost filter over the controls most likely to affect local GRPO training: learning rate, effective batch, entropy masking, entropy percentile, clipping, length penalty, and truncation masking. The baseline run established the reference behavior: terminal reward around the middle of the sweep, average responses around 425 tokens, and a truncation ratio just above 0.08. A candidate only qualified for the next phase when the terminal snapshot agreed with the rest of the diagnostics:

- no clear drift into the response cap;
- usable reward variation inside the grouped objective;
- response lengths that remained within a practical range;
- no evidence that the loss had become mostly inactive.

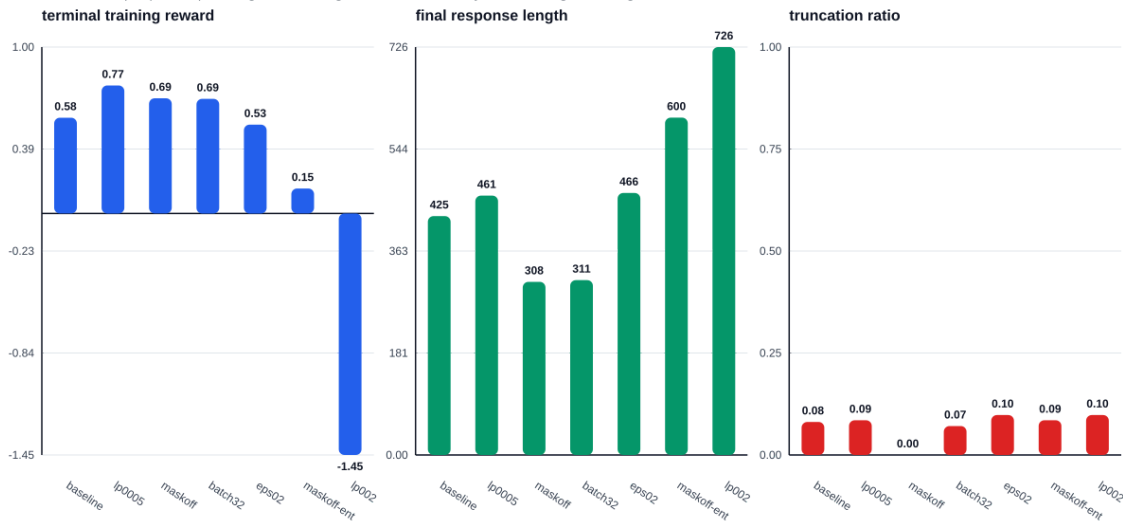
Runs with similar throughput were therefore separated by reward trajectory, response control, and whether the final rows still contained a meaningful group-relative signal. Table 7.1 reports the subset of values that shaped the next stage.

Configuration	Terminal training reward	Final response length	Avg truncation ratio	Throughput tok/s	Campaign decision
baseline	0.5752	424.8	0.081	65.75	reference
length_penalty_0005	0.7694	461.2	0.085	64.64	promote
mask_truncated_off	0.6920	308.0	n/a	66.47	promote as interaction candidate
batch_eff_32	0.6890	311.0	0.071	64.50	promote cautiously
eps_high_02	0.5340	466.0	0.098	66.09	keep as conservative candidate
entropy_mask_off	0.1500	600.0	0.085	63.65	demote
length_penalty_002	-1.4510	725.5	0.098	64.07	reject

Table 7.1: Quick-screen configuration outcomes.

CDIA Quick Screen Outcomes

The first sweep separated promising reward settings from truncation-heavy or reward-negative configurations.



Sources: 2026-04-16-cdia-benchmark-artifacts/benchmarks/output/cdia/results.json

Figure 7.1: Quick-screen terminal reward.

Figure 7.1 shows the terminal reward snapshot for the quick screen. The decisions after this screen depended on the whole diagnostic pattern:

- **Promoted:** the mild length penalty was promoted because its reward was higher than the baseline, its response length did not enter the severe cap regime, and truncation remained close to the baseline.
- **Kept as an interaction candidate:** disabling truncation masking was not adopted directly as a default, but it ended with short responses and no immediate loss-starvation signal.
- **Kept cautiously:** the larger effective batch shortened responses and did not fail mechanically, but it changed the number of optimizer updates under a fixed trainer-step budget. The higher clipping threshold also remained as a conservative candidate because it avoided the mechanical failures that eliminated other branches.
- **Demoted or rejected:** the 0.002 length penalty combined negative terminal reward with the longest final responses in the screen. Removing entropy masking produced longer responses and a worse terminal snapshot than the baseline, while the entropy-percentile variants and the lower learning-rate branch did not show a clear advantage.

The next phase therefore tested interactions rather than rerunning the top terminal reward number: no length penalty as a saturation contrast, mild length penalty, mild length penalty with truncation

masking disabled, mild length penalty with higher clipping, and batch-size variants that needed a fairer optimizer-step interpretation.

7.2 FINALIST SELECTION AND REPEATED-SEED RUNS

The finalist follow-up tested interactions among the quick-screen survivors. This phase changed the selection rule from "which branch looks plausible after one cheap screen" to "which branch still looks usable when the local controls are combined." The mild length penalty combined with truncation masking disabled became the strongest short-run candidate because its terminal reward stayed close to its peak, the peak-to-final drop was small, the final response length was 378.2 tokens, and the final truncation ratio was zero. The mild length penalty plus higher clipping also remained useful because its final state did not show the same collapse symptoms. By contrast, some variants with attractive earlier rewards collapsed to the response cap.

Configuration	Terminal training reward	Peak reward	Peak-to-final drop	Final response length	Final truncation	Optimizer steps	Decision
length_penalty_0	1.0000	1.0000 @1	0.0000	402.8	0.000	30	keep as reward-saturation contrast
length_penalty_0005_mask_truncated_off	0.8109	0.9095 @111	0.0986	378.2	0.000	30	promote
length_penalty_0005_eps_high_02	0.7404	0.8855 @56	0.1451	519.2	0.000	30	keep
length_penalty_00025	-0.0060	0.9562 @35	0.9622	1024.0	1.000	30	reject
length_penalty_0005_batch_eff_32	-0.5120	0.8727 @107	1.3847	1024.0	1.000	15	reject

Table 7.2: Finalist follow-up outcomes.

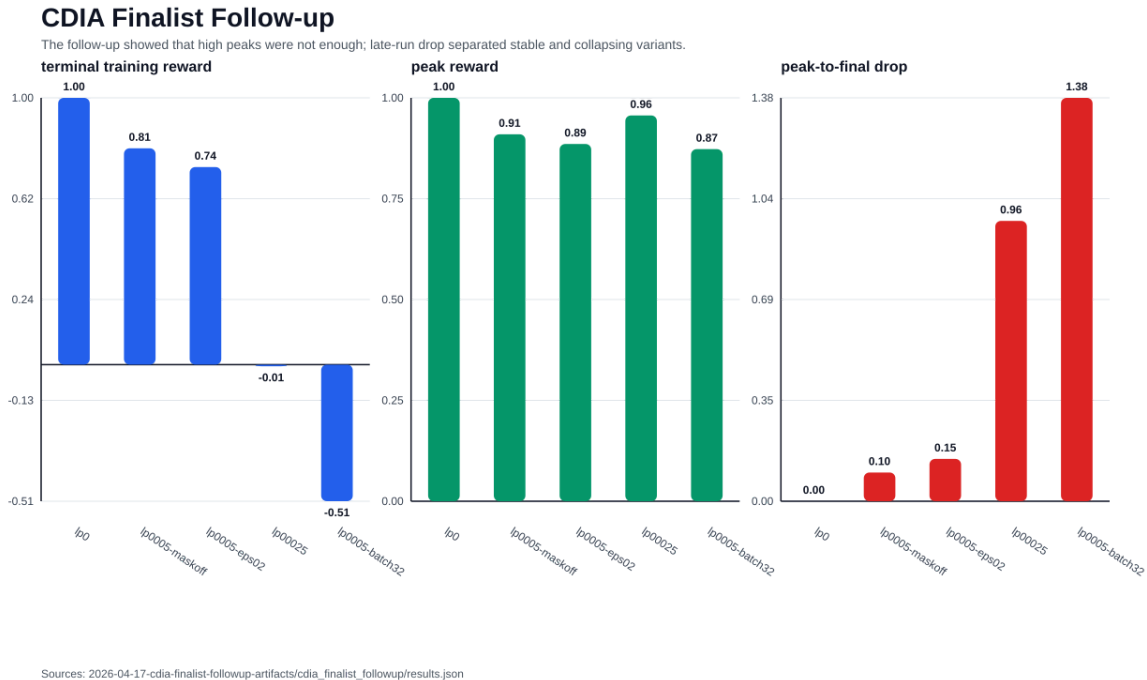


Figure 7.2: Finalist follow-up terminal reward.

Figure 7.2 makes the promotion rule visible. A high peak did not matter if the final state had already collapsed into capped responses. The strict comparison reduced the finalist space to `length_penalty_0` and `length_penalty_0005_mask_truncated_off`, but it still did not support declaring one clean winner:

- Capped high-peak branches:** `length_penalty_00025` and `length_penalty_0005_batch_eff_32` both reached attractive reward windows, but their final states were response-cap failures with full final truncation. The larger effective-batch branch was also less comparable because it completed only 15 optimizer steps under the same trainer-step budget.
- No-penalty family:** the follow-up ended with a perfect terminal reward, but the peak appeared at step 1 and the run spent long periods in a reward-saturated regime. In a centered group-relative objective, groups with identical rewards produce zero advantages. In the strict comparison, the two no-penalty seeds ended at 0.5000 and 1.0000 terminal reward, with final response lengths of 999.2 and 493.0 tokens.
- Mild-penalty mask-off family:** this family produced the best non-saturated candidate, but it was seed-sensitive. Its two strict-comparison seeds split sharply: one collapsed to 0.0277 terminal reward and 944.5 response length, while the other stayed usable at 0.7592 terminal reward and 481.5 response length.

- **Other follow-up branches:** plain length_penalty_0005 was discarded because it lost too much reward after its peak. The higher-clipping penalized variant remained useful as evidence, but it did not beat the mask-off interaction. The difficulty-weighted run was kept separate because its difficulty signal depended on SENT ordering. It therefore changed both advantage weighting and curriculum condition, so it could not be mixed into the strict finalist ranking.

The no-penalty family looked better on raw reward snapshots, but it also produced more zero-loss behavior and made reward saturation harder to interpret. The penalized mask-off family produced the strongest non-saturated candidate, but not with enough seed stability to settle the selection.

The endurance selector was launched because the previous phase answered the wrong question for final selection. It showed which branches were plausible, but it did not show whether the families survived a longer and more comparable budget. The selector design was therefore narrow:

- two remaining families: length_penalty_0 and length_penalty_0005_mask_truncated_off;
- three seeds per family;
- 240 trainer steps and 60 optimizer steps per run;
- held-out benchmark snapshots, because terminal training reward could no longer rank the families reliably.

Table 7.3 reports the final checkpoint held-out benchmark values from that selector.

Family and seed	Terminal training reward	Final response length	Final checkpoint held-out accuracy	Format compliance	Eval avg length	Campaign reading
length_penalty_0_seed1	1.0000	434.2	0.74	0.98	451.3	strong but reward-saturated
length_penalty_0_seed2	0.5000	765.8	0.76	0.98	456.7	usable, longer tail
length_penalty_0_seed3	0.7500	880.8	0.68	0.92	445.7	weaker held-out value

length_penalty_00 05_mask_truncate d_off_seed1	0.8148	370.5	0.76	0.98	440.9	strongest stable penalized seed
length_penalty_00 05_mask_truncate d_off_seed2	0.3720	756.0	0.80	0.98	454.4	best held-out snapshot, weaker terminal reward
length_penalty_00 05_mask_truncate d_off_seed3	0.0444	911.2	0.70	0.98	447.2	late degradation

Table 7.3: Endurance selector held-out snapshots.

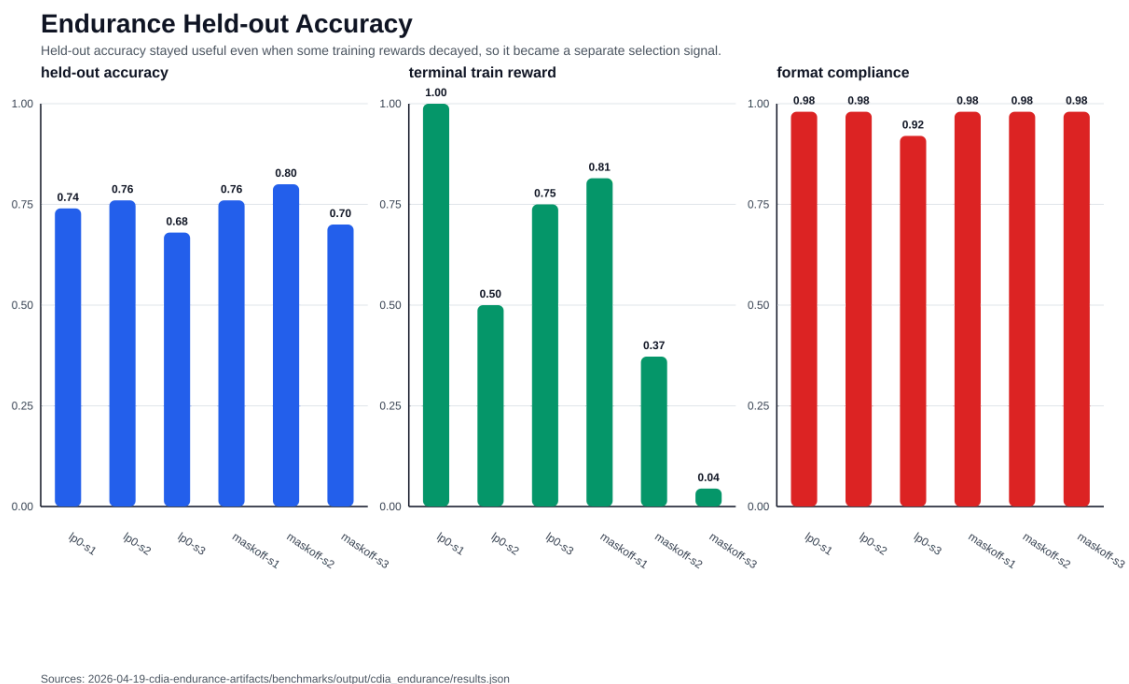


Figure 7.3: Endurance held-out accuracy.

The endurance stage did not produce a final model-quality claim; it produced the frozen finalist set. The penalized mask-off family had the best single final checkpoint held-out value (0.80) and one

strong stable seed, but also one late-degrading seed. The no-penalty family remained competitive and easier to compare as a contrast, but its training reward was less interpretable because reward saturation often removed the group-relative signal. The campaign search therefore did not select a single checkpoint from one scalar metric. It narrowed the experimental position: keep the no-penalty family as the saturated high-reward reference, keep the mild-penalty mask-off family as the best non-saturated finalist, and stop spending compute on broad hyperparameter expansion.

7.3 SENT AND STAGE-BASED TRAINING ATTEMPTS

The curriculum branch was launched after the finalist space had narrowed. Its purpose was to test whether a staged semantic-entropy curriculum could make the remaining training signal more useful than direct GSM8K training. The intended comparison was a two-stage curriculum against a matched non-curriculum control. Stage 1 was meant to expose the model to easier or cleaner examples before stage 2 shifted toward harder examples. That comparison never became clean, because the early curriculum attempts did not survive to the planned later stage under the original settings.

The sequence of attempts clarified the failure mode:

1. **Initial long-run attempt:** responses moved toward the cap, reward became dominated by length penalties or capped completions, selected-token ratios fell, and many rows reached zero loss.
2. **Retry with truncation masking restored:** this was launched because the earlier mask-off evidence suggested that capped completions could poison the update. Restoring the mask changed the failure mode but did not remove it. Masking capped completions prevented one kind of bad update, but when too many completions were capped it also removed most of the learning signal.
3. **Gate run with stronger response control:** a 0.001 length penalty and a 640 token cap attacked the same failure from the response-budget side. It delayed the failure but still degraded before the curriculum could be treated as a completed treatment.

The later artifact comparison corrected the attribution. SENT was not isolated as the cause of collapse. This was an inference from preserved runs, not a matched ablation:

- **Non-SENT collapse-like evidence:** a non-SENT finalist had already ended with 911.2 average response length and a weak terminal reward, and the length_penalty_00025 non-SENT follow-up reached full truncation with zero selected tokens and zero loss.
- **SENT evidence without the same hard-collapse tail:** the later lower-learning-rate run kept SENT enabled and did not enter the same terminal collapse pattern.
- **Interacting factors:** the evidence points to learning rate, response cap, length pressure, truncation masking, sparse verifier rewards, and dataset difficulty acting together, rather than to SENT alone.

The stage-2 continuations were still useful because they separated "stage schedule" from "stage-1 survival." A forced stage-2 continuation from the better checkpoint around step 2000 produced a temporary useful window and showed that the harder stage was not immediately impossible. It then collapsed later, with truncation and selected-token starvation becoming dominant. A lower-penalty no-mask continuation tested the opposite trade-off: keep more tokens in the loss by disabling truncation masking and weakening the length penalty. That kept selected tokens alive better, but held-out behavior degraded earlier and the run was not promoted. These continuations showed that stage choice could change the trajectory, but they did not produce a stable completed curriculum treatment.

Table 7.4 compares the main collapse and flat-run diagnostics.

Run	SENT	LR	Cap	Length penalty	Mask truncated	Tail avg length	Tail truncation	Tail select ed-token ratio	Zero-loss rows	Reading
non-SENT length_penalty_00025	no	1e-4	1024	0.00025	yes	485.0	0.102	0.349	8/120	ended at full cap and zero selected tokens
non-SENT lp0005_maskoff_selected3	no	1e-4	1024	0.0005	no	504.0	0.102	0.395	5/240	late response drift
SENT gate lp001_len640	yes	1e-4	640	0.001	yes	351.9	0.344	0.185	141/500	hard stage-1 degradation
SENT stage-2 continuation	yes	decayed from 1e-4	640	0.001	yes	592.3	0.879	0.022	446/500	hard collapse tail

lower-LR GSM8K flat run	yes	2e-5	640	0.0004	yes	452.8	0.183	0.448	67/500	flat but not hard-collapsed
-------------------------------	-----	------	-----	--------	-----	-------	-------	-------	--------	--------------------------------

Table 7.4: Collapse and flat-run diagnostics.

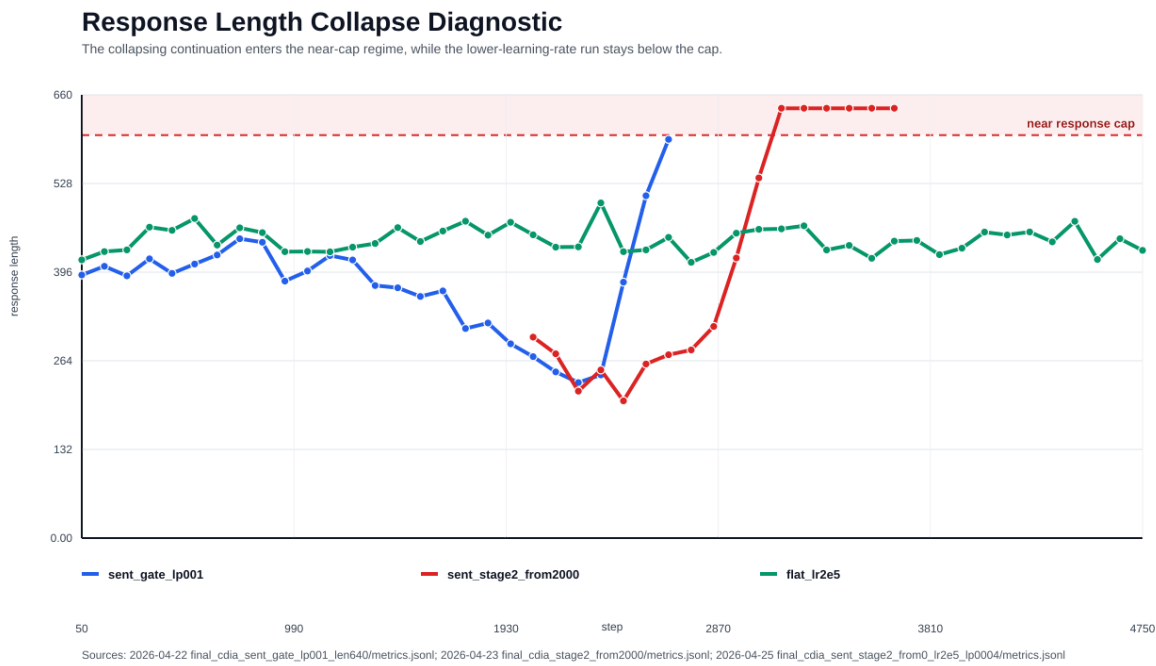


Figure 7.4: Collapse response-length trajectories.

Selected-token Collapse Diagnostic

The collapsing runs lose selected-token coverage, while the lower-learning-rate run keeps it alive.

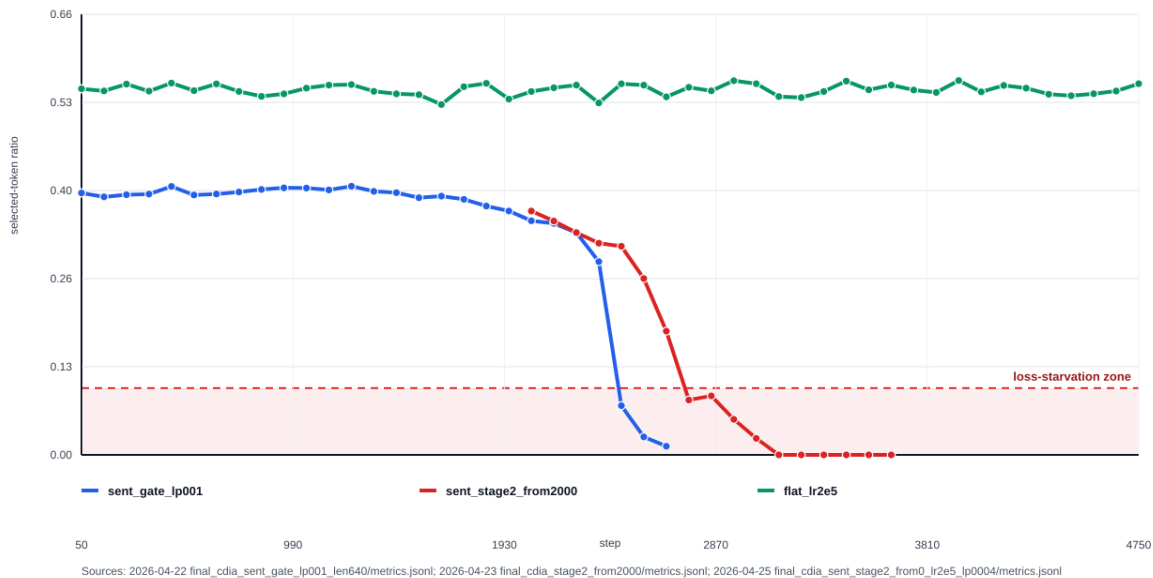


Figure 7.5: Collapse selected-token trajectories.

Figures 7.4 and 7.5 use 100-step medians rather than raw per-step values, because the raw batches are too noisy to show the long-run mechanism clearly. Their interpretation is deliberately limited:

- **Collapsed trajectory:** the stage-2 continuation moves into the near-cap response-length band and then stays there, while the selected-token plot falls below the 0.10 loss-starvation threshold.
- **Lower-learning-rate contrast:** the later GSM8K run stays outside that regime, with selected-token coverage still alive at the end.
- **Changed settings:** the lower-learning-rate run changed several knobs at once: learning rate dropped to $2e-5$, the length penalty was reduced to 0.0004, and the entropy percentile was raised to 0.7.
- **Mechanistic reading:** those changes plausibly reduced the hard-collapse pressure in different ways. The lower learning rate slowed policy movement after noisy rewarded batches, the weaker length penalty reduced the chance that long capped completions all received the same penalty-dominated reward, and the higher entropy percentile made the update more selective by keeping fewer, more uncertain tokens in the loss. Together, they made it easier for the run to keep nonzero selected-token coverage, even though they did not create a clear upward accuracy trend.
- **Inference boundary:** the later run did not prove model improvement; its held-out accuracy remained noisy and mostly flat. It did show why the earlier attribution had to be corrected:

SENT appears in both the collapsed and non-hard-collapsed long runs, while learning rate, length pressure, masking, and response-control settings change substantially.

Collapse Dynamics: Accuracy, Reward, and Loss Inactivity

Accuracy uses benchmark snapshots; reward uses 100-step medians; loss inactivity is the share of rows with zero loss.

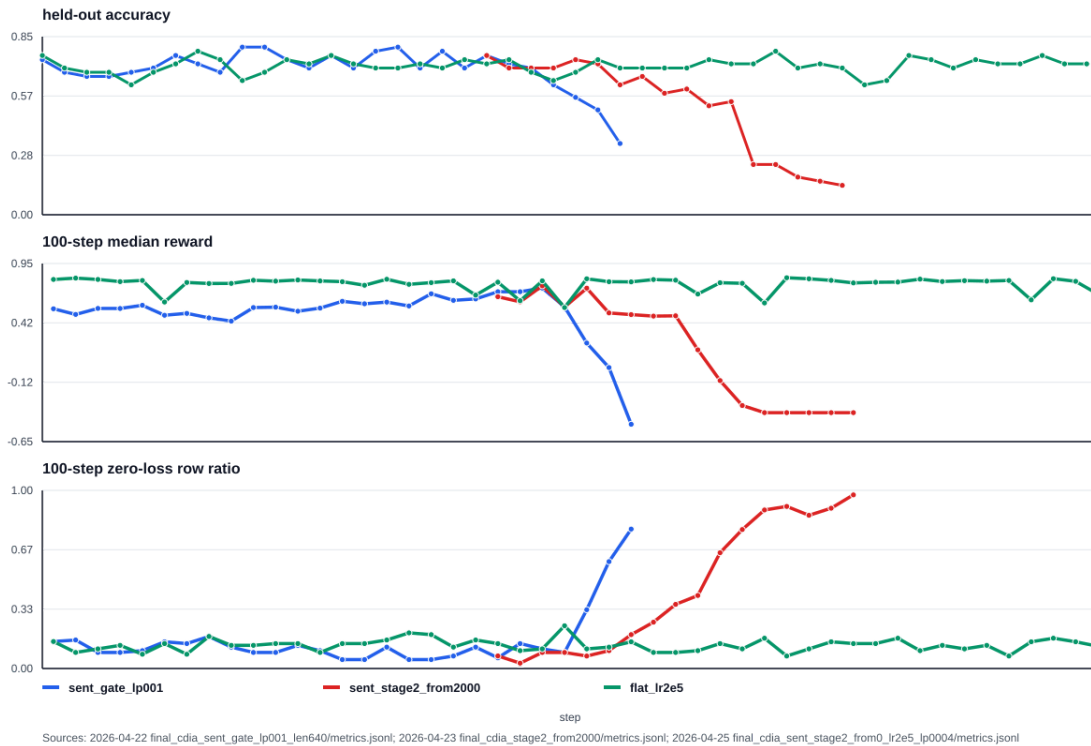


Figure 7.6: Collapse accuracy, reward, and loss inactivity.

Figure 7.6 adds the accuracy, reward, and loss-inactivity view for the same long-run metric streams. The hard-collapse continuations show validation accuracy falling as the tail moves into capped or near-capped behavior, while the lower-learning-rate run keeps accuracy in a narrower band. The reward panel uses 100-step medians and shows the same late deterioration for the collapsed continuations. The loss panel is expressed as zero-loss row ratio, because raw loss has large outliers and does not show the failure as clearly as the share of rows where the update has become inactive.

The practical conclusion is bounded. The curriculum is not a completed final treatment result, and the evidence does not isolate it as the cause of collapse. Its value in the campaign is diagnostic: it exposed how fragile the local objective became when response length, masking, sparse rewards, learning rate, and dataset difficulty interacted. For that reason, the branch was not used as a final winner. It was used to decide that another long GSM8K-only run under similar conditions would be a poor use of time unless the data distribution changed.

7.4 DATASET PIVOT AND LATER DIAGNOSTIC RUNS

The failed and flat late runs raised a data question. GSM8K was a practical starting point because it is verifier-compatible, compact, and common in reasoning evaluation. It also became hard to improve because the distilled base model was already strong on the local GSM8K slice. The lower-learning-rate GSM8K run reached trainer step 4715 and optimizer step 295, but periodic held-out accuracy still oscillated rather than trending upward. That run was important precisely because it did not collapse: once the hard failure was removed, the remaining problem looked like weak learning signal rather than insufficient survival time.

That behavior changed the campaign objective. Another long GSM8K-only run with minor low-level changes was unlikely to add much evidence. The project needed to test whether broader or harder math data could provide more useful reward variation while still fitting the response cap and verifier.

The first candidates were Open-RS and DAPO-style data. They were attractive because related exact-model work uses broader reasoning datasets rather than plain GSM8K-only training. They were not promoted because the local constraint was response feasibility, not dataset prestige. Under the local response cap, the first full-dataset attempts produced too many capped completions and too many groups with identical length-penalty-only rewards. In the Open-RS diagnostic, 45 of the first 55 rollout rows had zero loss. The repeated pattern began when all four sampled completions reached $\text{max_response_length} = 768$, no valid final answer was extracted, and every completion received only the length penalty:

$$\text{reward} = -(768 \times 0.0004) = -0.3072$$

The zero-update mechanism then followed directly:

1. All four rewards in the group became -0.3072.
2. The group-relative baseline removed the whole signal.
3. Reward standard deviation became zero.
4. Advantage standard deviation became zero.
5. The positive-advantage ratio became zero.
6. The GRPO policy loss was also zero.

The trainer was still spending time generating tokens, but those batches were not changing the policy. Figure 7.7 summarizes this early Open-RS diagnostic and shows why the branch was stopped as a dataset-feasibility failure rather than treated as a failed optimization schedule. A broader dataset was only useful if enough prompts could finish inside the local cap or, at minimum, if capped and non-capped completions still produced different rewards.

Open-RS Admission Failure Under the 768-Token Cap

Early live observation: 45 of 55 rollout rows had zero loss after all four sampled completions hit the response cap.

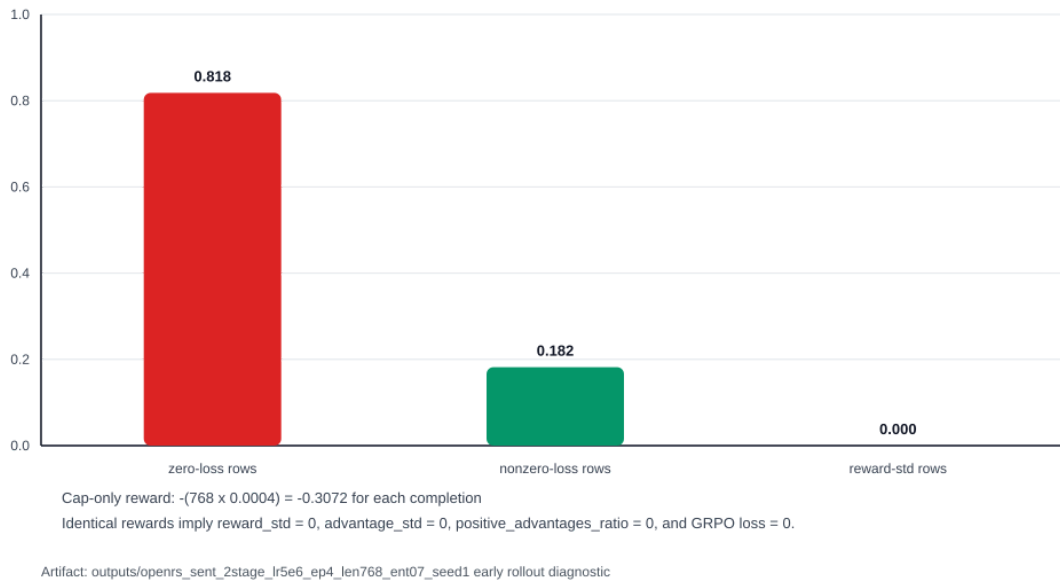


Figure 7.7: Open-RS signal failure under the token cap.

Length-only admission filtering was then tested as a dataset-feasibility probe. The filter sampled completions before training and kept prompts whose responses reliably finished under a stricter cap. This was not a new reward method and was not allowed to become the central result, because it would have changed the dataset into a convenience subset. It was a feasibility test for whether the dataset could supply enough prompts with usable response lengths. The result was negative for the main path: strict filtering retained too few examples, and relaxed filtering still left an insufficient subset.

The final pivot moved toward intermediate GSM-style datasets. The issue was not the name of the benchmark, but the kind of signal it could provide. GSM8K had become a weak main training target because the later low-learning-rate run survived without producing a clean upward accuracy trend. DAPO and Open-RS were closer to recent large-scale reasoning-training recipes, but the local 768-token cap turned too many groups into identical capped completions with no useful advantage signal. The next dataset had to satisfy four conditions:

- be harder than GSM8K;
- remain compatible with the existing numeric verifier;

- stay short enough for the local response budget;
- produce enough reward variation to make GRPO updates meaningful.

The admission probe compared GSM8K, GSM-Plus, GSM-Hard, SVAMP, ASDiv, and MultiArith under the same sampling regime: 100 examples per dataset, 4 completions per prompt, temperature = 1.0, top_p = 1.0, and max_response_length = 768. GSM-Plus was included because it is a GSM8K-derived robustness benchmark built from systematic perturbations of grade-school math problems [24]. The first version of the probe exposed extraction problems in the easier arithmetic datasets, so the comparison was repeated after normalizing SVAMP/ASDiv-style prompts and numeric gold answers. Table 7.5 reports the corrected aggregate values used for the decision.

Dataset	Accuracy	Extraction rate	Avg response length	Truncation ratio	All completions under cap	At least 3/4 under cap	Nonzero reward-std ratio	Campaign reading
GSM8K	0.575	0.792	534.5	0.307	0.510	0.620	0.470	useful baseline, weak next training target
GSM-Plu s	0.398	0.825	537.2	0.245	0.580	0.720	0.460	selected for training
GSM-Har d	0.205	0.738	610.3	0.367	0.400	0.540	0.340	hard but too truncation-heavy
SVAMP	0.802	0.968	322.0	0.025	0.940	0.980	0.350	feasible but too easy

ASDiv	0.760	0.902	398.0	0.172	0.730	0.790	0.320	feasible but too easy
MultiArith	0.973	0.998	294.0	0.007	0.980	0.990	0.070	saturated

Table 7.5: Corrected dataset admission probe.

The table changed the candidate ranking:

- **Rejected as too easy:** SVAMP, ASDiv, and MultiArith fit the response budget, but after the extraction fixes they no longer supplied enough difficulty. MultiArith was almost saturated at 0.973 accuracy, and its reward-variation ratio was only 0.070. SVAMP and ASDiv were less saturated, but their accuracies of 0.802 and 0.760 still placed them closer to auxiliary sanity checks than to main training data.
- **Rejected as too truncation-heavy:** GSM-Hard gave the opposite result. Its accuracy of 0.205 showed real difficulty, but the average response length rose to 610.3, the truncation ratio reached 0.367, and only 0.540 of prompt groups had at least three of four completions below the cap. That profile was too close to the capped-completion failure already observed in DAPO/Open-RS.
- **Selected with risk:** GSM-Plus sat in the useful middle. Its accuracy was lower than GSM8K (0.398 against 0.575), so it offered more room for improvement. It also stayed close enough to GSM-style arithmetic for the existing final-answer extractor and verifier to remain useful. Its extraction rate was slightly better than GSM8K (0.825 against 0.792), its truncation ratio was lower (0.245 against 0.307), and the share of groups with at least three under-cap completions was higher (0.720 against 0.620).
- **Reward-variation check:** the GSM-Plus reward-variation ratio was almost the same as GSM8K (0.460 against 0.470), which mattered more than the all-under-cap value. A GRPO group can still produce a learning signal when some completions truncate, as long as the rewards do not all collapse to the same length-penalty value.

A larger GSM-Plus-only probe confirmed that the choice was usable but still risky. In that probe, GSM-Plus reached 0.454 accuracy, 0.827 extraction, 540.4 average response length, 0.254 truncation, 0.528 all-under-cap groups, 0.678 groups with at least three completions under the cap, and 0.440 nonzero reward-std ratio. The p90 response length was exactly 768.0, so the dataset was not comfortably short. It was accepted because the remaining signal was not dominated by identical capped groups.

GSM-Plus was not hard-filtered before training because filtering could have reduced wasted rollouts at the cost of removing many of the harder prompts and making the dataset closer to the easier arithmetic sets. The first main attempt therefore used raw GSM-Plus, with stop checks based on zero-loss rows, repeated zero reward standard deviation, repeated zero advantage standard deviation, and truncation above roughly 0.35.

This was the final campaign transition. The search stopped asking whether another small change to the GSM8K recipe would be enough. It instead changed the source of reward variation while keeping the same verifier family, the same local response-budget discipline, and explicit failure checks for the capped-completion pattern that had made DAPO/Open-RS infeasible.

7.5 EXPERIMENT TRACEABILITY

The campaign remained interpretable because every stage preserved artifacts with configuration and metric provenance:

- the quick screen, finalist follow-up, final validation, and endurance selector provide results.json, report.md, suite plans, per-run configuration files, and metric streams;
- the long-run and continuation branches preserve config.json, metrics.jsonl, and checkpoint metadata where available.

This traceability matters because the campaign compares runs with different roles:

- quick screens are filters;
- finalist suites test whether a short-run signal survives interaction and seed variation;
- SENT attempts are diagnostic curriculum evidence;
- dataset probes are feasibility evidence, not model-quality evaluations.

A run can therefore be empirically important even when it is not a final checkpoint.

8. RESULTS AND MODEL EVALUATION

8.1 EVALUATION PACKAGE AND CHECKPOINT SET

The final evaluation package compares the original DeepSeek-R1-Distill-Qwen-1.5B policy with the LoRA adapters produced by the GSM-Plus training campaign. The evaluated adapters are small rank-16 updates on the attention projection matrices q , k , v and o . They do not replace the frozen 1.5B backbone. The largest adapter contains 4,358,144 bfloat16 (BF16) parameters, roughly 0.29% as many parameters as the base model. The empirical question is therefore specific: whether a small GRPO-trained adapter changes mathematical-reasoning behavior under these prompts, datasets and response limits.

The evaluated adapters come from the final GSM-Plus training pair. The preceding campaign had fixed the practical setting: GSM-Plus supplied the final training data as the GSM8K-derived robustness target, the response cap remained 768 tokens, and the lower-rate configuration with length penalty 0.0004 was used to avoid the hard-collapse pattern seen in earlier long runs [24]. The SENT branch kept semantic-entropy curriculum ordering enabled. The no-SENT branch disabled that ordering while retaining the same base model, LoRA setup, verifier family, group size, learning rate, entropy mask and response-length controls. The final comparison therefore evaluates the base model, the SENT-trained adapters and the no-SENT endpoint under the best locally selected GRPO settings.

The comparison uses three evaluation views. The first is a broad deterministic benchmark over twelve datasets with an 8192-token generation budget. The second is a stricter sampled benchmark over four core mathematics datasets, with eight generations per example and the same 8192-token budget. The third repeats the four-dataset sampled evaluation with the training prompt style and the 768-token response cap used during training. These views are not interchangeable. The broad deterministic run measures breadth under one decoded answer per example. The sampled run measures solution coverage and stability. The prompt-matched run is the closest final stress test for the actual local training regime. The mathematics rows include GSM8K for grade-school arithmetic and Math500 as a MATH-derived row [22] [25].

Model key	Meaning	Evaluation availability	Role in the comparison
base	Original DeepSeek-R1-Distill-Qwen -1.5B model before local GRPO training.	All three evaluation views.	Reference policy.
sent_best	Validation-selected checkpoint from the GSM-Plus SENT run.	All three evaluation views.	Checkpoint-selection and curriculum evidence.
sent_final	Final adapter from the completed GSM-Plus SENT run.	All three evaluation views.	Endpoint result for the curriculum treatment.
no_sent_final	Final adapter from the closest GSM-Plus run without SENT.	All three evaluation views.	Main non-curriculum control.
no_sent_best	Validation-selected checkpoint from the non-SENT branch.	Broad deterministic evaluation only; not carried into sampled runs.	Secondary checkpoint-selection contrast; the broad sweep did not improve over no_sent_final.

Table 8.1: Final evaluation checkpoint set.

The benchmark set was fixed before analysis. The selected checkpoints come from the training and validation process, not from searching the final benchmark for a favorable row. `sent_best` is a validation-selected checkpoint, while `sent_final` and `no_sent_final` are endpoint adapters. The non-SENT best checkpoint was evaluated in the broad deterministic sweep, but it was not promoted to the sampled runs because its macro average was below the no-SENT final adapter, 0.4660 against 0.4724. Table 8.2 records the three result views used for the final comparison.

Benchmark view	Snapshot	Protocol and cap	Datasets	Model keys
Broad deterministic reasoning evaluation.	reasoning_eval_1day_20260504-155349.	One deterministic generation per example; 8192-token generation budget; seed 42.	Twelve datasets: GSM8K, GSM-Plus, SVAMP, Math500, AIME 2024, AIME 2025, AMC 2023, Minerva, GPQA-Diamond, ARC-Challenge, HumanEval and MBPP.	base, sent_best, sent_final, no_sent_final and no_sent_best.
Core sampled reasoning evaluation.	reasoning_eval_core_sampled8_20260505-102324.	Eight samples per example; 8192-token generation budget; seed 46.	GSM8K, GSM-Plus, Math500 and AIME 2024.	base, sent_best, sent_final and no_sent_final.
Prompt-matched 768-token sampled evaluation.	reasoning_eval_core_sampled8_trainprompt_length768_20260506-130523.	Eight samples per example; training prompt; 768-token response cap.	GSM8K, GSM-Plus, Math500 and AIME 2024.	base, sent_best, sent_final and no_sent_final.

Table 8.2: Final benchmark inventory.

The measured metrics are exact final-answer correctness, pass at k , average correctness across k samples, response length, truncation and format compliance. For sampled evaluations, pass at k records whether at least one of the eight sampled generations solves the example. Average at k records the mean correctness of the eight samples and is therefore the more direct stability measure. Pass at 1 records the first sampled answer and is closest to ordinary single-response use.

The evaluation does not claim broad mathematical superiority, human-level reasoning, or state-of-the-art performance. It reports behavior under the preserved local artifacts: the fixed

checkpoint set, the selected datasets, the response caps, the verifier and the prompt templates used by the benchmark jobs.

8.2 TRAINING-RUN EVIDENCE BEHIND THE EVALUATED ADAPTERS

The training-run records add the runtime and health context behind the checkpoint set used in the evaluation package. The SENT run completed the two-stage curriculum and produced both a validation-selected checkpoint and a final adapter. The no-SENT run kept the same local training regime without curriculum ordering, making it the closest non-curriculum counterpart. The logs provide health and cost evidence; final model quality is judged through the benchmark results.

Run	Logged training window	Duration signal	Mean step time	Final local test evidence	Main limitation
GSM-Plus SENT	Steps 1 to 9496.	87.13 h total, 85.68 h training-step time.	31.65 s.	0.5234 exact accuracy on 256 GSM-Plus test examples; mean length 487.18; format compliance 0.8789.	Stage 2 is much harsher than stage 1.
GSM-Plus no-SENT	Run ending at step 9496.	75.32 h logged.	31.10 s.	No matching final test row in the pulled metrics.	Training logs do not include a matched held-out test row.
Final training-prompt evaluation	10560 generated samples across four models and four datasets.	20.34 h recorded generation latency.	Not a training run.	Prompt-matched sampled comparison under the 768-token cap.	AIME fully truncates for every model.

Table 8.3: Training and evaluation cost evidence.

Together, these timings explain the narrow final campaign. With the available local hardware and project schedule, another serious run would have cost tens of hours before it could even be evaluated, and the final sampled benchmark alone added more than 20 hours of recorded generation

latency. For that reason, the evidence base is a small set of preserved, measured comparisons rather than a wider grid of extra seeds, datasets, response caps or ablation branches.

The SENT run ended successfully, but its internal dynamics were uneven. Stage 1 averaged reward 0.4305, response length 474.54 and truncation 0.1077. Stage 2 averaged reward only 0.0071, response length 584.84 and truncation 0.3021.

The no-SENT counterpart matters because it removes the curriculum intervention while retaining all the other settings. Its epoch summary reports average reward 0.2205, response length 529.09 and truncation 0.2064 over the pulled training window. That profile sits between the easy and hard SENT slices: healthier than SENT stage 2, weaker than SENT stage 1, and not enough by itself to decide final model quality without the later reasoning-evaluation package.

Earlier long-run diagnostics explain why stability is reported alongside accuracy. In those runs, high truncation left too few usable completion tokens for the masked GRPO objective, and late windows became dominated by zero-loss updates. The final runs therefore have to be read through reward, truncation, selected-token ratio and held-out behavior together, not through accuracy alone.

8.3 BROAD DETERMINISTIC REASONING EVALUATION

The broad deterministic benchmark evaluates five variants across twelve datasets: GSM8K, GSM-Plus, SVAMP, Math500, AIME 2024, AIME 2025, AMC 2023, Minerva, GPQA-Diamond, ARC-Challenge, HumanEval and MBPP. The protocol uses one deterministic generation per example, up to 100 examples per dataset, seed 42 and an 8192-token generation budget. The run produced both model-level and dataset-level summaries. It gives a wide post-training snapshot, but it does not measure sampled robustness.

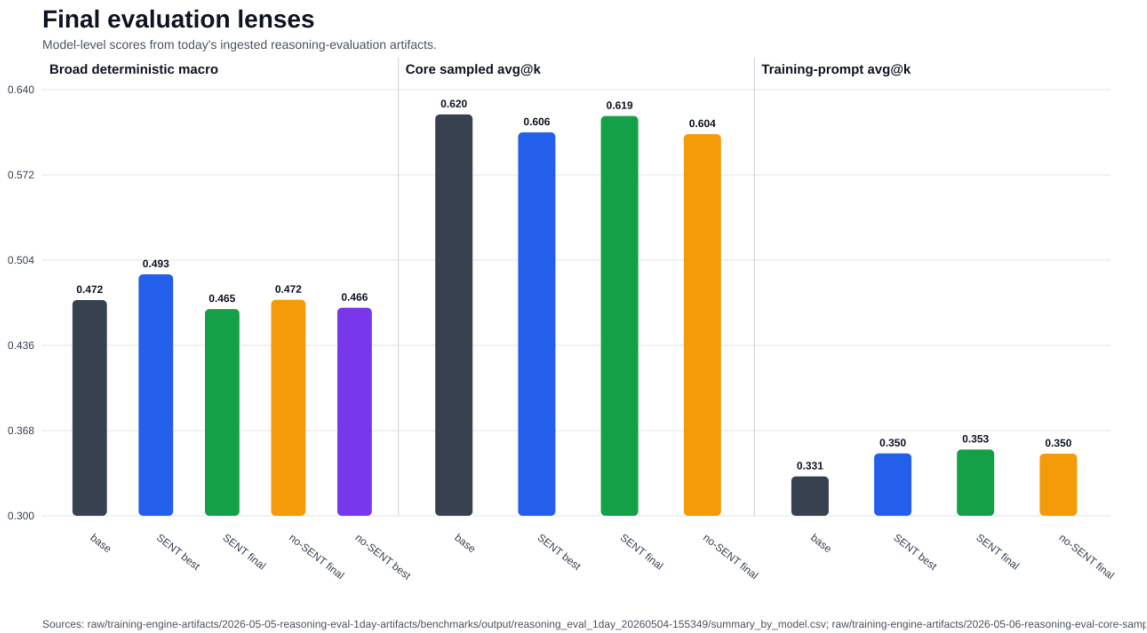


Figure 8.1: Final evaluation lenses.

Figure 8.1 compares the three model-level result lenses. The deterministic panel favors sent_best. The sampled panels are more cautious and show why endpoint and checkpoint-selection claims must remain separate.

Model	Macro accuracy	Generated tokens	Deterministic result
sent_best	0.4927	3,912,304	Best model-level score in this view.
no_sent_final	0.4724	3,942,196	Strong non-curriculum endpoint.
base	0.4722	3,965,761	Nearly tied with no_sent_final.
no_sent_best	0.4660	3,997,622	Below the final no-SENT endpoint.

sent_final	0.4650	3,939,688	Weakest macro score in this view.
------------	--------	-----------	-----------------------------------

Table 8.4: Broad deterministic benchmark by model.

The deterministic result supports a checkpoint-selection claim. The best SENT checkpoint is not the final SENT endpoint; it is the validation-selected checkpoint. The final SENT adapter is the weakest macro variant, so the broad benchmark cannot be used to claim that the curriculum endpoint cleanly beats the non-curriculum endpoint. The same model-level table also shows that the selected SENT checkpoint is not winning by spending more tokens: it has the lowest weighted truncation rate in this broad run, 0.3269, and the highest weighted format compliance, 0.6376. Those values are still modest because the benchmark includes hard rows with heavy truncation, but they make the checkpoint result cleaner than a pure accuracy-only reading. The dataset-level summary in Table 8.5 shows that the same pattern appears in different forms across the benchmark family: sent_best is often the strongest single checkpoint, but the final no-SENT endpoint remains competitive and several rows are ties or near ties.

Dataset	Base accuracy	Best deterministic row	Main reading
GSM8K	0.80	sent_best and no_sent_final tie at 0.82.	Small gain over base, but sent_final falls to 0.75.
GSM-Plus	0.52	sent_best, sent_final and no_sent_final tie at 0.55.	The target family improves, but this row does not isolate the curriculum.
SVAMP	0.89	base and sent_best tie at 0.89.	No trained endpoint advantage on this easier arithmetic row.
Math500	0.66	sent_final leads at 0.72.	This row gives the clearest deterministic endpoint gain.
AIME 2024	0.1667	sent_best leads at 0.2333.	A small checkpoint gain under very high truncation.

AIME 2025	0.20	sent_best leads at 0.2333.	Same stress pattern as AIME 2024, not a robust endpoint result.
AMC 2023	0.2892	sent_best and no_sent_best tie at 0.3253.	Checkpoint selection matters more than final endpoint selection.
Minerva	0.76	sent_best leads at 0.77.	The difference is too small for a strong claim.
GPQA-Diamond	0.18	no_sent_final leads at 0.19.	No SENT advantage and heavy length pressure.
ARC-Challenge	0.53	sent_best leads at 0.57.	A non-math transfer signal for the selected checkpoint.
HumanEval	0.58	sent_best leads at 0.60.	A small coding-row gain, secondary to the math objective.
MBPP	0.09	sent_best leads at 0.10.	Very low absolute accuracy, so the difference is weak evidence.

Table 8.5: Deterministic dataset-level review.

The broad benchmark also includes hard datasets where truncation is high even with an 8192-token budget. AIME and GPQA rows are evidence about stress behavior, not standalone proof of improved mathematical reasoning under the local adapter. The coding and science rows broaden the evaluation, but they are secondary because the training task and verifier were built around mathematical final-answer correctness.

8.4 CORE SAMPLED REASONING EVALUATION

The core sampled benchmark narrows the evaluation to GSM8K, GSM-Plus, Math500 and AIME 2024. It uses eight generations per example, up to 100 examples per dataset except the smaller AIME slice, seed 46 and an 8192-token generation budget. It is stricter than the broad deterministic snapshot

because it measures whether a model can repeatedly sample correct answers, rather than whether one deterministic generation is correct.

Model	Macro pass at k	Macro average at k	Generated tokens	Result reading
base	0.7792	0.6202	8,609,682	Best macro sampled model by both measures.
sent_final	0.7625	0.6190	8,452,004	Strongest trained variant; essentially base-level by average at k.
sent_best	0.7600	0.6059	8,521,085	Weaker transfer than in the deterministic benchmark.
no_sent_final	0.7550	0.6045	8,421,170	Competitive, but below base and sent_final by sampled stability.

Table 8.6: Core sampled benchmark by model.

This benchmark does not reproduce the simple sent_best victory from the deterministic snapshot. Base has the highest macro pass at k and the highest macro average at k. Among trained variants, sent_final is the strongest endpoint and is almost tied with base by average at k, but it does not exceed base at macro pass at k. Parser and format metrics remain healthy in this longer-cap sampled run: weighted format compliance stays between 0.9576 and 0.9758, invalid or unparsable outputs stay near one percent, and sent_final has the lowest weighted truncation rate at 0.1451. The loss against base is therefore not mainly a parsing failure; it is a difference in sampled solution coverage.

The dataset-level results explain why the aggregate sampled benchmark is mixed rather than a clean win for one model. Table 8.7 reports the full four-model comparison for each core dataset from the sampled benchmark logs.

Dataset	Model	Pass at 1	Pass at k	Average at k	Average length	Truncation
AIME 2024	base	0.3000	0.5667	0.2458	7321.3	0.7375
AIME 2024	sent_best	0.3000	0.5000	0.2250	7418.5	0.7125
AIME 2024	sent_final	0.2333	0.5000	0.2458	7255.9	0.7083
AIME 2024	no_sent_final	0.1667	0.5000	0.2042	7385.8	0.7625
GSM-Plus	base	0.6400	0.7400	0.6200	3102.6	0.0938
GSM-Plus	sent_best	0.5800	0.7400	0.6063	3019.0	0.0913
GSM-Plus	sent_final	0.6300	0.7600	0.6288	3028.4	0.0975
GSM-Plus	no_sent_final	0.6200	0.7200	0.6175	2966.8	0.0988
GSM8K	base	0.8700	0.9700	0.8763	1998.7	0.0275
GSM8K	sent_best	0.8500	0.9700	0.8700	1980.5	0.0288
GSM8K	sent_final	0.8900	0.9600	0.8763	1920.2	0.0175
GSM8K	no_sent_final	0.8500	0.9400	0.8613	1933.4	0.0263

Math500	base	0.7300	0.8400	0.7388	3464.4	0.1488
Math500	sent_best	0.7100	0.8300	0.7225	3426.2	0.1550
Math500	sent_final	0.6900	0.8300	0.7250	3439.7	0.1513
Math500	no_sent_final	0.7400	0.8600	0.7350	3410.5	0.1575

Table 8.7: Core sampled benchmark by dataset.

The table keeps the interpretation bounded. `sent_final` is strongest on GSM-Plus and reaches the best GSM8K first sample, but `base` remains strongest or tied on several sampled-coverage and stability measures. No trained variant improves AIME 2024 pass at k , where the average response length remains close to the 8192-token cap.

Table 8.8 checks the same result with matched examples. Each trained adapter is compared with the base model on the same sampled benchmark examples, and the bootstrap resamples those examples to estimate whether the trained adapter has a reliable advantage. The result agrees with Table 8.6: in the 8192-token sampled run, the trained adapters do not show a broad win over the base model.

Comparison versus base	Pass at k difference	Average at k difference	Interpretation
sent_final	-0.0167, interval -0.0667 to 0.0325.	-0.0013, interval -0.0174 to 0.0149.	Closest trained endpoint to base; no clean sampled win.
sent_best	-0.0192, interval -0.0642 to 0.0275.	-0.0143, interval -0.0280 to -0.0012.	Deterministic checkpoint strength does not transfer cleanly to sampled stability.
no_sent_final	-0.0242, interval -0.0658 to 0.0125.	-0.0157, interval -0.0324 to -0.0009.	Strong control, but weaker than base by sampled stability.

Table 8.8: Core sampled bootstrap comparison.

The supported result is not a broad sampled win over the base model. The best trained endpoint is close to base overall and stronger on the GSM-Plus target family, while the other trained variants lose sampled stability more clearly.

8.5 PROMPT-MATCHED 768-TOKEN FINAL STRESS TEST

The prompt-matched benchmark is the closest final evaluation to the actual training setup. It uses the training prompt style, the 768-token response cap, four core datasets and eight samples per example. The same cap that made training feasible also makes this a harder evaluation. AIME 2024 fully truncates for every model, so the AIME row is reported as a negative stress result rather than as evidence for improvement.

The 8192-token sampled benchmark was used first because it follows common math-reasoning evaluation practice: the prompt asks the model to show its reasoning and place the final answer in a boxed expression. Similar boxed-answer prompts appear in recent reasoning evaluations for Qwen, Qwen-Math and RLVR-trained models [20] [13] [21] [19], while the more explicit benchmark descriptions also extract and verify the boxed answer [20] [13]. That setting gives the DeepSeek-R1-Distill model family room to generate long reasoning traces. The four-model sampled run produced about 34 million tokens, yet the hardest rows still showed substantial truncation. That result made the prompt-matched run necessary: it keeps the training chat format and restores the 768-token response budget used during GRPO. Under those conditions the same model set produced about 6.3 million tokens. The result therefore tests whether the adapters help under the local training constraint, not only under a more permissive benchmark-style reasoning prompt.

The most informative prompt comparison uses GSM8K and GSM-Plus, where the response cap is demanding but not fully saturated. On those two datasets, switching from the benchmark-style prompt to the training prompt cuts generated tokens by about 80% for every model. Accuracy falls under the shorter budget, but the checkpoint ordering changes: the base model loses more average sampled correctness than the trained adapters, and the trained variants become the stronger prompt-matched options on average at k.

Model	8192-token avg length	768-token avg length	Token reduction	Avg at k, 8192-tok en	Avg at k, 768-tok en	Checkpoint reading
base	2550.6	512.2	79.9%	0.7481	0.5575	Strong with the long reasoning prompt, but weaker than the trained adapters under the training prompt.

sent_ best	2499.8	493.8	80.2%	0.7381	0.5756	Validation-selected checkpoint; best prompt-matched first-sample result on the practical slice.
sent_f inal	2474.3	484.1	80.4%	0.7525	0.5806	Best average at k in both prompt regimes on these two datasets.
no_se nt_fin al	2450.1	481.5	80.3%	0.7394	0.5775	Shortest prompt-matched aggregate; shows that the short-answer effect is not exclusive to SENT.

Table 8.9: Practical prompt-regime comparison.

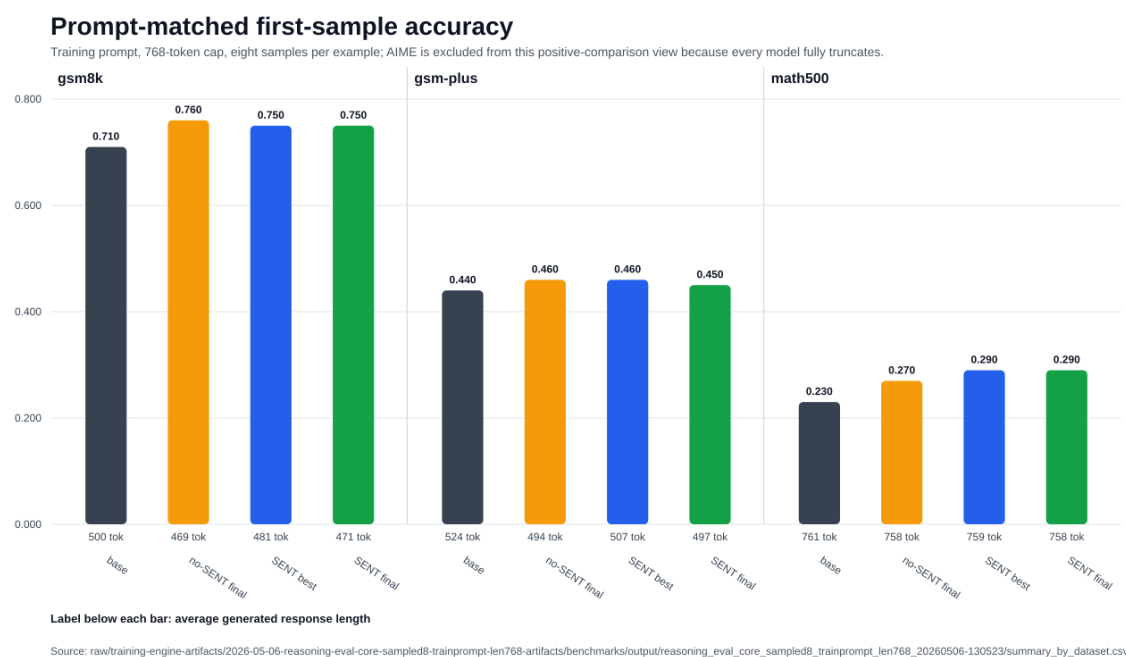


Figure 8.2: Prompt-matched accuracy and length.

Figure 8.2 reports first-sample exact accuracy on the three practical non-AIME datasets and prints the average response length below each bar. This view gives the strongest positive result for the local adapter comparison: trained variants improve first-sample accuracy over base on GSM8K, GSM-Plus and Math500 while also producing shorter answers on the same rows.

Model	Macro pass at 1	Macro pass at k	Macro average at k	Weighted avg length	Result reading
base	0.3533	0.5533	0.3315	610.8	Best sampled coverage, weakest average stability.
sent_best	0.3750	0.5508	0.3498	599.0	Best macro pass at 1.
sent_final	0.3725	0.5458	0.3529	592.8	Best macro average at k.
no_sent_final	0.3725	0.5483	0.3496	591.2	Shortest trained aggregate and strong pass at 1.

Table 8.10: Prompt-matched 768-token sampled benchmark.

The prompt-matched benchmark supports a narrower and more defensible positive claim than the 8192-token sampled run. Trained adapters improve macro pass at 1 and macro average at k over base while generating fewer tokens. Base still has the highest macro pass at k, so it preserves a small rare-resolution coverage advantage. The trained adapters are better described as more compact and more stable on average, not as universally more diverse. The parser-side metrics move in the same direction as the length result: weighted format compliance rises from 0.6333 for base to 0.6515 or 0.6636 for the trained variants, and weighted truncation falls from 0.5341 to roughly 0.48 to 0.50. The short-cap evaluation is still difficult, but the trained adapters do not obtain their gains by producing less parseable answers.

Dataset	Base row	Best trained row	Length and truncation	Observed result
GSM8K	Pass at 1 0.71, pass at k 0.92, average at k 0.6637.	no_sent_final reaches 0.76 pass at 1; sent_final reaches 0.6950 average at k; sent_best reaches 0.94 pass at k.	Average length drops from 500.4 to 469.5 with no_sent_final; truncation drops from 0.2238 to 0.1487.	First-sample accuracy, stability and response length all improve, although the strongest metric is split across trained variants.

GSM-PI us	Pass at 1 0.44, pass at k 0.63, average at k 0.4512.	sent_best and no_sent_final reach 0.46 pass at 1; sent_best reaches 0.4675 average at k and ties base at 0.63 pass at k.	Average length drops from 524.1 to 493.6 with no_sent_final; truncation drops from 0.2650 to 0.1800.	The target-family prompt-matched result improves modestly without increasing sampled coverage beyond base.
Math50 0	Pass at 1 0.23, pass at k 0.63, average at k 0.2025.	sent_best and sent_final reach 0.29 pass at 1; sent_best reaches 0.2437 average at k; no_sent_final ties base at 0.63 pass at k.	All variants stay near the cap: base length 760.8 and truncation 0.9738, compared with about 758 tokens and about 0.96 truncation for trained variants.	First-sample and average sampled correctness improve, but the row remains length-limited.
AIME 2024	Pass at 1 0.0333, pass at k 0.0333, average at k 0.0083.	No trained variant improves pass at 1 or pass at k; sent_final only raises average at k to 0.0167.	All variants average 768.0 tokens, all truncate at 1.0, and format compliance is 0.0.	Short-cap stress failure, not a positive improvement claim.

Table 8.11: Prompt-matched dataset-level result.

The AIME row prevents overstatement. The same cap that makes training feasible also prevents useful AIME reasoning in this prompt-matched run. The positive result is therefore limited to practical GSM-style and Math500 examples under the local response budget.

8.6 QUALITATIVE SAMPLED EXAMPLES

The sampled traces show why the aggregate metrics are mixed. Some trained adapters reduce truncation and produce correct solution patterns that are absent from the base samples. Other examples show regressions or wrong attractors introduced by the trained checkpoint. The qualitative evidence is therefore used to explain metric disagreement, not to replace the numeric tables.

Trace key	Example pattern	Observed behavior	Result meaning
Core sampled, GSM-Plus example 65.	SENT final helps on a GSM-Plus distance problem.	Base solves 4 of 8 samples and truncates 3 of 8; sent_final solves 7 of 8, has a correct first sample and no truncations.	Target-family benefit: more stable sampling and less truncation.
Core sampled, Math500 example 25.	SENT final finds a Math500 functional-equation solution pattern.	Base repeats the wrong answer in all eight samples; sent_final samples the correct answer in 3 of 8.	The adapter can add a correct solution path even when pass at 1 remains wrong.
Core sampled, Math500 example 88.	SENT final regresses on a Math500 surjectivity problem.	Base solves all eight samples; sent_final solves 4 of 8 and produces more long or malformed trajectories.	Training can introduce wrong attractors on harder algebra.
Core sampled, GSM-Plus example 61.	SENT best adds a GSM-Plus restart-based solution pattern.	Base has no correct samples; sent_best produces the correct answer in 2 of 8 samples.	Checkpoint selection can expose useful solution patterns absent from the base samples.
Core sampled, GSM8K example 37.	SENT best loses a simple GSM8K accounting problem.	Base solves 3 of 8 samples with a correct first sample; sent_best samples only wrong answers.	The deterministic sent_best win does not transfer uniformly to sampled behavior.

Table 8.12: Qualitative sampled behavior patterns.

The qualitative examples match the numeric pattern. The trained adapters can make answers shorter, reduce truncation, and produce correct solution patterns that the base samples miss. They can also remove a correct base behavior or increase malformed trajectories. This explains why pass at 1, average at k and pass at k do not all move in the same direction.

8.7 RESULT SUMMARY

The final empirical result is bounded. The local LoRA GRPO training changed the solution distribution, but the direction of the change depends on the evaluation view. Under the broad deterministic evaluation, the validation-selected SENT checkpoint is the best model-level variant. Under the core 8192-token sampled evaluation, the base model remains strongest at macro sampled coverage and is essentially tied with `sent_final` by sampled stability. Under the prompt-matched 768-token evaluation, trained adapters improve macro pass at 1 and average at k while producing shorter responses; the base model keeps the best macro pass at k.

The most favorable result is the prompt-matched one:

- trained adapters improve first-sample exact accuracy on GSM8K, GSM-Plus and Math500 under the training prompt;
- trained adapters shorten responses on the same practical non-AIME datasets;
- `sent_final` gives the best macro average at k in the 768-token prompt-matched run;
- `sent_best` remains important as a checkpoint-selection result in the broad deterministic benchmark.

The main negative or ambiguous findings are equally important:

- `sent_final` is the weakest macro variant in the broad deterministic run;
- base remains the best model by macro pass at k in both sampled evaluations;
- the no-SENT run is not a perfectly matched control because the preserved metrics lack the final 256-sample test row;
- the SENT run completes, but its harder stage shows lower reward, longer responses and more truncation.

The result to carry forward is therefore narrow: small LoRA adapters trained with the local GRPO pipeline improve prompt-matched first-sample correctness and average sampled correctness on practical mathematics datasets while shortening responses, but they do not dominate the base model across every metric, dataset or response budget.

9. STATISTICAL INTERPRETATION AND DISCUSSION

9.1 EVIDENCE HIERARCHY

The final evidence does not collapse into one context-free score. The trained adapters behave differently depending on the prompt, the decoding policy, the response budget and the metric. The statistical reading therefore separates the result that supports bounded enhancement from the checks that limit it.

The prompt-matched sampled evaluation at 768 tokens carries the strongest positive evidence. It preserves the training prompt, the local response cap and the practical mathematics datasets where the budget is not immediately saturated. In that view, the trained adapters improve first-sample correctness and average sampled correctness on GSM8K, GSM-Plus and Math500 while producing shorter responses. That is the empirical basis for saying that the local GRPO pipeline enhanced small-language-model reasoning behavior under the conditions that the project could actually train.

The sampled core evaluation at 8192 tokens has a different role. It tests whether the same adapters remain stronger when the generation budget is much larger and each example receives eight completions. The result is more restrictive: `sent_final` is the strongest trained endpoint by some sampled measures, but the base model keeps the best macro pass at `k` and macro average at `k`. The paired bootstrap intervals do not justify a broad trained-over-base superiority claim.

The broad deterministic benchmark answers a third question. It is useful for checkpoint selection across many datasets, not for proving endpoint robustness under sampling. In that view, `sent_best` is the strongest model-level variant, while `sent_final` is not. Training-run diagnostics sit below these final evaluations. Reward saturation, zero-loss rows, response-length drift, selected-token coverage and truncation explain why a run became useful, flat or unstable, but they are not final model-quality scores.

Evidence layer	Interpretive status
Prompt-matched sampled evaluation at 768 tokens	Strongest evidence for bounded enhancement under the actual local training regime.
Sampled core evaluation at 8192 tokens	Robustness check that blocks a general trained-over-base sampled superiority claim.

Broad deterministic benchmark	Checkpoint-selection evidence across many datasets, not endpoint superiority evidence.
Training-run and campaign diagnostics	Mechanistic context for optimization behavior, not final evaluation.

Table 9.1: Evidence hierarchy for final interpretation.

The hierarchy sets the boundary of the interpretation. Enhancement means a measured change in reasoning behavior under a defined setup, not dominance across all mathematical benchmarks. A result that appears in the prompt-matched benchmark and agrees with response-length and qualitative evidence carries substantial weight. A result that weakens or reverses in the sampled robustness view has to be reported as metric-dependent.

9.2 WHY THE METRICS DISAGREE

The evaluations disagree because they ask different statistical questions. Post-training can make the first completion more reliable while reducing rare-solution coverage, or it can improve a validation-selected checkpoint while leaving the final endpoint weaker. Those patterns are not contradictions. They are different views of the same output distribution.

Metric or view	Question answered	Role in the final comparison
Deterministic accuracy	Does one fixed decoding path reach the correct answer?	Useful for breadth and checkpoint selection.
Pass at 1	Is the first sampled answer correct?	Closest to ordinary single-response use under sampling.
Average at k	How often is the model correct across sampled completions?	Measures sampled stability better than a single lucky completion.

Pass at k	Does at least one sampled completion solve the example?	Measures rare-solution coverage and sampled diversity.
-----------	---	--

Table 9.2: Metric meanings behind the mixed results.

The prompt-matched result is the direct basis for the enhancement claim because it measures the practical operating mode: the same prompt family, the same response cap and datasets where the cap still leaves room for a full answer. Improving pass at 1 and average at k while shortening answers means that the adapter has made the typical sampled response more useful in that regime. It does not mean that the model has become more diverse or more capable under every decoding budget.

The pass-at-k result keeps the conclusion narrower. A model receives credit if any sampled completion solves the example. The base model retaining a small advantage on this metric means that it still samples some rare correct solution patterns that the trained adapters miss. The trained adapters are therefore better described as more compact and more stable in the prompt-matched setting, not as universally better samplers.

The deterministic sent_best result belongs to the training trajectory. A validation-selected checkpoint can preserve a useful region before later updates move the endpoint away from it. That is plausible in this setting because the reward is sparse, the response budget is tight, and the second curriculum stage changes the prompt distribution. The broad deterministic benchmark shows that the SENT branch can produce a strong selected checkpoint. The sampled benchmarks show that the same strength does not transfer uniformly to stochastic robustness.

The no-SENT endpoint prevents over-attribution. It remains competitive with the SENT variants and leads some dataset-level sampled measures. Since both GSM-Plus branches share the same backbone, LoRA configuration, response cap, verifier path, length controls and entropy masking, the observed gains cannot be assigned only to semantic-entropy curriculum ordering. The safer reading is that the custom GRPO regime produced measurable behavioral change, while SENT changed the checkpoint and distribution profile inside that regime.

9.3 SENT, ENTROPY AND DISTRIBUTION SHAPING

The semantic-entropy components are best interpreted as distribution-shaping mechanisms. The question is not whether semantic entropy alone makes the model better. The evidence is whether entropy-aware training changes which solution patterns appear, how stable they are and where useful checkpoints occur.

The earlier literature sets the boundary for this interpretation. DeepSeek-R1, Dr. GRPO and SENT motivate the local design, but they come from settings with different scale, rollout budgets,

normalization choices and curriculum assumptions [1] [13] [18]. The local result is therefore evidence for a small LoRA-based GRPO regime under strict response and hardware limits, not a direct reproduction of larger reasoning-RL systems.

Under that narrower setting, the evidence supports a bounded SENT role. The validation-selected SENT checkpoint is the leading variant in the broad deterministic benchmark. The final SENT endpoint is the strongest trained endpoint in the sampled core benchmark. Qualitative traces also show cases where the trained model reduces truncation or produces a correct solution pattern that is absent from the base samples. Those observations fit the distribution-shaping hypothesis: entropy-guided selection can change the accessible solution patterns and the useful point along the training path.

The same evidence rejects a stronger claim. SENT final is not best in every view, the no-SENT endpoint remains competitive, and the completed GSM-Plus SENT run becomes harsher in its second stage. The final prompt-matched benchmark also shows a pattern shared by the trained variants: better first-sample and average sampled correctness with shorter outputs. SENT is therefore not an isolated accuracy explanation. It is a mechanism that helps organize curriculum pressure and checkpoint behavior within the broader custom GRPO setup.

Semantic entropy also has to remain a proxy. A low-entropy problem can still be confidently wrong. A high-entropy problem can reflect unstable answer forms, formatting drift or sampling noise rather than useful mathematical difficulty. The same caution applies at token level. Entropy masking can focus updates on uncertain positions, but those positions are not automatically the causal reasoning steps.

Entropy masking stayed in the final setup for an empirical reason. Earlier long-run diagnostics showed that once responses drifted toward the cap, many updates carried little usable completion signal: selected-token coverage collapsed and zero-loss rows became frequent. Entropy masking does not solve truncation by itself, but it makes the update more selective when only part of a completion is informative. In the final comparison it helps define the shared custom GRPO regime. Both SENT and no-SENT use it, so the observed prompt-matched gains are not attributed to SENT alone. They are evidence for a broader local training recipe in which entropy masking, response-length control, verifier rewards and checkpoint selection have to be read together.

The mechanism evidence is therefore interpretive rather than causal. Table 9.3 summarizes the role of the three mechanisms that shaped the final runs and the boundary that prevents over-attribution.

Mechanism	Role in the evaluated runs	Observed evidence	Boundary of the evidence
Entropy masking	Focuses the training update on higher-uncertainty generated tokens instead of applying the same pressure to the whole response.	Retained after earlier diagnostics showed that low selected-token coverage and zero-loss rows could dominate unstable long runs.	Shared by SENT and no-SENT, so it supports the custom GRPO regime but is not isolated as a separate final ablation.
SENT curriculum	Orders training through semantic-entropy stages and produces both a final adapter and a validation-selected checkpoint.	sent_best is the strongest broad deterministic variant, while sent_final is the strongest trained endpoint in the core sampled benchmark.	The final SENT endpoint is not the strongest variant in every evaluation view, and stage 2 is less healthy than stage 1.
Length and truncation controls	Keep training and evaluation inside the local response budget used by the 768-token setup.	In the prompt-matched evaluation, trained adapters improve first-sample accuracy and average sampled correctness while generating shorter responses on the practical non-AIME datasets.	AIME 2024 fully truncates at 768 tokens for every model, so the same cap also marks the limit of the local setup.

Table 9.3: Mechanism evidence and boundaries.

9.4 REWARD, LENGTH, TRUNCATION AND EXPERIMENTAL BREADTH

Reward, length and truncation are not secondary logs. They decide whether a GRPO update contains contrast, whether the verifier can extract the answer and whether a completion still carries useful signal after the response cap is reached. The reward measures verifiable final-answer behavior. That makes the training auditable, but it does not turn reward into a complete measure of reasoning quality.

Group-relative optimization depends on differences inside a sampled group. Uniformly correct, uniformly wrong or identically shaped rewards create little useful contrast. This explains why terminal reward has to be read together with reward dispersion, zero-loss frequency, selected-token ratio, length and truncation. A high or flat reward trace alone is not enough to say that reasoning improved.

Response length carries the same ambiguity. Shorter outputs are useful when correctness and stability are preserved, as in the prompt-matched results on the practical non-AIME datasets. Shorter outputs are not automatically better reasoning. Longer outputs are not automatically deeper reasoning either. They can help when a problem needs more steps, but they also increase truncation risk, hide the final answer from the extractor and raise the cost of every completion.

Truncation marks the hard edge of the local setup. AIME 2024 at 768 tokens is not a failed proof of reasoning improvement; it is a stress result showing that the chosen budget is too short for that dataset. Math500 near the cap has to be read with the same caution. The dataset pivot to GSM-Plus follows from this constraint: a training dataset had to preserve reward variation inside the group while keeping enough completions inside the response budget.

Observed variable	Effect on interpretation
Reward contrast	Shows whether the sampled group can provide a useful policy-gradient signal.
Terminal reward	Works as a training diagnostic, not as a standalone quality score.
Format behavior	Supports extraction and measurement but does not replace mathematical correctness.
Response length	Improves the interpretation only when shorter outputs keep correctness and stability.
Truncation	Limits both optimization and evaluation when capped completions lose the final answer.
Dataset admission	Requires reward variation, answer extractability and feasible completion length at the same time.

Table 9.4: Statistical variables behind reward interpretation.

9.5 DEFENSIBLE CLAIM BOUNDARIES

The final interpretation separates supported conclusions from plausible mechanisms and non-claims. Claims about enhancement, custom GRPO and semantic-entropy strategies are defensible only when they stay tied to the local evidence.

The experimental breadth is also part of that boundary. The completed GSM-Plus SENT run required multi-day local training, and the no-SENT comparison was also a long run. The final training-prompt sampled evaluation alone recorded 20.34 hours of generation latency. Those durations matter statistically because each additional seed, response cap, dataset or ablation would have multiplied the same cost while competing with the fixed local hardware budget and project schedule. The resulting evidence base is therefore narrower than a full interaction grid, but the limitation follows directly from running sampled reinforcement-learning evaluation locally with long mathematical completions and multiple checkpoint variants.

This constraint does not invalidate the results. The work compares trained variants under a small number of deliberately chosen evaluation views, but it cannot estimate every interaction among seed, dataset, response cap, curriculum stage and entropy mechanism. The conclusions therefore remain bounded empirical findings rather than population-level claims about all small-language-model reasoning training.

Claim category	Defensible statement	Boundary
Enhancement	The trained adapters improve first-sample and average sampled correctness under the prompt-matched 768-token regime while shortening outputs.	The sampled core benchmark prevents a general trained-over-base superiority claim.
Custom GRPO	A local GRPO-family pipeline can produce measurable behavioral change with verifier rewards, response controls and adapter-only training.	The result is tied to the selected model, datasets, prompt, reward design and hardware budget.
LoRA efficiency	A small adapter fraction can shift the behavior of a frozen SLM backbone, consistent with parameter-efficient fine-tuning evidence [10] and reasoning-adapter results [9].	The evidence is not a full comparison against dense fine-tuning or other adapter ranks.

Semantic-entropy strategies	SENT and entropy masking plausibly shape the training distribution, checkpoint behavior and sampled solution patterns.	The final evidence does not isolate them as the only cause of the gains.
Scope	The conclusions apply to local mathematical reasoning experiments with sampled evaluation and strict response budgets.	They do not cover all reasoning domains, larger models, multi-GPU training or unlimited generation.

Table 9.5: Claim boundaries for final interpretation.

Stronger claims would require a wider experimental grid: more random seeds, more datasets, more response caps, explicit entropy-mask ablations, direct SENT-versus-no-SENT repeats, LoRA rank sweeps and longer-budget evaluations on hard datasets such as AIME. The local runs make clear why that grid was not attempted here. Each added branch would have meant another long reinforcement-learning run or another expensive sampled benchmark, and the project already had to spend multi-day runs on the core comparison.

The supported conclusion is precise rather than broad. A frozen SLM backbone can be shifted with a small LoRA update, and the local GRPO-family trainer improves prompt-matched sampled mathematical behavior under the tested budget. Semantic-entropy strategies help explain checkpoint behavior and sampled solution patterns, but the interpretation still depends on dataset, response cap, metric and training trajectory.

10. ETHICS

The ethical question in this project is narrow: under what conditions is it responsible to claim that reinforcement learning improved mathematical reasoning in a language model? The answer depends on social impact, including bias, access, sustainability, misuse and academic integrity, and on data-science responsibility: dataset representativeness, reward validity and truthful evaluation.

Ethics is therefore part of the analytical work, not a declaration added after the experiments. A verifier-compatible math task makes the training signal auditable, but narrows what can be claimed about reasoning. A compact experimental setting improves access, but limits statistical power. AI-assisted tools can help with log analysis, debugging and command preparation, but require traceability and human responsibility.

10.1 BIAS AND FAIRNESS

Verifier-compatible training sets the first fairness boundary. This creates six practical limits:

- **Task scope.** The reward does not measure reasoning quality in general; it measures whether an extracted final answer matches the expected numeric value under a particular dataset, prompt format, parser and normalization rule. The experiment can show behavior on a controlled family of mathematical tasks, but it cannot show equal reliability across cultures, languages, educational backgrounds, problem formats or real-world user groups.
- **Dataset bias.** Math word problems may look socially neutral because they ask for numerical answers, but they still encode assumptions about language, school curricula, units, everyday situations and familiar contexts. A model that improves on this task family may have learned the benchmark style, not robust reasoning across quantitative communication. If the evaluation set is narrow, the demonstrated competence is narrow too.
- **Verifier boundary.** The local verifier judges what the parser can recognize, not every valid way to state an answer. It can miss a correct answer written in an unsupported form. More generally, outcome-level rewards can also reward a correct final value reached through a flawed explanation, so verifier scores are bounded evidence rather than a complete measure of reasoning quality [26].
- **Parser-friendly style.** Binary rewards can privilege answer styles that fit the parser over other human-facing qualities, so format compliance is not treated as a universal measure of quality.
- **Claim boundary.** GSM-style evaluation cannot prove social representativeness, multilingual robustness, accessibility for different learners or safety in high-stakes domains. Claims must stay tied to the dataset, verifier, response cap, reward logs, held-out evaluation and training configuration. A narrow experiment is acceptable when presented as narrow; it becomes misleading when benchmark gains become general claims about intelligence, reliability or educational value.

- **Gender perspective.** This perspective is relevant but proportional. The system does not classify people, rank candidates or make user-facing decisions, so gender is not the central material risk axis. The defensible claim is that reducing the experimental cost threshold can support wider participation, including for underrepresented groups in computing, when paired with clear limitations, responsible documentation and careful language around access.

These limits do not invalidate the experiment, but they do restrict what the results can responsibly mean.

10.2 SECURITY AND MISUSE

The main security risk is not that the trained model becomes a frontier dangerous system. The scale is small, the task is narrow, and the work is academic. The realistic risk is over-optimization against a simplified reward: the model may look better in logs while failing to improve the intended behavior, and readers may overinterpret the result.

Reward hacking is the clearest example. In preference-based RL, a policy can exploit weaknesses in a reward model and increase proxy reward while true quality stagnates or degrades. Recent work frames this as unreliable advantage signs during optimization [12]. This project does not use a learned reward model for the main GSM-style verifier, but a deterministic verifier can also be exploited if it overemphasizes formatting, answer markers or superficial final-answer patterns. Reward is evidence about one measurable objective, not proof of the intended reasoning process.

The practical response is to keep four safeguards visible:

- **Configuration selection.** The local experiments showed why selection cannot depend on one scalar value. Reward has to be read beside response-budget health, held-out correctness and manual review.
- **Reasoning scope.** A model can solve a sampled math problem and still be unreliable on a differently worded problem, a longer proof, or a domain where the answer is not verifiable. Survey work on reinforcement learning for reasoning models links current progress to mathematics and coding partly because those domains have clearer reward definitions than open-ended generation [11]. The project studies RL for verifiable reasoning under local constraints; it does not certify the model for tutoring, finance, medicine, law or autonomous decisions.
- **Model release.** Sharing code, configurations, benchmark reports and LoRA adapters can support reproducibility, but intermediate checkpoints, collapsed runs and models selected before final validation are research artifacts. If a checkpoint is shared, it needs task scope, training data, response budget, known failure modes and evaluation limits.

- **Human oversight.** The verifier cannot inspect every property that matters. It can check final-answer correctness, but not pedagogical quality, memorization, run comparability after configuration changes, or practical checkpoint reliability. Misuse risk is reduced by keeping release and claim boundaries narrow.

10.3 SUSTAINABILITY AND ACCESSIBILITY

The sustainability issue is not only where the computation runs, but how experimental evidence is produced. Repeated reinforcement-learning runs consume energy even at small scale, and weak experimental discipline can waste compute through avoidable reruns. For that reason, the project treats short screening runs, bounded benchmarks, response caps and preserved artifacts as part of responsible experimentation, not only as engineering conveniences.

Environmental work shows that language-model development can account for a large share of impact [27], and other work links emissions to model size, repository activity, commit frequency and repeated versioning [28]. The ethical claim is therefore not that local training is automatically sustainable. The defensible claim is that compute use is more responsible when each run has a clear purpose, failed branches remain documented, and evidence can be reused instead of regenerated.

The 768-token cap also has an evaluation role. It limits generation cost while keeping enough room for evaluable reasoning, so the model is not given an unbounded budget to produce long traces whose extra cost may not add useful evidence.

Accessibility depends on documentation and auditability. Reward paths, dataset checks, benchmark reports and logs make the experiment inspectable rather than a black box. This connects the work to the SDG framing around quality education, reduced inequalities, responsible consumption and climate action in a restrained sense [29]. The social value lies in making RL-based reasoning adaptation understandable and auditable, not in claiming that local training is automatically sustainable or inclusive.

10.4 USE OF AI ASSISTANCE AND ACADEMIC INTEGRITY

AI assistance was part of the working environment. It helped with code exploration, debugging support, command preparation, analysis of long logs and drafting support. That support does not remove personal responsibility. An AI system can suggest alternatives or help find mistakes, but it cannot decide whether an experiment is valid or whether a conclusion follows from the evidence.

The relevant safeguards were authorship and verification. AI-generated text was not treated as final academic prose: any generated wording used during drafting was revised, rewritten and checked by the author before inclusion. No result was accepted because an assistant produced a plausible

explanation. AI tools were not used as bibliographic authority; references, technical claims and experimental conclusions were checked against papers, documentation, preserved metrics, logs, checkpoints, tests and configuration files.

Traceability is the main academic safeguard: claims about reward behavior, benchmark ranking, collapse or model quality must lead back to checkable artifacts, and final responsibility remains with the author.

10.5 ALTERNATIVES, DECISIONS AND ETHICAL ASSESSMENT

The ethical risks lead to concrete project decisions. The project rejected easier alternatives that would have weakened its ethical basis. The adopted position was conservative: keep the study focused on verifier-compatible mathematical reasoning, and pair that choice with bounded claims, preserved evidence, explicit metric interpretation, limited release interpretation and responsibility for the final conclusions.

Ethical issue	Rejected alternative	Adopted decision	Justification
Dataset and verifier bias	Treat GSM-style reward as general reasoning quality	Restrict conclusions to the evaluated task family, parser, prompt, response cap and held-out checks	Prevents broad fairness, safety or reasoning claims from a narrow benchmark
Reward hacking and proxy optimization	Select configurations from scalar reward alone	Interpret reward with length, truncation, zero-loss behavior, selected-token coverage, held-out correctness and manual review	Reduces the risk of optimizing the verifier while losing intended behavior
Model release and misuse	Present every checkpoint as ready for non-experts	Treat intermediate and collapsed checkpoints as research artifacts	Supports reproducibility without implying operational reliability

Statistical validity	Treat one seed or small held-out slice as conclusive	Separate diagnostic patterns, repeated-seed evidence and final held-out claims	Keeps conclusions proportional to the evidence
Experimental cost	Treat available compute as justification for undocumented reruns	Use staged screening, finalist narrowing, response caps and preserved artifacts	Keeps experimentation proportional and avoids repeating undocumented work
Accessibility	Describe compact experimentation as automatically democratizing	Present it as modest access improvement with remaining limits	Avoids overstating inclusion while preserving educational value
Data governance and artifacts	Treat public datasets and local execution as removing governance obligations	Keep dataset provenance, model licenses, prompts, logs, checkpoints, benchmark outputs and access boundaries explicit	Preserves auditability without implying that absence of human-participant data removes all responsibility
Academic integrity	Describe AI assistance vaguely or omit it	Disclose material support and tie claims to reviewed artifacts	Keeps responsibility clear and prevents polished unsupported prose

Table 10.1: Ethical alternatives, decisions and justification.

These decisions keep the SDG connection restrained: the project supports education, access and responsible consumption only as far as the experiment remains inspectable, compute use stays bounded, and evidence can be reused.

The ethical position is based on disciplined narrowing. Verifier-based reinforcement learning is useful when it makes training auditable without pretending to measure the full reasoning process. Compact experimentation is valuable when it lowers the barrier to study without excusing weak statistical claims. AI assistance is acceptable when it supports debugging and log analysis without replacing validation, judgment or accountability. The project remains responsible only as long as its costs, risks and claims stay inspectable.

11. CONCLUSIONS AND FUTURE WORK

11.1 OBJECTIVE FULFILLMENT REVIEW

The project set out to study whether a compact reasoning model could be improved through a custom GRPO-family training pipeline under local hardware constraints. The objectives defined in Section 1.4 were achieved in a bounded form. The work produced a complete local training and evaluation path around DeepSeek-R1-Distill-Qwen-1.5B, LoRA adaptation, verifiable mathematical rewards, group-relative optimization, entropy-aware token selection, semantic-entropy curriculum control and sampled reasoning evaluation.

The completed work is mainly a technical and methodological result. It did not remain at the level of a literature review or a simple demonstration. It defined a learning problem, implemented the training loop, preserved the main experiment artifacts, compared trained adapters against the base model, and interpreted the result through deterministic, sampled-core and prompt-matched evaluation views. The final claim is evidence-backed, but narrower than the phrase "improved reasoning" suggests without context.

Objective reference and area	Closure status	Final assessment
O1: local GRPO-family training pipeline	Achieved	A native local pipeline trained LoRA adapters with group-relative rewards, response controls and diagnostic logging.
O2: algorithmic analysis of GRPO	Achieved	The study explains group-relative rewards, sparse or saturated reward contrast, response length, truncation and length-normalization concerns instead of treating GRPO as a black box.
O3: model-quality improvement	Partially achieved	The trained adapters improve prompt-matched first-sample and average sampled correctness with shorter outputs on GSM8K, GSM-Plus and Math500, but they do not dominate the base model across every metric.

O4: task-aligned validation	Achieved	The verifier-based setup supports reproducible mathematical evaluation, while the final analysis keeps reward, format, length, truncation and model quality separate.
O5: selective controls	Achieved with bounded evidence	Entropy masking and SENT were integrated and evaluated, but their contribution is best supported as signal shaping and checkpoint shaping rather than a universal endpoint improvement.
O3-O5: experimental breadth	Bounded by design	The long duration of serious runs and the local hardware envelope limited seeds, datasets, caps and ablations.

Table 11.1: Objective closure summary.

O2 was fulfilled because GRPO was examined as an optimization regime rather than just as a training command. The final analysis connects group-relative rewards, weak or zero-contrast updates, response length, truncation and Dr. GRPO-style length and normalization concerns to the interpretation of each run [13].

O3 remains the most constrained objective. Table 11.1 summarizes the positive prompt-matched 768-token result and its main boundary: the same adapters do not show broad superiority over the base model.

The link with the shared engineering work remains important. Low-level memory management, execution staging, profiling and recovery mechanisms made the local experiments possible. The learning contribution built on that platform and focused on the behavior of the objective, reward signal, curriculum, dataset fit and evaluation evidence. Both parts are connected, but the conclusion here concerns the learning study: what changed in the model behavior, how strong that evidence is, and which limits remain.

11.2 FINAL CONCLUSIONS

The final result supports a bounded empirical claim: a frozen small-language-model backbone can be shifted with a very small LoRA update. The trained adapters update only a small fraction of the model parameters, yet they change the sampled solution distribution under the tested mathematical prompts.

This agrees with the broader parameter-efficient fine-tuning premise of LoRA [10] and with recent evidence that compact adapter-based reasoning training can be useful when the evaluation scope is stated carefully [9].

The final conclusions are:

1. **Feasibility.** Local GRPO-style post-training for a compact reasoning model is feasible when the experiment is designed around the platform constraints. Generation cost, response caps, group size, adapter-only updates, reward extraction and logging define what can be trained, what can be evaluated and how strong the final statistical claim can be.
2. **Measured enhancement.** The best-supported improvement is a prompt-matched gain in practical short-budget mathematical reasoning behavior: better first-sample correctness, better average sampled correctness and shorter responses on the practical datasets. It is not a universal win over the base policy, which still keeps a sampled-coverage advantage in some views.
3. **GRPO and reward design.** The training loop showed that sparse rewards, weak group contrast, response length and truncation can change the meaning of a training curve. Final evaluation therefore remains necessary before a favorable run can be treated as a model-quality improvement.
4. **Semantic-entropy strategies.** SENT and entropy masking shaped checkpoint behavior and sampled solution patterns, but the evidence supports them as part of the local training recipe rather than as an isolated endpoint method that is always superior.
5. **Experimental scope.** The limited number of final long runs follows from the local budget. Multi-day training runs and more than 20 hours of generation latency for the final prompt-matched sampled evaluation made a small set of preserved comparisons more defensible than an unfinished broad grid over many seeds, datasets, response caps and ablations.

The project enhances reasoning capacities only in the specific sense supported by the artifacts: a custom local GRPO-family pipeline with LoRA and semantic-entropy controls improves prompt-matched sampled mathematical behavior while leaving several broader metrics unresolved or base-favorable. The contribution is empirical and methodological: it provides a trained-adapter result and a disciplined way to read small-scale reasoning-RL experiments under local constraints.

The ethical conclusion follows the same boundary. The project is responsible to the extent that its evidence remains traceable, its compute use is justified by preserved artifacts, and its claims stay tied to the evaluated datasets, verifier, response cap and checkpoints. AI assistance, release decisions and sustainability claims therefore require the same discipline as the metrics: they are acceptable when they support inspection and reproducibility, not when they replace validation or overstate the social value of a narrow benchmark result.

11.3 LIMITATIONS

Dataset scope. The experiments concentrate on GSM-style and short mathematical reasoning tasks with checkable final answers. These tasks are appropriate for verifier-based reinforcement learning, but they do not cover open-ended explanation quality, formal proof, multilingual reasoning, real-world decision making or human preference alignment. The result cannot be generalized beyond the evaluated mathematical setting without new evidence.

Statistical power. The final comparison uses preserved artifact-backed runs, but it does not estimate the full variance across many random seeds, response caps, learning rates, dataset variants or curriculum settings. Serious runs lasted many days, and sampled evaluation was itself expensive, so small differences cannot be treated as stable effects without further repeated validation.

Response-budget dependence. The 768-token cap is central to both the feasibility and the boundary of the project. It makes local training possible, but it also causes a hard failure mode on AIME 2024 and keeps Math500 near the cap. Any claim about reasoning improvement must therefore remain tied to the response budget under which it was measured.

Reward and verifier design. Final-answer rewards are auditable and reproducible, but they do not certify the reasoning process. A response can be correct for the wrong reason, fail because of extraction, or remain fluent while being mathematically weak. This limitation is common to lightweight RLVR-style training and becomes especially important under sparse binary feedback [1].

Causal isolation. The final trained variants combine several mechanisms: LoRA adaptation, verifier rewards, response-length controls, entropy masking and, in one branch, SENT. The final evaluation supports a combined local training strategy and gives useful contrasts between SENT and no-SENT variants, but it does not isolate every mechanism through a complete ablation grid.

Hardware-constrained algorithm selection. The study uses small groups, adapter-only updates, strict response caps and a limited set of final comparisons because the runs had to remain local. Larger batches, longer contexts, stronger repeated-seed designs, process rewards, persistent reference-model-heavy regularization and broader benchmark suites remain outside the completed experimental envelope. These omissions define the scale of the claim.

11.4 FUTURE WORK

Future work should strengthen the evidence before expanding the claim. The most direct extensions are:

1. **Repeat the final comparison over more seeds.** The prompt-matched trained-over-base improvement needs repeated runs of the selected trained variants and the closest control.

The comparison needs to report first-sample accuracy, average sampled correctness, pass at k, response length, truncation, format compliance and confidence intervals.

2. **Separate the entropy mechanisms through ablations.** Entropy masking, SENT curriculum ordering and length controls need long matched branches. This would clarify whether the observed improvements come from curriculum ordering, token-level selection, response-budget control or their interaction.
3. **Expand datasets through admission probes.** Additional datasets belong after answer extractability, response-length feasibility, truncation risk and within-group reward variation checks. A dataset that mostly produces capped or identical-reward groups is not a useful training target for O3 or O4.
4. **Test intermediate and adaptive response budgets.** The project already compared the 768-token training cap with an 8192-token sampled evaluation. Future work should test intermediate or adaptive caps, especially for AIME 2024, where the 8192-token run still produced truncation rates above 0.70. This would clarify whether hard examples need more budget, better stopping behavior, different prompting or a different training regime.
5. **Improve reward and verification signals.** Better answer normalization, verifier ensembles, process-level checks or lightweight step validators could reduce the gap between final-answer correctness and reasoning quality. These signals would need their own validation to avoid rewarding parser artifacts or superficial formatting.
6. **Study scale as an experimental variable.** Larger memory budgets, faster generation or multi-GPU execution would allow larger groups, more seeds and longer responses. That would test whether the observed limits come from the GRPO and entropy strategy themselves or from the local operating envelope.

With more time and compute, the next priority would be causal isolation. The final runs combine curriculum ordering, entropy masking, response-budget control and checkpoint selection, so the evidence supports the recipe more strongly than any single mechanism. More seeds, explicit entropy-mask ablations and less truncating response budgets would clarify which effects improve endpoint behavior and which mainly help locate useful checkpoints.

Future experiments should keep training reward, model quality, response length, truncation and sampled stability separate. Higher reward alone is not better reasoning, and shorter answers count as an improvement only when correctness and stability are preserved.

The final position is narrow. A custom local GRPO-family training pipeline with semantic-entropy strategies can produce bounded enhancement of small-language-model mathematical reasoning under a strict local budget. The enhancement appears in prompt-matched sampled behavior and in the distribution of useful checkpoints, not in every benchmark view. The remaining work is to test whether that result survives more seeds, cleaner ablations, broader datasets and less restrictive response budgets.

12. BIBLIOGRAPHY

- [1] D. Guo, D. Yang, H. Zhang, et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, 633-638, 2025. <https://doi.org/10.1038/s41586-025-09422-z>
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>
- [3] L. Ouyang, J. Wu, X. Jiang, et al., "Training Language Models to Follow Instructions with Human Feedback," *Advances in Neural Information Processing Systems*, vol. 35, 27730-27744, 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html
- [4] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024. <https://arxiv.org/abs/2402.03300>
- [5] MSR Avinash, "Profiling LoRA/QLoRA Fine-Tuning Efficiency on Consumer GPUs: An RTX 4060 Case Study," *arXiv preprint arXiv:2509.12229*, 2025. <https://arxiv.org/abs/2509.12229>
- [6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems*, vol. 36, 10088-10115, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html
- [7] "Reinforcement Learning (RL) Guide," <https://unsloth.ai/docs/get-started/reinforcement-learning-rl-guide> (accessed May 16, 2026).
- [8] "Tutorial: Train your own reasoning model with GRPO," <https://unsloth.ai/docs/get-started/reinforcement-learning-rl-guide/tutorial-train-your-own-reasoning-model-with-grpo> (accessed May 16, 2026).
- [9] S. Wang, J. Asilis, O. F. Akgul, E. B. Bilgin, O. Liu, and W. Neiswanger, "Tina: Tiny Reasoning Models via LoRA," *International Conference on Learning Representations (ICLR 2026)*, 2026. <https://openreview.net/forum?id=P2OXYO3bEe>
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations (ICLR 2022)*, 2022. <https://openreview.net/forum?id=nZeVKeeFYf9>

- [11] K. Zhang, Y. Zuo, B. He, Y. Sun, R. Liu, C. Jiang, Y. Fan, K. Tian, G. Jia, P. Li, Y. Fu, X. Lv, Y. Zhang, S. Zeng, S. Qu, H. Li, S. Wang, Y. Wang, X. Long, F. Liu, X. Xu, J. Ma, X. Zhu, E. Hua, Y. Liu, Z. Li, H. Chen, X. Qu, Y. Li, W. Chen, Z. Yuan, J. Gao, D. Li, Z. Ma, G. Cui, Z. Liu, B. Qi, N. Ding, and B. Zhou, "A Survey of Reinforcement Learning for Large Reasoning Models," *arXiv preprint arXiv:2509.08827*, 2025. <https://arxiv.org/abs/2509.08827>
- [12] S. Ono, J. Ackermann, S. Nishimori, T. Ishida, and M. Sugiyama, "Mitigating Reward Hacking in RLHF via Advantage Sign Robustness," *arXiv preprint arXiv:2604.02986*, 2026. <https://arxiv.org/abs/2604.02986>
- [13] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin, "Understanding R1-Zero-Like Training: A Critical Perspective," *Conference on Language Modeling (COLM 2025)*, 2025. <https://openreview.net/forum?id=5PAF7PAY2Y>
- [14] J. Zhang and C. Zuo, "GRPO-LEAD: A Difficulty-Aware Reinforcement Learning Approach for Concise Mathematical Reasoning in Language Models," *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 5642-5654, 2025. <https://doi.org/10.18653/v1/2025.emnlp-main.287>
- [15] A. Rad, K. Filom, D. Keivan, P. Mohajerin Esfahani, and E. Kamalinejad, "Rate or Fate? RLVR: Reinforcement Learning with Verifiable Noisy Rewards," *arXiv preprint arXiv:2601.04411*, 2026. <https://arxiv.org/abs/2601.04411>
- [16] P.-C. Pan, Y. Liang, and S. Lin, "Reward Modeling for Reinforcement Learning-Based LLM Reasoning: Design, Challenges, and Evaluation," *arXiv preprint arXiv:2602.09305*, 2026. <https://arxiv.org/abs/2602.09305>
- [17] M. Y. R. Yang, H. Bai, I. Wu, G. Yang, A. Setlur, and A. Kumar, "InT: Self-Proposed Interventions Enable Credit Assignment in LLM Reasoning," *International Conference on Learning Representations (ICLR 2026)*, 2026. <https://openreview.net/forum?id=YAhTj2VgBw>
- [18] H. Cao, Z. Bai, Z. Peng, B. Wang, T. Yang, J. Huo, Y. Zhang, and Y. Gao, "Efficient Reinforcement Learning with Semantic and Token Entropy for LLM Reasoning," *arXiv preprint arXiv:2512.04359*, 2025. <https://arxiv.org/abs/2512.04359>
- [19] S. Wang, L. Yu, C. Gao, C. Zheng, S. Liu, R. Lu, K. Dang, X.-H. Chen, J. Yang, Z. Zhang, Y. Liu, A. Yang, A. Zhao, Y. Yue, S. Song, B. Yu, G. Huang, and J. Lin, "Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning," *NeurIPS 2025*, 2025. <https://openreview.net/forum?id=yfcpdY4gMP>
- [20] K. Huang, H. Meng, J. Wu, J. Lu, C. Ma, Z. Chen, X. Wang, B. Ding, J. Wu, X. Wang, X. He, G. Wang, and J. Zhou, "Beyond Magnitude: Leveraging Direction of RLVR Updates for LLM

- Reasoning," *International Conference on Learning Representations (ICLR 2026)*, 2026. <https://openreview.net/forum?id=r6Pw3RiMYL>
- [21] Z. Gao, J. Kim, W. Sun, T. Joachims, S. Wang, R. Y. Pang, and L. Tan, "Prompt Curriculum Learning for Efficient LLM Post-Training," *International Conference on Learning Representations (ICLR 2026)*, 2026. <https://openreview.net/forum?id=zqOCacBD3P>
- [22] K. Cobbe, V. Kosaraju, M. Bavarian, et al., "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021. <https://arxiv.org/abs/2110.14168>
- [23] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al., "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," *arXiv preprint arXiv:2503.14476*, 2025. <https://arxiv.org/abs/2503.14476>
- [24] Q. Li, L. Cui, X. Zhao, L. Kong, and W. Bi, "GSM-Plus: A Comprehensive Benchmark for Evaluating the Robustness of LLMs as Mathematical Problem Solvers," *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2961-2984, 2024. <https://doi.org/10.18653/v1/2024.acl-long.163>
- [25] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring Mathematical Problem Solving with the MATH Dataset," *NeurIPS 2021 Datasets and Benchmarks Track*, 2021. <https://openreview.net/forum?id=7Bywt2mQsCe>
- [26] C. Ye, Z. Yu, Z. Zhang, H. Chen, N. Sadagopan, J. Huang, T. Zhang, and A. Beniwal, "Beyond Correctness: Harmonizing Process and Outcome Rewards through RL Training," *MATH-AI: The 5th Workshop on Mathematical Reasoning and AI*, 2025. <https://neurips.cc/virtual/2025/loc/san-diego/131081>
- [27] J. Morrison, C. Na, J. Fernandez, T. Dettmers, E. Strubell, and J. Dodge, "Holistically Evaluating the Environmental Impact of Creating Language Models," *International Conference on Learning Representations (ICLR 2025)*, 2025. <https://openreview.net/forum?id=04qx93Viwi>
- [28] S. Agarwal and M. Chakraborti, "The Hidden AI Race: Tracking Environmental Costs of Innovation," *arXiv preprint arXiv:2511.22781*, 2025. <https://arxiv.org/abs/2511.22781>
- [29] United Nations Department of Economic and Social Affairs, "The Sustainable Development Goals Report 2025," *United Nations*, 2025. <https://unstats.un.org/sdgs/report/2025/The-Sustainable-Development-Goals-Report-2025.pdf>

Abbreviations

AI: Artificial Intelligence

AIME: American Invitational Mathematics Examination

AMC: American Mathematics Competitions

API: Application Programming Interface

ARC: AI2 Reasoning Challenge

BF16: BFloat16

CPU: Central Processing Unit

CRISP-DM: Cross-Industry Standard Process for Data Mining

CSV: Comma-Separated Values

CUDA: Compute Unified Device Architecture

DAPO: Decoupled Clip and Dynamic Sampling Policy Optimization

Dr. GRPO: Group Relative Policy Optimization Done Right

FDP: Final Degree Project

FP16: 16-bit Floating Point

GB: Gigabyte

GPQA: Graduate-Level Google-Proof Q&A

GPU: Graphics Processing Unit

GRPO: Group Relative Policy Optimization

GRPO-LEAD: Group Relative Policy Optimization with Length-Dependent Accuracy Reward, Explicit Penalty, and Difficulty-Aware Advantage Reweighting

GSM8K: Grade School Math 8K

GSM-Plus: Grade School Math Plus

JSON: JavaScript Object Notation

JSONL: JSON Lines

KL: Kullback-Leibler Divergence

KV: Key-Value

LLM: Large Language Model

LoRA: Low-Rank Adaptation

LR: Learning Rate

MBPP: Mostly Basic Python Problems

MATH: Mathematical Problem Solving Dataset

MATH500: 500-problem subset of the MATH benchmark

NF4: NormalFloat 4-bit

OOM: Out of Memory

Open-RS: Open Reasoning in Small LLMs

PEFT: Parameter-Efficient Fine-Tuning

PPO: Proximal Policy Optimization

QLoRA: Quantized Low-Rank Adaptation

RL: Reinforcement Learning

RLHF: Reinforcement Learning from Human Feedback

RLVR: Reinforcement Learning from Verifiable Rewards

SDG: Sustainable Development Goal

SENT: Semantic-Entropy Curriculum Learning

SLM: Small Language Model

STEM: Science, Technology, Engineering, and Mathematics

SVAMP: Simple Variations on Arithmetic Math Word Problems

TINA: Tiny Reasoning Models via LoRA

TRL: Transformer Reinforcement Learning

VRAM: Video Random-Access Memory

W&B: Weights & Biases

Appendix A: Use of AI assistance

This appendix records how I used Codex, Gemini Deep Research and NotebookLM during the Data Science and Artificial Intelligence FDP. They were part of the working process, mainly for experiment organization, analysis and writing revision. They did not replace my judgment, the validation work or my responsibility for the conclusions.

The main form of day-to-day assistance was Codex in the working environment. Gemini Deep Research and NotebookLM were used more narrowly to organize background material, compare notes and identify claims that needed manual checking. I limited all of these uses to tasks that could be checked against independent evidence: experiment configurations, benchmark outputs, logs, checkpoints, papers, official documentation, source code and preserved project artifacts. I did not treat suggested wording as final academic prose. When I used drafting support, I reviewed the text, rewrote it where necessary and checked it before including it in the memory.

No result in the memory was accepted because an AI assistant produced a plausible explanation. Claims about reward behavior, benchmark ranking, collapse, response length, truncation, model quality and statistical interpretation remain tied to the artifacts and cited sources used throughout the memory.

AI tool	FDP stage	Purpose of use	Human verification and boundary
Codex	Experiment planning and command preparation	Organizing candidate runs, preparing commands, checking configuration implications and documenting experiment intent.	I checked commands and configurations against the training code, configuration files, runtime constraints and preserved experiment artifacts.
Codex	Log and benchmark analysis	Reading long logs, benchmark tables and metric summaries, then separating useful evidence from failed or diagnostic runs.	I verified accuracy, reward, truncation, response-length and ranking claims against preserved metrics, benchmark outputs, checkpoints and configuration files.

Codex	Debugging and failure interpretation	Formulating possible explanations for collapse, zero-signal behavior, truncation pressure and dataset-feasibility problems.	I accepted failure interpretations only when they were consistent with logs, metrics, artifact chronology and the experimental setup.
Codex, Gemini Deep Research and NotebookLM	Statistical interpretation and claim boundaries	Organizing result interpretation, identifying overclaiming risks and keeping conclusions tied to the evaluated datasets, response caps and seeds.	I checked the final interpretation against the experimental protocol, benchmark evidence, limitations and cited literature.
Codex, Gemini Deep Research and NotebookLM	Technical writing and revision	Reorganizing prose, reducing repetition and keeping explanations focused on the Data Science perspective.	I reviewed and rewrote assisted wording. I did not use the assistant as a source for factual claims.
Codex, Gemini Deep Research and NotebookLM	Consistency and source-check flags	Checking terminology, chapter overlap, repeated claims and result interpretations that needed manual verification.	I checked the actual sources and evidence myself and prepared the bibliography separately. The assistant was not used to write bibliography entries or as authority for citations.

Table A.1: AI assistance used in the Data Science FDP.

The final responsibility for the memory remains mine. The assistant helped me inspect and explain the experimental record, but it did not determine whether a run was valid, whether a model improved, whether a limitation was acceptable or whether a conclusion followed from the evidence.

Appendix B: Repository and experiment tracking links

This appendix gives the external access points for the implementation repository and the experiment tracking workspace used during this FDP.

Resource	Link	Role in the FDP
GitHub repository	https://github.com/eblancocabana/grpo-training-engine	Public source repository for the GRPO training engine, including the implementation, benchmark utilities, configuration entry points and project.
Weights & Biases project	https://wandb.ai/e-blanco-universidad-de-deusto/grpo-training	Experiment tracking workspace for training runs, metric curves and run-level metadata associated with the project.

Table B.1: Public repository and experiment tracking links.

Weights & Biases was used for online run tracking, apart from the local artifacts preserved during the project.